

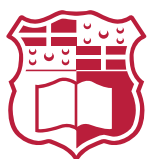
Explainability of Reinforcement Learning Agents Within Simulated Environments

Luke Camilleri

Supervisor: Dr. Vanessa Camilleri

June 2025

*Submitted in partial fulfilment of the requirements
for the degree of Bachelor of Science in Information Tech (Hons) (Artificial
Intelligence.*



L-Università ta' Malta

**Faculty of Information &
Communication Technology**

Abstract

This dissertation explores the use of model-agnostic Explainable Artificial Intelligence (XAI) techniques for interpreting the behaviour of Reinforcement Learning (RL) agents trained in Unity-based environments. As RL systems grow in complexity and are applied in critical domains, the need for interpretable decision-making becomes increasingly important. Traditional Deep Reinforcement Learning (DRL) policies often lack transparency, creating challenges for both developers and end-users in understanding agent behaviour and ensuring reliability. Improving explainability in this context is essential not only for debugging and validation but also for enhancing trust in autonomous systems.

The study involved training agents across three distinct Unity environments using the ML-Agents toolkit, followed by the application of explainability techniques including surrogate models, SHAP feature attribution, and saliency mapping. Observation-action data was collected during inference and analysed using Python-based tooling. Custom logic was used to preprocess inputs, train interpretable models, and extract feature attributions aligned with the agents' input structures. These techniques were tailored to the characteristics of each environment, accounting for differences in action space dimensionality and sensor design. Selected results were integrated into Unity through a seeded post-hoc visualisation interface to enable real-time inspection of decision-making processes using in-engine UI elements.

Results demonstrate that semantic explanations such as SHAP can meaningfully highlight key features driving agent behaviour, and that Unity can serve as a viable platform for embedding visual explanations. Surrogate models offered reliable approximations of discrete agent policies, though they struggled with continuous control due to observation complexity and the noisier nature of regression targets. Decision tree models provided interpretable symbolic representations but were constrained by the high dimensionality of the agent observations. While real-time explanation remains computationally infeasible, the framework developed in this project lays the groundwork for scalable, interpretable reinforcement learning pipelines. These contributions aim to bridge the gap between algorithmic transparency and user-facing explainability within high-fidelity simulation environments.

Acknowledgements

I would like to express my deepest gratitude and sincere appreciation to the many individuals who have supported me throughout my academic journey.

First and foremost, I would like to thank my project supervisor, Dr. Vanessa Camilleri. Not only did she provide the foundation for this project by proposing its title, but she also offered consistent guidance and support, always helping steer me toward my goals.

I am also incredibly grateful to my family and those closest to me, whose unwavering love and encouragement have sustained me through every stage of this journey. Their belief in me was a constant source of motivation and strength I could rely on.

To my close group of friends made throughout this course, thank you for being the ones I could rely on—for motivation, shared stress, absurd ideas, and laughter. Your presence made the hardest days bearable and turned this journey into something far more than just academic. Our countless discussions not only sparked countless insights and ideas, but also pushed me to stay focused and keep aiming higher. The unforgettable memories we've created together over the years are something I will carry with me always, and I will continue to cherish them deeply.

Finally, among these friends, I wish to specifically honour the memory of our dear friend Oleg, who tragically passed away during our first year. His energy, humour, and presence were deeply cherished by all of us. He is sorely missed in every moment we share, and may his memory live on through me and all our friends.

Contents

Abstract	i
Acknowledgements	ii
Contents	v
List of Figures	vii
List of Tables	viii
List of Abbreviations	ix
Glossary of Symbols	1
1 Introduction	1
1.1 Problem Definition	1
1.2 Motivation	2
1.3 Aims and Objectives	2
1.4 Proposed Solution	2
1.5 Document Structure	3
2 Background and Literature Review	4
2.1 Background	4
2.1.1 Unity Game Engine	4
2.1.2 Unity ML-Agents Toolkit	5
2.1.3 Reinforcement Learning	6
2.1.4 Explainability	7
2.1.5 Explainable Reinforcement Learning	8
2.2 Literature Review	9
2.2.1 Model-Agnostic Post-Hoc Explainability in Reinforcement Learning (RL)	9
2.2.2 Surrogate and Causal Models for Policy Interpretation	11
2.2.3 Visualisation and Real-Time Feedback Systems	12
2.2.4 Simulation-Based Explainability: Unity as a Platform	13

2.2.5	Human-Centred Delivery and Evaluation of Explanations	14
2.2.6	Conclusion and Research Gap	15
2.3	Summary	16
3	Methodology	17
3.1	Designing and Implementing the Unity Training Environments	17
3.1.1	Environment 1: Pyramids	17
3.1.2	Environment 2: Dungeon Escape	19
3.1.3	Environment 3: Food Collector	21
3.2	Training Agents using the ML-Agents Toolkit	23
3.3	Logging Agent Data	24
3.4	Applying Explainability Methods Using Python	24
3.4.1	Surrogate Model Training and Interpretation	25
3.4.2	Semantic SHapley Additive exPlanations (SHAP) Explanations	25
3.4.3	Step-wise Semantic SHAP Explanations	26
3.4.4	Grid-Based Saliency Mapping	26
3.5	Real-Time Unity Explanations	27
4	Results and Evaluation	28
4.1	Environment Performance and Feasibility	28
4.2	Agent Training	29
4.2.1	Pyramids Environment	29
4.2.2	Dungeon Escape Environment	30
4.2.3	Food Collector Environment	31
4.3	Explainability Results and Insights	31
4.3.1	Surrogate Modelling and Decision Tree Explanations	32
4.3.2	Overall and Step-Wise SHAP Explanations	34
4.3.3	Grid-Based Saliency Mapping	36
4.4	Real-Time Unity Visualisation	37
4.5	Chapter Summary	38
5	Conclusion	39
5.1	Revisiting the Aims and Objectives	39
5.2	Project Criticisms	39
5.3	Future Work	40
A	Agent Training	47
A.1	Pyramids Hyperparameters	47
A.2	Dungeon Escape Hyperparameters	48
A.3	Food Collector Hyperparameters	50

A.4	Environment Descriptions from Unity Documentation	53
A.4.1	Pyramids	53
A.4.2	Dungeon Escape	54
A.4.3	Food Collector	55
A.5	In-Editor Agent Parameters	57
A.5.1	Pyramids	57
A.5.2	Dungeon Escape	58
A.5.3	Food Collector	58
A.6	Training Results	59
A.6.1	Pyramids	59
A.6.2	Dungeon Escape	61
A.6.3	Food Collector	63
B	Text-Based Decision Tree Explanation	65
B.1	Discrete Actions Explanation	65
B.2	Continuous Actions Explanation	67
C	SHAP Calculation Figures	75
C.1	Pyramids	75
C.2	Dungeon Escape	78
C.3	Food Collector	83
D	Food Collector Saliency Grid Results	84
E	Dungeon Environment Real-Time SHAP Visualisation	85
F	Code Availability	86

List of Figures

Figure 2.1	Unity Interface	4
Figure 2.2	Interaction loop between the agent and the environment [14]	6
Figure 2.3	Local Interpretable Model-agnostic Explanations (LIME) explanation visualisation based on COMPAS dataset [30]	9
Figure 3.1	A DFD Overview of the Solution Implemented	17
Figure 3.2	Agent approaching the green brick goal after switch activation [47]. .	18
Figure 3.3	Initial state of the Dungeon Escape environment.	20
Figure 3.4	Agents navigating the Food Collector environment to collect green food and avoid red poison [47].	22
Figure 4.1	Training metrics for the Pyramids environment.	29
Figure 4.2	Training metrics for the Dungeon Escape environment.	30
Figure 4.3	Training metrics for the Food Collector environment.	31
Figure 4.4	Confusion matrices for surrogate classifiers across all environments. .	33
Figure 4.5	Food Collector Decision Tree	33
Figure 4.6	Example of IF–THEN Rules	34
Figure 4.7	Semantic SHAP importance for the two most frequently chosen ac- tions in the Pyramids environment.	34
Figure 4.8	Semantic SHAP comparisons across all three agents in the Dungeon Escape environment.	35
Figure 4.9	Semantic SHAP importance for one continuous action and one dis- crete action for the Food Collector environment	35
Figure 4.10	Step-Wise SHAP for Agent 2 Within Dungeon Escape	36
Figure 4.12	Semantic saliency comparisons for a single input feature across all ac- tions in the Food Collector environment.	37
Figure 4.13	Real-Time Pyramids SHAP visualisation. The agent is seen prioritising the switch once it is in view.	38
Figure A.1	The Pyramids environment.	53
Figure A.2	The Dungeon Escape environment.	54
Figure A.3	The Food Collector environment.	55

Figure A.4	Configuration screenshots for the Pyramids agent and its three stacked ray perception sensors.	57
Figure A.5	Configuration screenshots for the Dungeon Escape agent and its ray perception sensor.	58
Figure A.6	Configuration screenshots for the Food Collector agent and its Grid sensor.	58
Figure A.7	Training metrics for the Pyramids environment (1/2).	59
Figure A.7	Training metrics for the Pyramids environment (2/2).	60
Figure A.8	Training metrics for the Dungeon Escape environment (1/2).	61
Figure A.8	Training metrics for the Dungeon Escape environment (2/2).	62
Figure A.9	Training metrics for the Food Collector environment (1/2).	63
Figure A.9	Training metrics for the Food Collector environment (2/2).	64
Figure C.1	Smoothed Semantic SHAP Values over Time for Pyramids Environment	75
Figure C.2	Semantic SHAP Importance for Action 0 (Pyramids)	75
Figure C.3	Semantic SHAP Importance for Action 1 (Pyramids)	76
Figure C.4	Semantic SHAP Importance for Action 2 (Pyramids)	76
Figure C.5	Semantic SHAP Importance for Action 3 (Pyramids)	77
Figure C.6	Semantic SHAP Importance for Action 4 (Pyramids)	77
Figure C.7	SHAP comparison between agents - Action 0 (Dungeon)	78
Figure C.8	SHAP comparison between agents - Action 1 (Dungeon)	78
Figure C.9	SHAP comparison between agents - Action 2 (Dungeon)	79
Figure C.10	SHAP comparison between agents - Action 3 (Dungeon)	79
Figure C.11	SHAP comparison between agents - Action 4 (Dungeon)	80
Figure C.12	SHAP comparison between agents - Action 5 (Dungeon)	80
Figure C.13	SHAP comparison between agents - Action 6 (Dungeon)	81
Figure C.14	Smoothed SHAP values for Dungeon Agent 0	81
Figure C.15	Smoothed SHAP values for Dungeon Agent 1	82
Figure C.16	Smoothed SHAP values for Dungeon Agent 2	82
Figure C.17	Semantic SHAP Importance Across Actions in the Food Environment .	83
Figure D.1	Continuous Action Saliency Maps	84
Figure D.2	Discrete Action Saliency Maps	84
Figure E.1	Dungeon Escape SHAP Visualisation	85

List of Tables

Table 3.1	Revised group reward structure for the Dungeon Escape environment .	21
Table 3.2	Explainability techniques applied across environments	24
Table 4.1	Discrete Surrogate Model Performance Across Environments	32
Table 4.2	Continuous Surrogate Regression Metrics for Food Collector	32

List of Abbreviations

AI Artificial intelligence.

DQN Deep Q-Network.

DRL Deep Reinforcement Learning.

FI Feature Importance.

LIME Local Interpretable Model-agnostic Explanations.

MA-POCA Multi-Agent Posthumous Credit Assignment.

MDP Markov Decision Process.

ML Machine Learning.

PPO Proximal Policy Optimisation.

RL Reinforcement Learning.

SHAP SHapley Additive exPlanations.

XAI Explainable Artificial Intelligence.

XDRL Explainable Deep Reinforcement Learning.

XRL Explainable Reinforcement Learning.

1 Introduction

Artificial Intelligence (AI) has become increasingly prevalent in a wide range of daily applications, often taking on tasks that are too complex or time-consuming for humans to perform efficiently [1]. Its influence can be seen everywhere, from smartphone voice assistants to automated production pipelines and self-driving vehicles. As AI systems advance into more sensitive and critical fields, concerns arise regarding their safety and reliability. In response, explainability, and the broader field of Explainable Artificial Intelligence (XAI) play a vital role in fostering trust, acceptance, and a deeper understanding of these models and agents [2]. However, within the field of RL, this concept goes relatively unexplored. Tools [3] which allow the simulation of a RL agent to take actions and explore within its environment provide the perfect area in which to analyse how effective these techniques may actually be.

This dissertation focuses on this concept of taking simulated agents, applying current XAI practices onto them and observing the levels of explainability which are able to be achieved.

1.1 Problem Definition

RL has seen widespread adoption across various domains, including robotics, finance, healthcare, and natural language processing, due to its ability to learn optimal policies through interaction with complex environments [4]. Despite these advancements, RL models often function as opaque black boxes, making it challenging to interpret their decision-making processes [5].

The inherent complexity and high dimensionality of RL systems contribute to their lack of transparency. Agents process vast amounts of data and make sequential decisions, where the impact of an action may only become apparent after several time steps, complicating the attribution of specific outcomes to individual actions or observations [6].

Current XAI techniques, such as SHAP and LIME, have been primarily developed for supervised learning models and often fall short when applied to RL agents. These methods may not adequately capture the temporal dependencies and dynamic nature of RL environments, leading to explanations that are difficult to interpret in the context of agent behaviour [7].

Evaluating the effectiveness of explainability techniques in RL is further complicated by the real-time nature of these systems. Testing and validating interpretability methods require controlled environments where agents can be observed and analysed systematically. Simulation environments provide a platform for

such evaluations, enabling researchers to assess explainability methods in a reproducible and scalable manner [8].

1.2 Motivation

When RL is applied to real-world scenarios — particularly in high-stakes domains like healthcare or autonomous systems — understanding why an agent made a decision becomes essential for debugging unexpected behaviour and ensuring system reliability. Unlike traditional models, RL agents learn through trial and error in dynamic environments, where the consequences of actions may only become apparent after several steps. This delayed feedback, combined with high-dimensional observations, makes it difficult to isolate the root cause of misbehaviour. Explainability techniques help bridge this gap by highlighting which inputs or features influenced an agent's decisions, enabling faster, more targeted diagnostics and increasing trust in deployed systems.

1.3 Aims and Objectives

This project aims to investigate whether current model-agnostic XAI techniques are effective when applied to RL agents, and whether the Unity game engine can serve as a viable platform for the meaningful visualisation of agent decision-making. To support this aim, the following objectives have been identified:

- O1** Design and implement multiple Unity environments suitable for training agents using the ML-Agents toolkit.
- O2** Train agents in these environments and log observation-action data by running inference on exported policy models.
- O3** Identify and implement appropriate model-agnostic XAI techniques in Python, based on the observation structure and agent design of each environment.
- O4** Integrate and evaluate these techniques within Unity to determine their feasibility for real-time, in-engine visualisation.

1.4 Proposed Solution

To address the aims and objectives outlined above, the project adopts a staged pipeline that combines Unity-based simulation, reinforcement learning model training, external explainability analysis, and in-engine visualisation.

For **O1**, multiple environments from the Unity ML-Agents toolkit [3] are selected and adapted to align with the needs of explainability research. This includes modifying reward structures, agent behaviours, and built-in ML-Agents mechanisms to support detailed observation and control.

In support of **O2**, agents are trained via the ML-Agents command-line interface, allowing for controlled experimentation using configurable training parameters. After training, the resulting policy models are used to run deterministic inference episodes, during which observation-action pairs are recorded externally for analysis.

To fulfil **O3**, current model-agnostic XAI techniques are applied using Python. These include SHAP-based semantic explanations and surrogate models such as decision trees, tailored to each environment's observation structure.

Finally, **O4** involves integrating successful techniques back into Unity using C#. This enables real-time, step-wise visualisation of the agent's reasoning process, including overlays for SHAP bar charts and semantic heatmaps.

1.5 Document Structure

The remainder of this document is organised as follows:

- **Chapter 2 – Background and Literature Review:** Covers reinforcement learning concepts, the need for explainability, and relevant methods like SHAP and policy approximation. It also reviews prior work in simulated environments and highlights research gaps.
- **Chapter 3 – Methodology:** Details the Unity-based training environments, the RL agent pipeline, and the integration of explainability techniques for various observation types.
- **Chapter 4 – Evaluation and Results:** Presents results from applying SHAP and tree-based techniques to trained agents, assessing how effectively visualisations reflect policy and feature importance.
- **Chapter 5 – Conclusion and Future Work:** Summarises findings, discusses limitations, and proposes directions for further development.

2 Background and Literature Review

In this chapter, a comprehensive overview of the technologies and concepts relevant to this project is presented, along with a critical review of existing literature and prior research in the field.

2.1 Background

Before elaborating on explainability and RL concepts, the tools being used at the centre of this project must first be explained, this being the Unity game engine and its corresponding ML-Agents toolkit.

2.1.1 Unity Game Engine

The Unity Game Engine is considered a top-tier cross-platform game engine used to "develop 2D and 3D video games, non-game simulations for computers, virtual reality, consoles, and mobile device platforms" [9]. Unity is one of the more popular game engines due to its focus on accessibility, allowing the leveraging of its in-built tools for rapid iteration and prototyping [10].

Below is an example of the interface a user may see when using Unity:

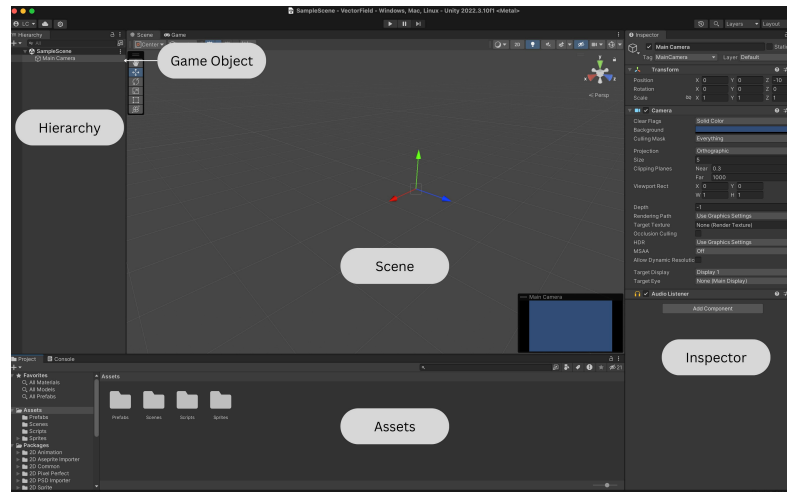


Figure 2.1 Example Unity Interface.

Within Figure 2.1, five key sub-windows constitute the Unity editor: the Hierarchy, Scene View, Game View, Inspector, and Assets. The **Hierarchy** provides a structured list of all Game Objects present in the current scene, enabling users to organise, rearrange, and manage objects through drag-and-drop interactions. The editor also offers two distinct visual interfaces: the **Scene View** and the **Game View**.

The Scene View functions as the primary workspace where developers construct and edit the game environment. It allows for unrestricted camera movement, offering precision and flexibility when positioning elements. In contrast, the Game View simulates the runtime experience by displaying the scene through the active in-game cameras, enabling developers to preview how the game will appear to the player without the need to compile or export the project.

The **Inspector** panel is the central interface for viewing and modifying the properties of individual Game Objects. It allows developers to adjust parameters to fine-tune behaviour and design. All attached components—such as scripts, physics configurations, colliders, and audio elements—are displayed and configurable within this panel. Moreover, it enables the assignment and modification of variables exposed by user-defined scripts. Lastly, the **Assets** window functions as a built-in file explorer, allowing developers to import, organise, and manage project files directly within the editor.

The game engine allows the use of three programming languages, a JavaScript-like language called UnityScript, C# and Boo [11]. However, for the purpose of this dissertation, all work done within the engine was carried out using C#.

2.1.2 Unity ML-Agents Toolkit

The Unity ML-Agents Toolkit is an open-source framework that allows games and simulations built in Unity to be used as interactive environments for training intelligent agents. It supports a range of machine learning approaches, including reinforcement learning, imitation learning, and neuroevolution, which can be implemented through an accessible Python API [12].

Unity's ML-Agents Toolkit revolves around three key elements: **Agents**, **Sensors**, and the **Academy**. The Agent is a component attached to a Game Object, enabling it to perceive the environment, decide on actions, and receive feedback in the form of rewards. Observations are made possible through a range of sensors, including raycasts, camera inputs, or custom vectors, depending on the needs of the environment [13].

Each Agent relies on a policy to determine its behaviour. Policies are tied to behaviour names, allowing multiple agents to share the same decision logic and pool experiences during training. A single Unity scene can host multiple policies, enabling complex multi-agent interactions where each group or individual follows a unique behavioural strategy.

Policies themselves can be driven by various decision-making mechanisms: hard-coded logic, user input, embedded neural network models, or commands issued via the Python API. Developers can configure how often an Agent requests a decision,

either on a fixed schedule or dynamically, based on environmental context.

The reward system, crucial for reinforcement learning, is defined via Unity scripts. These scripts can assign or adjust rewards at any point in the simulation, guiding the Agent's learning process. Additionally, both Agents and environments can be terminated programmatically or when a specified step count is reached.

At the simulation's core lies the Academy, a singleton that orchestrates the global training process. It manages simulation steps and oversees environment configuration. Through the use of environment parameters, developers can modify runtime properties—such as physics settings, object sizes, textures, or object existence—offering dynamic variability during training to increase agent generalisability [3, 12, 13].

2.1.3 Reinforcement Learning

RL is a branch of Machine Learning where agents learn to take actions that maximise long-term rewards by interacting with an environment [14]. Unlike supervised learning, RL does not require labelled datasets and instead learns through trial-and-error, guided by feedback signals.

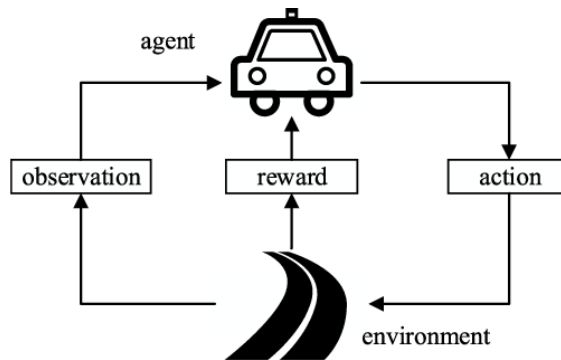


Figure 2.2 Interaction loop between the agent and the environment [14]

At each step, the agent observes the environment, selects an action based on its policy, and receives a reward. The environment then transitions to a new state, and the agent updates its policy accordingly. This interaction is typically formalised using a Markov Decision Process (MDP), defined by the tuple (S, A, P, R, γ) [15], where S is the state space, A is the action space, P represents transition probabilities, R is the reward function, and γ is the discount factor [16].

Value and Policy-Based Methods

Value functions estimate how beneficial it is for an agent to be in a particular state $(V(s))$ or take a specific action in that state $(Q(s, a))$ [17]. These estimates guide the

agent's decision-making relative to future rewards.

A core challenge in RL is the exploration-exploitation trade-off: agents must explore new actions to discover better outcomes while exploiting known strategies to gain rewards [18]. Strategies such as epsilon-greedy, where an agent selects a random action with a small probability ϵ to explore while exploiting the best-known action otherwise, help navigate this trade-off effectively.

Value-based approaches use value functions to select actions, whereas policy-based methods directly learn a policy mapping states to action probabilities. Policy gradient methods such as REINFORCE adjust parameters to maximise expected returns [19]. Actor-critic methods combine both strategies, enabling more stable and efficient learning [20].

Deep Reinforcement Learning

Deep Reinforcement Learning (DRL) extends traditional RL by using deep neural networks to approximate policies, value functions, or both [21]. This enables agents to operate in high-dimensional environments where tabular approaches are not feasible.

Techniques like Deep Q-Networks (DQNs) and Proximal Policy Optimisation (PPO) have shown success in areas such as games and robotics [22]. However, Deep RL introduces challenges including sample inefficiency, sensitivity to hyperparameters, and training instability [23].

2.1.4 Explainability

Explainability is a subfield of machine learning dedicated to understanding and interpreting the internal workings, logic, and decision-making processes of machine learning systems. A model is considered more explainable when its training and decision-making processes can be more easily understood and interpreted by humans, commonly referred to as XAI [2].

The terms explainability and interpretability are often used interchangeably in machine learning research. However, while they are closely related, key distinctions exist between the two. Notably, there is no universally accepted mathematical, formal, or technical definition for either term, nor are there standardised metrics for measuring them [24]. The primary difference lies in their focus: explainability seeks to provide insights into the internal workings of a model, whereas interpretability emphasises understanding the relationship between a model's inputs and outputs [25].

Intrinsic vs Post-Hoc Explainability

There are usually two approaches towards integrating explainability into a Machine Learning (ML) model: intrinsic and post-hoc.

A model is considered intrinsically explainable if it is designed to be inherently interpretable. Examples include regression models and decision trees, where the computational structure ensures transparency in decision-making. However, this type of architecture can limit computational capacity, potentially reducing prediction accuracy.

Post-hoc XAI tools are designed to mitigate the "black box" nature of machine learning models by providing interpretability for their predictions. When using model-agnostic methods, two types of explanations can be generated. Global explanations offer an overview of the model's overall behaviour and performance, while local explanations focus on providing detailed insights into individual predictions [26].

One of the most common post-hoc approaches involves evaluating the importance of individual features (FI) in determining a given outcome [27]. A prominent method for this is SHapley Additive exPlanations, derived from game theory, which assigns contribution scores to each feature. SHAP calculates these scores by analysing the average impact of a feature on the model's predictions across all possible combinations of features. This ensures a fair allocation of importance values, accounting for interactions and dependencies between features to provide a comprehensive understanding of their influence on specific predictions [28].

One other widely accepted model-agnostic post-hoc explanation method is Local Interpretable Model-agnostic Explanations (LIME). LIME provides insights by identifying and visualising the most important features influencing a specific prediction. Each identified feature is assigned a coefficient, indicating the direction of its influence (positive or negative) on the predicted outcome, which can be seen in Figure 2.3 below [29]. The magnitude of the coefficient reflects the strength of the feature's contribution to the model's decision. LIME's explanations are generated using local surrogate models, ensuring interpretability without access to the underlying structure.

2.1.5 Explainable Reinforcement Learning

Just as Explainable Artificial Intelligence is a subfield of artificial intelligence, Explainable Reinforcement Learning (XRL) and Explainable Deep Reinforcement Learning (XDRL) are specialised branches of XAI that focus on developing transparent and interpretable techniques specifically for real-time decision-making processes of RL agents.

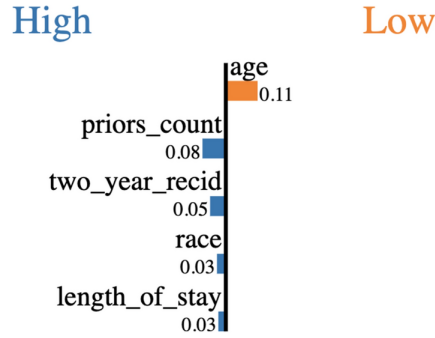


Figure 2.3 LIME explanation visualisation based on COMPAS dataset [30]

As discussed in Section 5 of [31], achieving a comprehensive understanding of an agent’s decision-making process in XRL introduces challenges beyond those addressed by XAI. These include analysing both the short- and long-term consequences of an agent’s behaviour, rather than focusing solely on the immediate reasoning behind a specific action. Additionally, it involves interpreting the agent’s learning objectives based on the reward structure of the environment.

Many techniques within XRL research extend from those currently existing within XAI research. One such technique is that of Feature Importance (FI). Some FI techniques can be used with intrinsically interpretable policies, while others create an interpretable policy by a post-hoc conversion of one which is non-interpretable. Other methods aim to generate a direct explanation for each action taken by an agent rather than learning an interpretable policy, taking the form of the environment state, natural language or even a saliency map [5].

2.2 Literature Review

While a large body of work exists on explainability in supervised learning, XRL is still maturing, particularly in dynamic, simulated environments such as Unity. This literature review surveys work across five themes: model-agnostic post-hoc explainability in RL, surrogate and causal models, visual and real-time feedback, simulation-based explainability (with a focus on Unity), and human-centred delivery of explanations.

2.2.1 Model-Agnostic Post-Hoc Explainability in RL

Model-agnostic post-hoc explanation techniques have gained significant attention in RL due to their applicability across various architectures. These methods operate by querying a trained model as a black box, providing insight into decisions without requiring access to the underlying structure. Two of the most widely adopted approaches are SHAP and LIME, originally developed for supervised learning but is now

widely explored in DRL contexts [28, 29]. While both approaches have been applied to DRL agents for visual and state-based policies, significant limitations have emerged.

A key challenge is that both methods assume static input–output mappings, which fail to reflect the temporal dynamics of RL. Hickling et al. [7] argue that these explanations are insufficient for policy-level understanding, as they cannot capture dependencies across trajectories or delayed rewards. Wells and Bednarz [2] further demonstrate that input perturbations in RL often yield semantically invalid states, especially in continuous control environments, undermining the reliability of LIME and SHAP outputs.

Moreover, Sequeira and Gervasio [32] highlight that explanations that are limited to single time steps fail to capture the strategic nuances of an agent’s behaviour over an episode. They advocate for summarisation techniques that highlight decision patterns over time, which are more aligned with human evaluators’ needs in simulated environments.

Evaluation of explanation effectiveness also remains underdeveloped. Vilone and Longo [33] critique the lack of standardised metrics for assessing post-hoc methods in RL, calling into question the generalisability of results based purely on visual inspection or sparsely defined accuracy measures. Ali et al. [1] echo this concern, pointing out that human-centred evaluations are rare and often neglect usability under time-sensitive or interactive conditions.

In practical deployments, trade-offs between interpretability and computational cost must be considered. Wells and Bednarz [2] compare SHAP, LIME, and Learning Model Trees (LMTs), concluding that while SHAP offers superior fidelity, its runtime complexity limits use in real-time settings. LMTs, while less granular, provide interpretable decision structures suitable for deployment within latency-constrained environments.

In response to these limitations, hybrid approaches have emerged. Hayes and Shah [34] propose combining visual representations with abstracted graph-based summaries to increase interpretive value. Although not model-agnostic in a strict sense, such integrations indicate a pathway toward augmenting post-hoc tools with trajectory-awareness or higher-order structure.

While SHAP and LIME remain foundational in model-agnostic interpretability, their direct application to DRL presents considerable challenges. The lack of temporal abstraction, vulnerability to invalid state perturbations, and computational overhead all restrict their utility. Consequently, these tools are best viewed as components within broader explanatory frameworks—ideally, ones that incorporate policy summarisation, trajectory analysis, or user-centred design.

2.2.2 Surrogate and Causal Models for Policy Interpretation

As an alternative to direct post-hoc attribution methods like SHAP and LIME, surrogate and causal models provide structured, interpretable approximations of agent behaviour in RL. These models aim to reconstruct the agent’s policy using inherently transparent systems, such as decision trees or symbolic rules, allowing for faithful approximations that can be analysed and debugged independently of the black-box model.

One prominent example is the work of Hachaj and Piekarczyk [35], who developed a hybrid explainability framework combining Grad-CAM saliency maps with semantic state abstraction. They aggregated visual features into 3×3 semantic grids and trained Classification and Regression Trees (CARTs) to approximate the agent’s policy. Their method demonstrated over 94% fidelity across 12 experimental configurations, validating the utility of surrogate models in environments with discrete, spatially grounded inputs. However, their approach was limited to low-resolution Unity scenes and required significant offline processing, which hinders real-time integration.

Causal explanation frameworks offer another avenue for interpretable RL. Madumal et al. [36] introduced a structural causal model to capture action influence in model-free RL agents. Their method supports contrastive explanations by identifying which variables causally influenced the agent’s decision. Unlike feature attribution tools, causal models allow for the generation of counterfactuals—answering questions such as “Why action A instead of B?”—which are essential for evaluating agent rationality in multi-step scenarios. Through human studies, the authors demonstrated that causal explanations increased both user satisfaction and trust compared to saliency-based or correlation-based baselines.

Despite their interpretability, surrogate and causal models are not without limitations. As highlighted by Verma et al. [37], symbolic approximations such as decision lists or decision trees may struggle with fidelity when agents operate in highly nonlinear or continuous domains. Additionally, constructing accurate causal models requires domain knowledge or strong assumptions about the environment’s structural dependencies, which may not always be available in RL applications.

To mitigate these issues, hybrid techniques have emerged. Juozapaitis et al. [38] proposed reward decomposition as a method to disentangle and explain the contributions of various reward sources in multi-objective RL. Their framework enables a more granular understanding of agent incentives, which can complement policy-level surrogate models or causal structures.

Despite their strengths, both surrogate and causal approaches face practical challenges. Hachaj and Piekarczyk [35] note that while decision trees trained on semantic abstractions provide high-fidelity approximations, their approach is limited by environment complexity and lacks real-time applicability due to the offline nature of

Grad-CAM processing. Madumal et al. [36] acknowledge that constructing structural causal models requires assumptions about variable dependencies, which may not be straightforward in all RL domains. Verma et al. [37] also report fidelity issues when using symbolic approximations like decision lists, particularly in high-dimensional state-action spaces. Juozapaitis et al. [38] propose reward decomposition as an alternative strategy, but their framework still depends on access to internal reward signals and does not generalise across different policy types. These findings suggest that while surrogate and causal models offer interpretability, their deployment remains sensitive to the structure of the environment, the form of the policy, and the computational constraints of the task.

2.2.3 Visualisation and Real-Time Feedback Systems

As Reinforcement Learning (RL) applications move toward real-time, safety-critical domains, the need for immediate, interpretable feedback becomes increasingly urgent. Visualisation-based explainability techniques provide a pathway for understanding agent behaviour, particularly when agents operate in simulated or physically embodied environments. These methods aim not only to interpret policies but also to render those interpretations in forms that are accessible to human observers—often during or immediately after an agent’s decision-making process.

Itaya et al. [39] introduce an Action Query Transformer (AQT) architecture capable of producing saliency-based visual explanations across all available actions in a decision space. Their system augments traditional saliency methods by generating heatmaps not only for the action taken but also for the alternatives, enabling richer contrastive explanations. Uniquely, the authors deploy these visualisations through augmented reality (AR), projecting saliency overlays directly onto a physical scene. Although not implemented in Unity, the work demonstrates a model-agnostic explanation system that operates in real-time and is spatially grounded, features highly desirable in simulation-based RL environments.

In a different context, He et al. [40] combine visual and textual explanation techniques to interpret the decisions of a UAV trained using DRL. Their method involves feature attribution via gradient-based saliency maps, complemented by human-readable summaries of decision factors. Evaluated both in simulation and in real-world flight scenarios, the approach illustrates how visual explanation can enhance human trust and agent transparency in autonomous systems.

Retrospective summarisation of behaviour also plays an important role in RL visualisation. Sequeira and Gervasio [32] present a framework for constructing highlight reels of agent performance based on measures of novelty, failure, and goal success. While not a real-time system, their work represents a valuable bridge

between behavioural logging and user-centric interpretability. By identifying moments of “interestingness,” the system facilitates human analysis of agent capabilities and limitations across entire episodes.

Hayes and Shah [34] focus on improving transparency in robotic controllers by combining policy summaries with graph-based visual structures. Although their system is applied to predefined controllers rather than DRL agents, it demonstrates how interactive visual feedback, particularly when coupled with symbolic abstraction, can support real-time policy interrogation by human users.

Although visual feedback systems offer intuitive benefits for interpreting agent behaviour, several implementation constraints persist. Itaya et al. [39] acknowledge that their AR-based visualisations, while effective in a lab setting, are limited by hardware requirements and camera calibration needs. He et al. [40] also note that real-time visual attribution in UAV navigation imposes significant computational demands, which can limit deployment in latency-sensitive tasks. Cruz et al. [41] report that while their goal-driven natural language explanations improved user satisfaction, generating real-time updates under dynamic goal conditions remains challenging, especially when explanations must align with shifting policy strategies. Meanwhile, Wang et al. [42] propose a model-agnostic RL framework for sentence-level feedback, but their evaluation highlights variability in explanation usefulness across user types, suggesting that visual or textual outputs alone may not suit all applications equally. These findings suggest that while real-time and interactive explanation systems hold promise, their design must account for user context, hardware constraints, and the temporal complexity of the environment itself.

2.2.4 Simulation-Based Explainability: Unity as a Platform

Simulation environments have become essential testbeds for XRL, offering controllable, repeatable conditions in which agent behaviour can be systematically evaluated. Among these platforms, Unity has gained prominence due to its flexible rendering engine, support for 3D physics, and compatibility with reinforcement learning frameworks through the ML-Agents Toolkit [12]. These features make Unity particularly attractive for research requiring both high-fidelity interaction and real-time feedback visualisation.

The ML-Agents Toolkit, introduced by Juliani et al. [3], allows agents to be trained in Unity environments using a Python API connected to PPO, SAC, or custom learning algorithms. Agents can receive vector, visual, and raycast observations and act continuously or discretely in highly customisable settings. However, while ML-Agents provides a robust base for agent training and observation, its support for interpretability is limited. Debugging tools are mostly focused on raw data export and

observation logging, leaving real-time interpretability or visual explanation as a task left to researchers.

Hachaj and Piekarczyk [35] present one of the few known attempts to embed explainability within Unity itself. Their framework combines Grad-CAM-based saliency extraction with semantic state abstraction to train decision tree surrogates approximating PPO agents. The resulting decision trees are displayed within the Unity environment using custom visual overlays. While effective at achieving high fidelity (up to 96.8%), the system requires significant offline processing, limiting its use for live feedback or user interaction.

Kibe et al. [8] investigate the application of explainability methods in real-time data stream analytics, where RL agents interact with dynamic, high-velocity inputs. While not explicitly focused on Unity, their architecture aligns with Unity’s design constraints and highlights challenges in deploying explainability under strict timing constraints. They emphasise the trade-off between model complexity and explainability, particularly in environments where explanations must be generated per step and visualised synchronously.

Despite its strengths, Unity has not yet seen widespread adoption as a native visualisation layer for model explanations. The existing tooling largely supports training and evaluation rather than embedded post-hoc interpretability. Most explanation methods in Unity-based RL experiments continue to rely on offline export of trajectories, saliency maps, or surrogate outputs for separate visualisation.

This gap presents a notable opportunity for future research. By embedding model-agnostic explainability tools such as SHAP visualisers or decision tree overlays directly into Unity’s runtime, researchers could evaluate explanations synchronously with agent behaviour. Such integration would not only facilitate debugging and policy analysis but could also support human-in-the-loop feedback, especially in environments that require safety guarantees or transparent decision pipelines.

2.2.5 Human-Centred Delivery and Evaluation of Explanations

Human-centred explainability in RL emphasises the importance of tailoring explanations to the needs and contexts of end-users, particularly non-experts. Recent studies have explored various methods to enhance the interpretability and usability of RL systems through natural language explanations, interactive interfaces, and user feedback mechanisms.

Peng et al. [43] introduced the Hierarchically Explainable Reinforcement Learning (HEX-RL) agent, which operates in text-based environments and provides real-time natural language explanations of its actions. By integrating a symbolic knowledge graph with a hierarchical attention mechanism, HEX-RL offers both

immediate and post-hoc explanations. Human evaluations indicated that participants unfamiliar with the environment found HEX-RL’s explanations significantly more understandable compared to baseline models.

Olson et al. [44] proposed a method for generating counterfactual state explanations in visual RL environments, such as Atari games. Their approach involves creating minimal modifications to game states to illustrate alternative actions the agent could have taken. User studies demonstrated that these counterfactual explanations improved non-expert participants’ ability to identify flawed agent behaviours, outperforming nearest-neighbour and no-explanation baselines.

Cruz and Igarashi [45] developed an interactive explanation system that enables two-way communication between users and RL agents. Implemented in a Super Mario Bros. environment, the system allows agents to explain their decision-making processes using natural language templates, while users can provide corrective feedback. This bidirectional interaction was shown to be effective in diagnosing and repairing agent behaviours based on user preferences.

Ehsan et al. [46] explored the concept of "rationalisation" by training RL agents to generate human-like justifications for their actions. Through a think-aloud protocol, they collected human rationales and trained an encoder-decoder model to produce similar explanations. User studies revealed that these rationalisations enhanced participants’ trust and understanding of the agent’s behaviour, particularly in turn-based environments like Frogger.

These systems, while promising, also reveal limitations in scalability, domain generality, and user-specific adaptation. Peng et al. [43] note that their approach is tailored to text-based environments, which restricts applicability to visual or embodied RL domains. Olson et al. [44] report that counterfactual explanations often rely on fragile state representations, with limited robustness to stochastic dynamics. Cruz and Igarashi [45] highlight the challenges of maintaining consistent agent behaviour under user-modified policies, particularly when corrective feedback introduces conflicting objectives. Ehsan et al. [46] raise concerns about rationalisations diverging from actual policy logic, potentially misleading users despite increasing subjective satisfaction. These findings suggest that explanation quality is tightly coupled with both environment design and explanation modality and that current systems remain brittle when transferred across RL domains or evaluated with diverse user profiles.

2.2.6 Conclusion and Research Gap

The literature reveals a fragmented landscape for explainability in reinforcement learning. Model-agnostic tools like SHAP and LIME, adapted from supervised learning, often fail in RL due to their lack of temporal context, unrealistic perturbations, and

limited performance in continuous or high-dimensional spaces [7, 33]. Surrogate models, such as decision trees trained on abstracted states, offer improved interpretability [35], but typically rely on offline computation. Causal models enable contrastive reasoning [36], but are constrained by structural assumptions that limit their generalisation.

Visual explanation systems improve interpretability during or post-execution [39, 40], yet are mostly confined to simplified environments or static overlays. Despite Unity’s popularity in RL, it has seen little use for synchronised, real-time explanation. Human-centred methods—like natural language and interactive feedback [45, 46]—remain disconnected from high-performance agents.

This project addresses the missing link: a real-time, model-agnostic explanation pipeline embedded directly in Unity. It combines SHAP and surrogate trees to deliver interpretable visual overlays aligned with agent behaviour as it unfolds.

2.3 Summary

In this chapter, we introduced the core technologies and theoretical foundations underpinning this project, including the Unity game engine, the ML-Agents toolkit, reinforcement learning, and explainability. We then surveyed existing research across five key areas: model-agnostic post-hoc explainability, surrogate and causal models, visualisation and feedback systems, simulation-based platforms such as Unity, and human-centred delivery of explanations. Through this review, we identified consistent limitations in applying explainability techniques within real-time, interactive environments. These include a lack of temporal awareness in model-agnostic methods, limited runtime integration of surrogate models, and poor alignment between explanation delivery and user needs. From this, we established a clear research gap: the absence of an embedded, model-agnostic explanation pipeline within Unity capable of generating interpretable, real-time feedback. Addressing this gap forms the basis of the work that follows.

3 Methodology

This chapter outlines the complete design and implementation process, detailing the techniques and tools used throughout. The project was developed using the Unity Game Engine, with C# and Python as the primary programming languages. Below is an overview of the implementation structure:

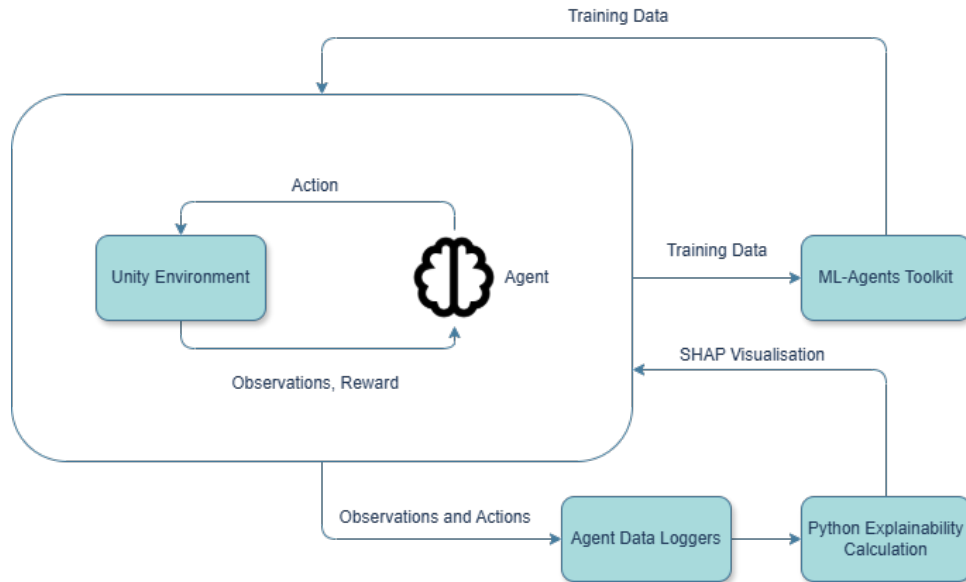


Figure 3.1 A DFD Overview of the Solution Implemented

3.1 Designing and Implementing the Unity Training Environments

This section will go into detail for the reasoning and implementation of the multiple Unity environments used for the training of the RL agents. This was done to use a series of widely varied training environments to highlight the limitations of the toolkit and the following explainability implementations.

Rather than creating multiple whole Unity environments from scratch, we opted to use pre-packaged environments provided by the ML-Agents toolkit ¹ as bases for us to then build upon and tailor to our needs.

3.1.1 Environment 1: Pyramids

The first environment implemented in this project is Pyramids, selected for its structural simplicity and suitability for evaluating sequential agent behaviour in a

¹<https://github.com/Unity-Technologies/ml-agents>

controlled setting. It consists of a single agent situated in a grid-based layout containing static walls, stacked bricks, and interactable objects.

The agent’s objective is composed of three ordered stages. First, it must locate and activate a switch, which triggers the appearance of a pyramid constructed from movable blocks. Upon activation, the agent must knock the structure over and collect a green brick that appears at the apex. This sequence introduces a clear task decomposition—navigation, interaction, and goal acquisition—while keeping the environment compact and deterministic.

Figure 3.2 illustrates the agent approaching the goal after triggering the switch and causing the pyramid to appear in the environment.

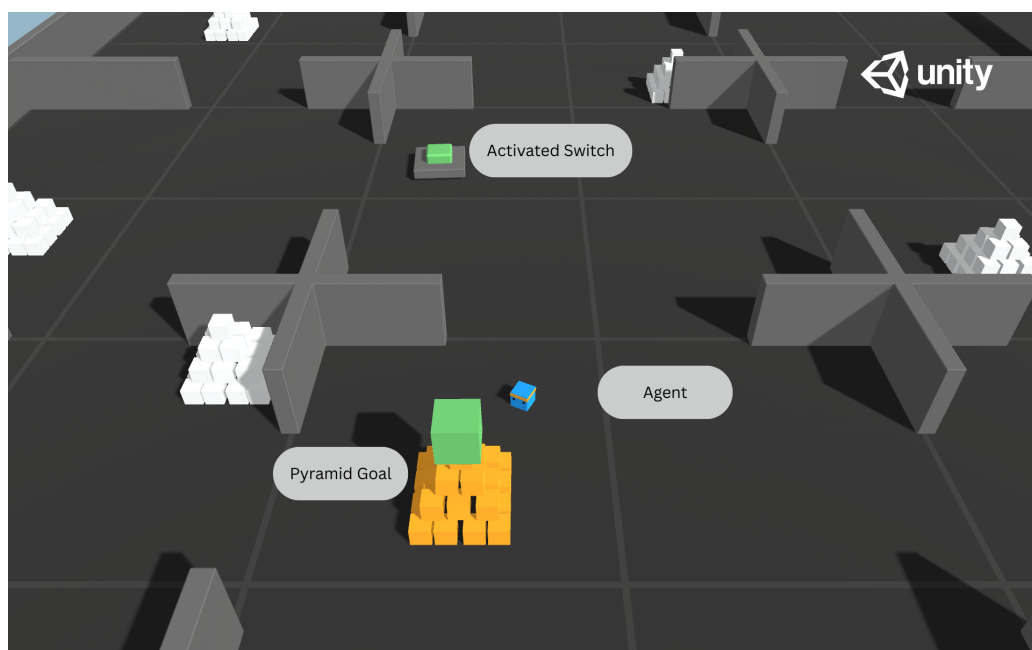


Figure 3.2 Agent approaching the green brick goal after switch activation [47].

The agent operates using a discrete action space with five possible actions: move forward, move backwards, rotate left, rotate right or do nothing. Observations are collected via a 172-dimensional vector, consisting of one-hot encoded raycast data, used to detect tagged objects (Block, Wall, Goal, SwitchOff, SwitchOn, Stone), along with the agent’s local x , y , and z velocities and the current state of the switch.

Initial testing with the environment’s default reward structure—comprising a terminal reward of $+2$ for reaching the goal and a step penalty of -0.001 —was unsuccessful, as the agent failed to complete the task within the maximum episode length of 5000 steps. As a result, the reward scheme was redesigned to better guide the agent’s progression through the environment.

The revised reward function consists of the following components:

1. **Existence Penalty:** A constant per-step penalty defined as:

$$R_{\text{step}} = -\frac{1}{\text{MaxStep}} = -0.0002$$

where $\text{MaxStep} = 5000$. This discourages inefficient behaviour and promotes faster completion.

2. **Switch Activation Reward:** A diminishing reward based on the number of steps taken before the switch is activated:

$$R_{\text{switch}} = \frac{5}{1 + (\text{StepCount}/1000)}$$

This encourages early activation and penalises delay.

3. **Goal Completion Reward:** Upon reaching the green brick, the agent's total cumulative reward is forcibly set to 10:

$$R_{\text{total}} = 10$$

This ensures that the final objective is consistently and strongly reinforced, independent of path efficiency.

This structured reward design improves agent learning by associating intermediate achievements with reward signals while preserving a strong incentive for task completion.

The environment was deliberately kept minimal in visual and spatial complexity to reduce noise during explanation evaluation. However, its sequential and conditional structure introduces sufficient behavioural variation to assess the clarity and accuracy of interpretability techniques. This makes *Pyramids* a strong candidate for controlled testing of real-time, model-agnostic explanation methods.

3.1.2 Environment 2: Dungeon Escape

The second environment used in this project is *Dungeon Escape*, which introduces multi-agent cooperation. It consists of three agents and a non-controllable enemy unit (the Dragon) placed in a dungeon scenario with obstacles, a locked door, and a key that must be retrieved to escape.

The agents must coordinate to complete a shared objective. One agent must locate and slay the Dragon, which drops a key upon defeat. This action is terminal for

the agent that engages the Dragon, functioning as a sacrifice. The remaining agents must then pick up the key and use it to unlock the dungeon door, completing the episode. If the agents take too long, the Dragon escapes through a cave, causing the environment to reset.

Figure 3.3 shows the initial setup of the environment, with all three agents and the Dragon holding its key, positioned in the dungeon.

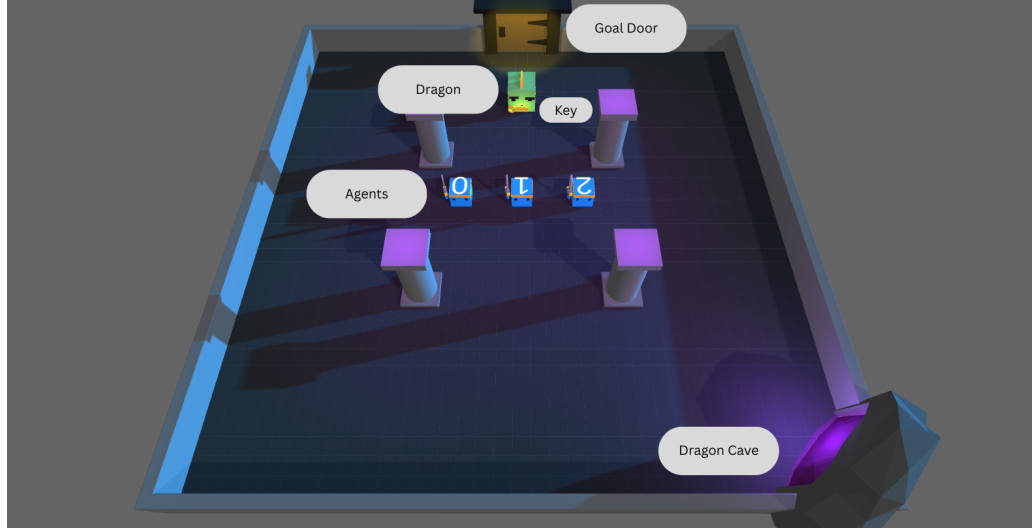


Figure 3.3 Initial state of the Dungeon Escape environment.

The agent group operates under a discrete action space with seven available actions: rotate clockwise, rotate counterclockwise, move forward, backwards, left, or right, and a no-op (do nothing). Each agent collects a 410-dimensional observation vector. This consists of raycast data from 51 rays, generated by stacking 17 directional rays across three vertical levels. Each ray can detect up to 6 tags: Wall, Agent, Dragon, Key, Lock, Portal, alongside a miss flag, and the normalised hit distance. This contributes 408 float values in total. Two additional float values are also added to the observations: one indicating whether the agent currently holds the key, and another flagging whether the Dragon has been defeated.

Agents do not share observations during runtime but instead contribute to training a single, shared policy. This decentralised execution with centralised training configuration encourages cooperative behaviour despite limited local observability.

Initial testing with the default reward configuration, which only granted a single group reward upon successful door unlock, did not yield reliable convergence within the episode length. Agents frequently failed to engage with the Dragon or coordinate key retrieval within time constraints.

To address this, a revised group-based reward scheme was implemented to provide more informative feedback and better support cooperative learning. All rewards and penalties are assigned to the agent group collectively, with no individual

agent receiving isolated feedback. The final reward structure is as follows:

Table 3.1 Revised group reward structure for the Dungeon Escape environment

Event	Reward / Penalty	Condition
Dragon defeated	+3.5	Any agent slays the Dragon
Key pickup	+1	Any agent collects the key
Door unlocked	+3	Any agent unlocks the door
Agent enters Dragon's cave	-3	Any agent crosses into cave causing it to die
Existence penalty (base)	-0.0001 per step	Applied to all steps during episode
Additional penalty (Dragon alive)	-0.002 per step	Until Dragon is defeated
Dragon escapes	-1	If Dragon exits via cave
Episode timeout	-1	If max steps reached without success

This revised structure introduces intermediate signals to reinforce progress toward the final objective, while maintaining a unified reward channel to support joint policy learning. The absence of agent-specific rewards encourages the emergence of cooperative strategies and facilitates the evaluation of group-level explainability in decentralised decision-making contexts.

This environment was selected for its emphasis on coordination, partial observability, and the trade-off between individual sacrifice and group success. These properties introduce unique challenges for both policy learning and interpretability. In particular, evaluating explanation methods in this context requires distinguishing individual agent intent from emergent group behaviour under sparse reward conditions.

3.1.3 Environment 3: Food Collector

The third environment used in this project is Food Collector, a multi-agent competitive task in which agents attempt to collect green food spheres while avoiding red poison spheres. Unlike the previous environments, which emphasised task sequences or cooperation, this scenario introduces competition and reward conflict across agents operating under identical behaviour policies.

The environment contains five agents within a bounded 2D arena populated by randomly distributed green and red spheres. Each agent attempts to maximise its score by collecting green spheres (+1 reward) while avoiding red ones (-1 penalty). No shared reward or cooperation is implemented—each agent is rewarded independently based on its interactions.

Figure 3.4 shows the Food Collector environment during execution, with agents navigating towards green food spheres and avoiding red poison.

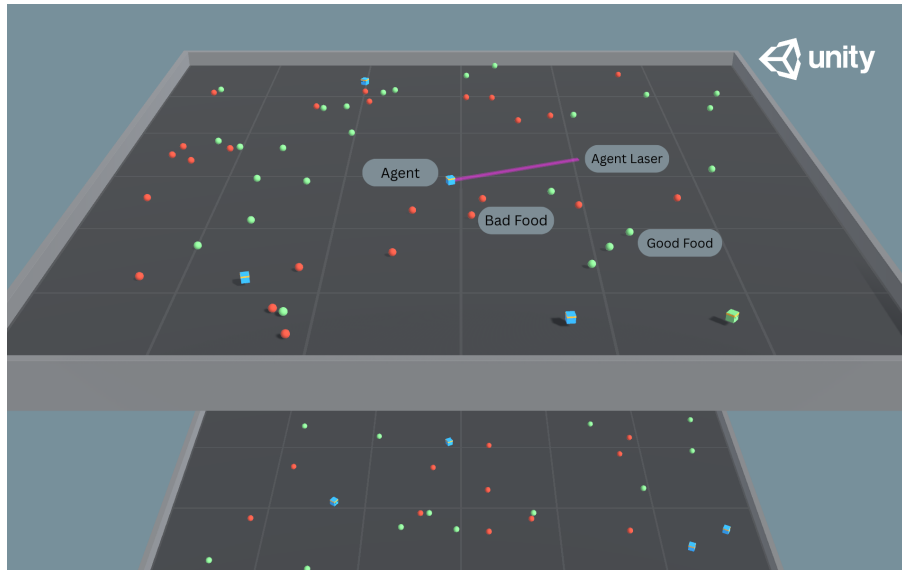


Figure 3.4 Agents navigating the Food Collector environment to collect green food and avoid red poison [47].

For this dissertation, each agent receives observations exclusively through a spatial 2D grid sensor. This sensor captures a 40×40 field of perception centred on the agent's forward direction, with each grid cell encoding the presence or absence of five distinct object types: Food, Agent, Wall, BadFood, and FrozenAgent. Each of these tags is represented as a separate channel.

The resulting observation tensor is flattened into an 8000-dimensional vector per timestep ($1600 \text{ cells} \times 5 \text{ tags}$), where each float value corresponds to the presence score of a given object type within a specific grid cell. This high-dimensional, spatially structured input makes the environment well-suited for testing explanation techniques that rely on spatial saliency or input attribution.

The action space consists of:

- Three continuous actions: forward/backwards motion, side motion, and rotation.
- Two discrete actions: shoot laser or do nothing.

This environment was selected to test whether explanation methods could perform well when placed into a continuous action space rather than a strictly discrete one. It also allowed different explainability techniques to be applied, as this agent uses grid-based observations rather than raycast-based ones.

3.2 Training Agents using the ML-Agents Toolkit

When training agents using the Unity ML-Agents Toolkit, two primary interfaces are available: training can occur directly within the Unity Editor or through a compiled executable of the environment. While both approaches were used during experimentation, the majority of training was conducted within the Editor to support rapid iteration and on-the-fly modification of environment logic and parameters without requiring recompilation.

Multiple copies of each environment were instantiated within the Unity scene to accelerate training. This parallelisation allowed several agents to collect experiences simultaneously, significantly increasing the learning rate without affecting environment dynamics.

Training was initiated via the ML-Agents command-line interface using the `mlagents-learn` command. This tool connects to an active Unity environment and begins communication once the simulation is launched.

Once the connection is established, pressing the Play button in the Unity Editor begins training. If using a standalone executable instead, the environment path must be specified with the `--env` argument when launching the trainer (e.g., `mlagents-learn --env=Environment.exe`).

Each of the three environments used distinct training configurations:

- **Pyramids:** Trained using the PPO algorithm within the Unity Editor for a total of 5,000,000 environment steps ².
- **Dungeon Escape:** Trained using Multi-Agent Posthumous Credit Assignment (MA-POCA), a variant of PPO designed for cooperative multi-agent environments, also for 5,000,000 steps; however, training was stopped after 2,300,00 steps as performance plateaued. ³.
- **Food Collector:** Trained using PPO for 2,000,000 steps. This shorter training duration was sufficient due to the simplicity and fast convergence of the task ⁴.

It is important to note that no automated hyperparameter tuning was performed. Instead, hyperparameters were adjusted manually through repeated trial runs, based on how well each agent performed in its environment. While this process was not formally optimised, it was effective in finding stable configurations that allowed each agent to learn the task reliably. To evaluate explainability across environments of varying complexity, this level of tuning was considered sufficient.

²Appendix A.1

³Appendix A.2

⁴Appendix A.3

3.3 Logging Agent Data

After training, the generated `.onnx` model files were imported into the Unity Editor and assigned to the agents. This enabled real-time inference using the trained models, allowing agents to act autonomously within the environment using all the behaviours learned during training.

By modifying the toolkit's built-in sensor components, it was possible to access the agent's internal observation array, which is normally encapsulated under several layers of inheritance and protection. This enabled real-time logging of the observations received by the agent and the actions taken in response. A custom logging function was implemented to capture these observation–action pairs and save them to corresponding `.bin` files, making them easy to export and analyse using Python.

In total, *Pyramids* yielded 12,696 logged pairs. For *Dungeon Escape*, Agent 0 recorded 9,656, Agent 1 recorded 7,247, and Agent 2 recorded 7,131 pairs. The *Food Collector* environment, which operates with a shared policy across agents, produced 34,950 logged pairs collectively.

3.4 Applying Explainability Methods Using Python

Before applying any explainability techniques, the neural networks stored in the exported `.onnx` files needed to be modified to expose the SoftMax output layer corresponding to the agents' discrete action distributions. By default, these models only output a single integer representing the selected action, which is insufficient for attribution-based methods that require access to the full action probability distribution.

This modification was performed using the `onnx` Python library. Specifically, the ArgMax node responsible for selecting the final action was located, and its input—the SoftMax probabilities—was forked as an additional output of the network.

With the logged observation-action pairs and the modified agents, the following techniques were implemented on the following agents:

Table 3.2 Explainability techniques applied across environments

Technique	Pyramids	Dungeon Escape	Food Collector
Surrogate decision tree	Yes	Yes	Yes
Semantic SHAP (overall)	Yes	Yes	Yes
Step-wise SHAP analysis	Yes	Yes	No
Grid-based saliency mapping	No	No	Yes

3.4.1 Surrogate Model Training and Interpretation

The surrogate modelling pipeline was implemented using several core Python libraries. `scikit-ensemble` was used to train both `RandomForestClassifier` and `RandomForestRegressor` models for policy approximation, as well as `scikit-learn` for `DecisionTreeClassifier` and `DecisionTreeRegressor` models for symbolic explanation. `onnxruntime` enabled direct inference from exported `.onnx` models, while `NumPy` and `Matplotlib` supported data handling and visualisation.

Observation–action pairs, collected during runtime, were preprocessed and used to train classification models for discrete action spaces and regression models for continuous ones. Each environment was configured with its respective ONNX model, observation shape, action logs, and multi-agent settings. Preprocessing steps included trimming raycast inputs (as in `Pyramids`) and downsampling $40 \times 40 \times 5$ spatial observations to $10 \times 10 \times 5$ in `Food Collector`, yielding 500 input features.

Random forest models were used to approximate the policies with high accuracy and to generate confusion matrices and feature importance plots, providing a quantitative assessment of how closely the surrogate matched the original agent. Decision tree models were then trained on the same data to generate symbolic, rule-based approximations of agent behaviour using interpretable IF–THEN logic. However, this interpretability layer was only fully implemented for the `Food Collector` agent due to the spatial structure of its input and the reduced complexity of its decision space.

3.4.2 Semantic SHAP Explanations

To identify which semantic components of the observation space influenced agent behaviour, a dedicated SHAP analysis was conducted for each environment using a consistent pipeline. Observations were grouped into semantically meaningful categories based on their structure, such as raycast object types, grid channels, or scalar values.

For each action dimension—whether discrete or continuous—an independent `XGBRegressor` was trained to approximate the ONNX model’s raw output (logit or confidence score) corresponding to that specific action. These regressors were trained using the full observation vector as input and the ONNX model’s output for that action as the target.

After training, SHAP values were computed for each regressor using the `shap.Explainer` API. These values, calculated per input feature, were aggregated according to the previously defined semantic groupings. This allowed for an interpretable summary of how different high-level observation categories contributed

to each predicted action. The resulting attributions were visualised using per-action bar charts, showing which semantic groups most strongly influenced the model's behaviour.

3.4.3 Step-wise Semantic SHAP Explanations

To generate fine-grained, local explanations of agent decision-making, a step-wise SHAP method was applied across all environments. First, the logged observation-action pairs were grouped by the action taken. For each action, a dedicated `XGBRegressor` was trained using only the observations that led to that specific action. This model learned to approximate the internal logic behind selecting that action based on the input features.

Once trained, each regressor was used in conjunction with SHAP to compute individual feature importance scores for every observation in its group. This produced a local attribution for each decision, revealing which semantic components of the observation most influenced the agent's choice in that specific instance.

Using these step-wise SHAP values, line plots were generated to visualise how the importance of each semantic feature changed over time. This allowed inspection of what the agent appeared to prioritise at each step, offering detailed insight into the evolving decision-making process throughout an episode.

3.4.4 Grid-Based Saliency Mapping

To further interpret the agent's decision-making in the `Food_Collector` environment, saliency maps were generated by analysing the gradients of the trained neural network's outputs with respect to its input observations. This approach identifies which spatial and semantic regions of the observation were most influential in determining the agent's chosen actions.

The exported `.onnx` model was first converted into a PyTorch-compatible format using the `onnx2pytorch` library. Observations previously logged during agent execution were loaded and reshaped into a $40 \times 40 \times 5$ tensor format, where each of the five channels represents the presence of a different entity class: `Food`, `Agent`, `Wall`, `BadFood`, and `FrozenAgent`.

Each observation was passed through the converted model, which produced two outputs: discrete logits (used to determine whether the agent chose to shoot) and continuous logits (used for directional movement such as forward, strafe, or turn). Gradients were computed for the logits corresponding to the actions taken, and the absolute values of these gradients were accumulated to construct class-specific saliency maps.

Separate saliency maps were generated and averaged for each discrete and continuous action class. For instance, individual saliency values were accumulated separately for Shoot and No Shoot decisions, as well as for Forward, Strafe, and Turn movements. This allowed for a visual comparison of which semantic channels contributed most strongly to different action categories.

The resulting maps were plotted as heatmaps, with one set of visualisations comparing Shoot versus No Shoot, and another set comparing Forward, Strafe, and Turn. Each plot showed activation intensities for each of the five semantic input channels, highlighting the spatial regions that were most influential in the agent's policy decisions.

3.5 Real-Time Unity Explanations

Because Unity does not support in-engine SHAP computation, step-wise SHAP values were calculated externally in Python using the method previously described. To ensure alignment between the logged data and the real-time Unity environment, all inference runs were performed under a fixed random seed. This guarantees consistent environment behaviour and agent trajectories across repeated runs.

During these seeded runs in the Pyramids and Dungeon Escape environments, observation–action pairs were logged and used to compute the step-wise SHAP values calculated in Section 3.4.3. These values were then exported to a `.json` file for later use. Rather than recalculating SHAP inside Unity, the previously computed values were reimported and synchronised with the agent's behaviour during playback.

In Unity, the agent still ran inference as usual, but its predicted actions were overridden at each step using the actions recorded during the seeded run. This meant that while the model was technically running, it never actually influenced behaviour—the agent simply replayed the same sequence of decisions as before. Since the environment was also seeded, everything unfolded in the same way, making it possible to align each action with its corresponding SHAP values. A custom UI element in Unity displayed these SHAP scores as a real-time bar chart, updating at every step to show which features were most important to the action being executed. In the Dungeon Escape environment, the SHAP scores for all agents were displayed side by side, with the visualisation for each agent pausing individually if that agent died during the episode. Recordings of these visualisations were then made using the OBS⁵ software and stored⁶ for later analysis.

⁵<https://obsproject.com>

⁶https://drive.google.com/drive/folders/1gWrgAdMedWHPMd5e9dAmqREs18mS_KLa?usp=sharing

4 Results and Evaluation

This chapter evaluates the overall effectiveness of the project across three key areas: the design of the environments, the performance of the trained agents, and the usefulness of the applied explainability techniques. The focus is on the extent to which the visual and semantic explanations reflect agent behaviour, their interpretability for human users, and the practicality of integrating such methods within Unity-based simulation workflows. Experiments were carried out across all three environments: *Pyramids*, *Dungeon Escape*, and *Food Collector* and on a computer with the following specifications: AMD Ryzen 7 5800x3D CPU, 16GB DDR4 RAM, Nvidia RTX 4090 GPU and running Windows 11.

4.1 Environment Performance and Feasibility

Each environment used in this project varied in its effectiveness for both agent training and integration of explainability techniques.

The *Pyramids* environment was the most straightforward and successful. After a minor reward structure adjustment, the agent trained reliably and consistently achieved its goal. As a single-agent environment with a simple, linear objective, it posed no major issues for the integration of explainability methods. The absence of multiple agents or complex interactions made visualisation clean and easily interpretable.

The *Dungeon Escape* environment was more challenging. While it was designed with generalisation in mind—featuring randomised agent and dragon positions, randomised speeds, and rotated layouts—this variability led to training inconsistencies. Even well-trained agents could fail due to unlucky configurations, such as dragons spawning directly at the exit or agents getting stuck on obstacles. Additionally, the dragon’s fixed linear movement logic sometimes resulted in ambiguous or edge-case behaviours that were difficult for agents to learn around. However, despite these issues, explainability integration was relatively straightforward due to the environment’s structural similarity to *Pyramids*.

The *Food Collector* environment trained extremely well with no design modifications required. However, it was by far the most difficult environment for explainability integration. The use of a grid sensor, continuous control, and shared policy across competing agents introduced multiple challenges. Visualising explanations in real time was impractical due to overlapping behaviours and the dynamic spawning of food. Furthermore, attempts to seed the environment proved unreliable, as the random generation of food after consumption made deterministic replication infeasible. While limited in visual explanation capabilities, this environment

still offered valuable contrast and insight when analysing generalisation and policy-level explanation.

4.2 Agent Training

During training with the ML-Agents toolkit, a TensorBoard event file is generated containing key performance metrics. From this file, the following statistics were extracted and visualised: cumulative reward, entropy, value loss, and episode length. The remaining training metrics for each agent may be found within this dissertation's Appendix.

4.2.1 Pyramids Environment

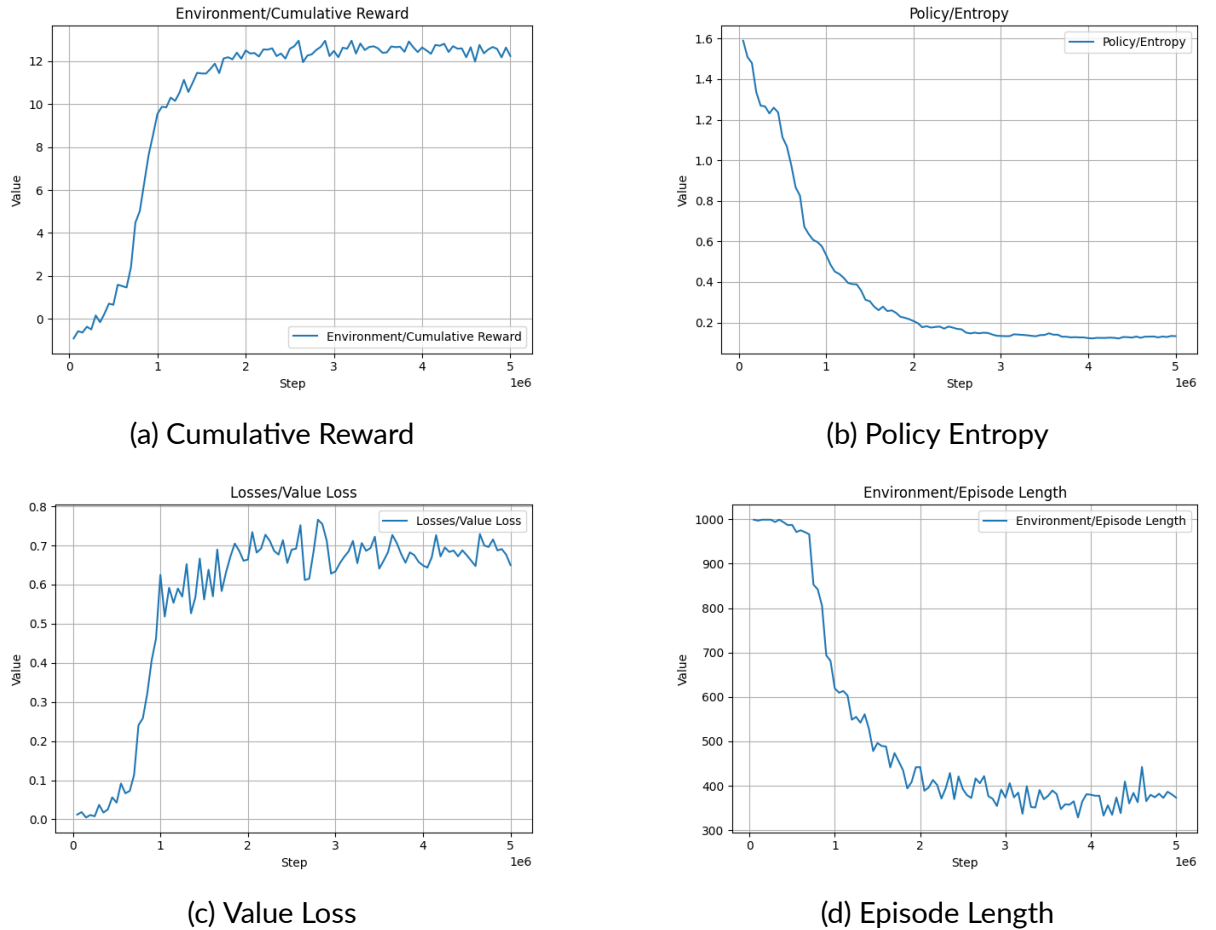


Figure 4.1 Training metrics for the Pyramids environment.

The training results from the Pyramids environment show that the agent learned quickly and steadily. Cumulative reward climbed sharply early on and then levelled off at a high value, which suggests the agent figured out a good strategy early in training

and kept improving consistently. At the same time, episode length dropped and then stabilised, meaning the agent was finishing tasks faster as it got better. Entropy followed a typical pattern – high at the start when the agent was exploring, then dropping off as it began to settle on confident, consistent actions. The value loss increased a bit at first but eventually levelled out at a low level, showing that the critic was learning to estimate returns reliably. Overall, the training process was smooth and efficient, with clear signs of fast convergence and stable behaviour.

4.2.2 Dungeon Escape Environment

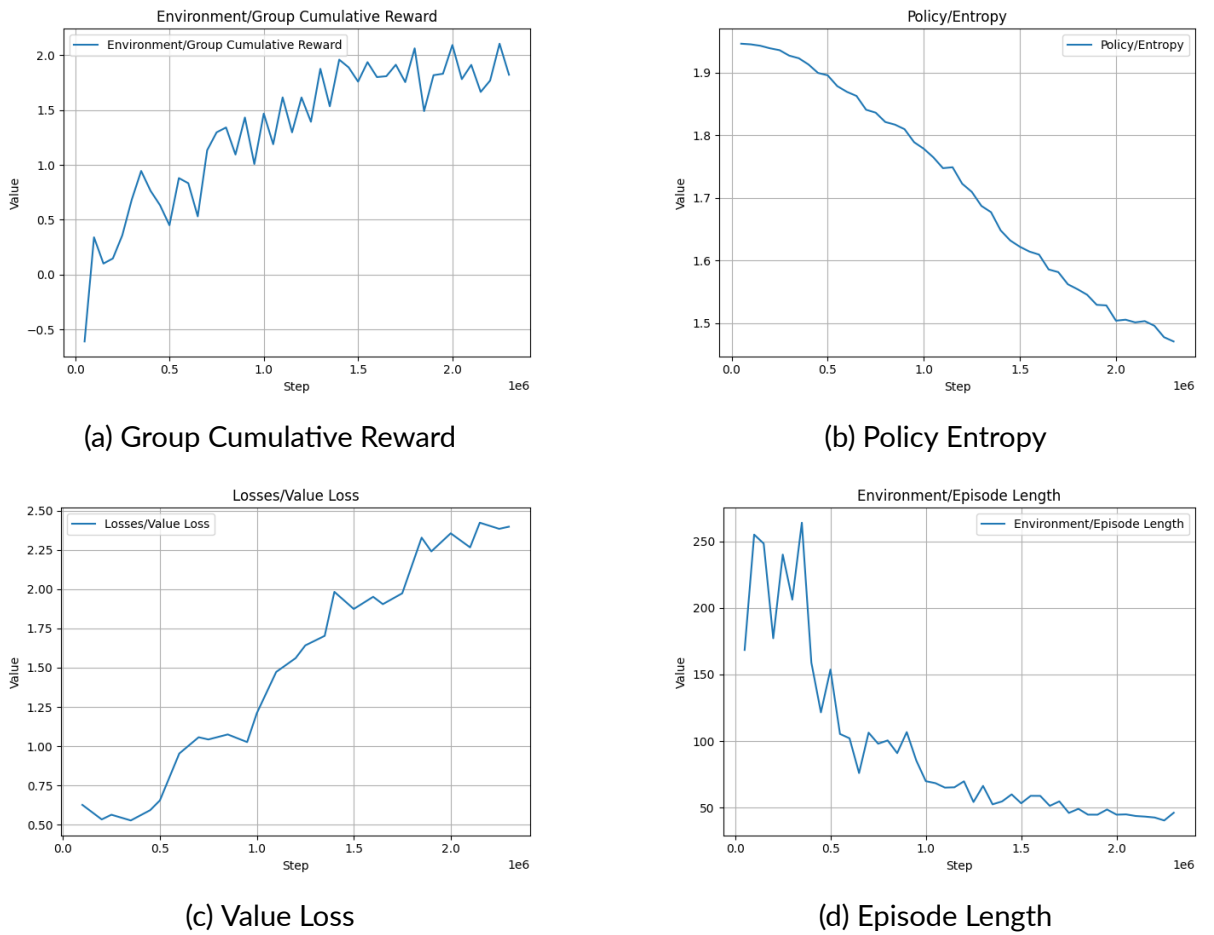


Figure 4.2 Training metrics for the Dungeon Escape environment.

Similarly to the previous two environments, the training results from the Dungeon Escape environment show that the agents generally learned to perform the task well over time. Cumulative reward increased steadily, suggesting that the group was getting better at reaching their goals and earning rewards. This was supported by a consistent drop in episode length, meaning the agents were completing episodes more efficiently as training progressed. Entropy gradually decreased, showing a typical shift from early exploration to more confident and consistent behaviour. The value loss,

however, kept rising throughout training, which could mean the critic struggled to estimate returns accurately, possibly due to the complexity or variability in the agents' interactions with the environment. Policy loss stayed flat and didn't show any major changes (see Appendix), so it didn't provide much additional insight. Still, the overall training process appears to have led to a functional set of policies capable of handling the environment reasonably well.

4.2.3 Food Collector Environment

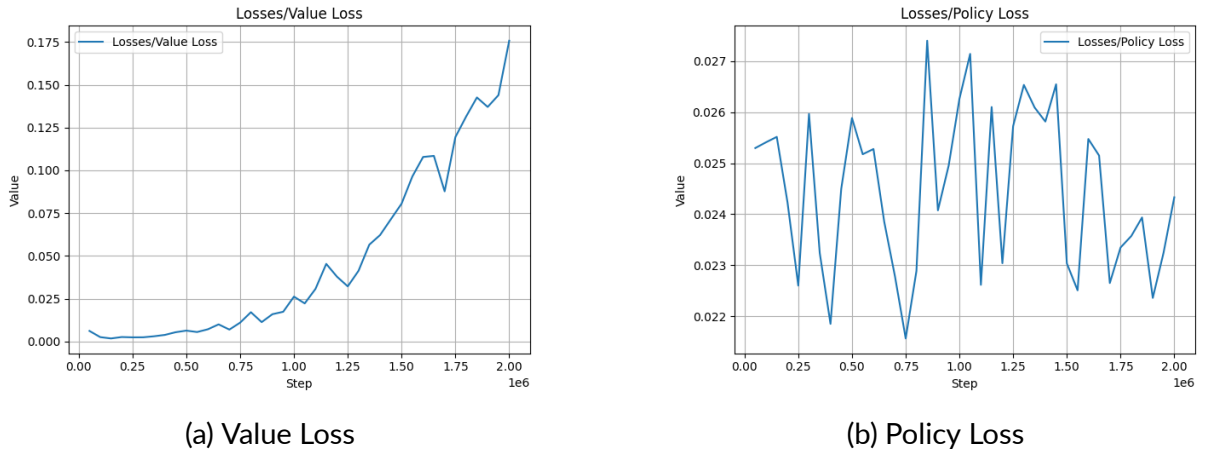


Figure 4.3 Training metrics for the Food Collector environment.

The training data from the Food environment shows a steady improvement in the agents' performance over time. Cumulative reward increases gradually across the entire training period, indicating that the agents consistently learned to collect more rewards within each episode's fixed duration. Entropy decreases smoothly, reflecting a standard shift from broad exploration to more focused, consistent decision-making. The value loss rises steadily, which suggests that the critic had increasing difficulty estimating returns accurately, possibly due to the non-terminal nature of the task or the growing variability in agent behaviour as they learned. Policy loss remains relatively stable, fluctuating slightly but without any major instability, which points to steady updates in the agents' policies. Overall, the trends suggest a slow but reliable learning process in which the agents improved their ability to gather rewards without encountering major training issues.

4.3 Explainability Results and Insights

The following section will evaluate the results of applying the XAI techniques outlined in Section 3.4 onto the trained agents.

4.3.1 Surrogate Modelling and Decision Tree Explanations

This section evaluates the effectiveness of the surrogate modelling approach in approximating and interpreting the behaviours of trained agents across the three environments. The goal was twofold: first, to assess how accurately random forest models could replicate the original ONNX-based policies; and second, to determine whether decision tree models could provide interpretable, rule-based explanations of agent decision-making. The evaluation considers both the predictive fidelity of the surrogates and the practicality of extracting meaningful symbolic representations, with attention to how observation complexity impacted results across environments.

The fidelity of the surrogate models was assessed using standard classification and regression metrics, depending on the agent's action space. Each model was evaluated on its ability to replicate the original policy behaviour using logged observation–action pairs.

Table 4.1 Discrete Surrogate Model Performance Across Environments

Environment	Accuracy	# Classes	Notes
Pyramids	0.989	5	One action unused; near-perfect performance
Dungeon Escape	0.843	7	Strong class balance; robust replication
Food Collector	0.714	2	Moderate performance; simple binary policy

For the Food Collector agent, which includes both discrete and continuous outputs, a secondary surrogate model was trained to approximate its continuous shoot strength. Regression results are summarised in Table 4.2.

Table 4.2 Continuous Surrogate Regression Metrics for Food Collector

Metric	R^2 Score	MAE	MSE
Food Collector	0.352	0.245	0.095

The results indicate that surrogate models effectively approximated the original policies, especially in Pyramids and Dungeon Escape, where accuracy scores exceeded 0.84. In Food Collector, the surrogate classifier demonstrated moderate fidelity in replicating the discrete shoot action. However, the continuous action surrogate performed poorly, with an R^2 score of just 0.352. This suggests the continuous component of the policy was too complex or noisy to be reliably approximated using the available features, limiting the interpretability of those actions.

These confusion matrices show where each surrogate model struggled to mimic the original agent's behaviour. The Pyramids surrogate was almost flawless, while

Dungeon Escape showed more confusion, especially between similar movement actions like strafing and turning. In Food Collector, the model often mixed up the decision to shoot or not, likely due to overlapping or ambiguous inputs.

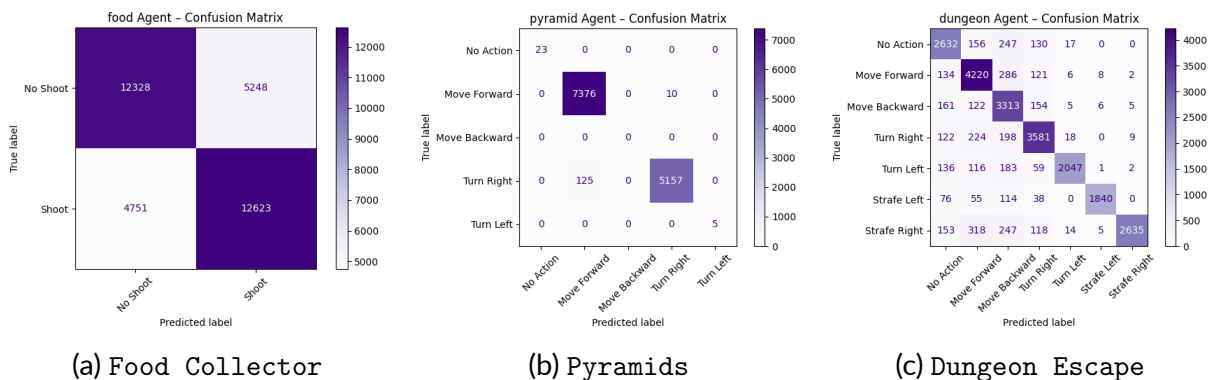


Figure 4.4 Confusion matrices for surrogate classifiers across all environments.

Decision Tree explanations were attempted to be implemented using these surrogate models, however, the immense dimensionality of the inputs makes explaining exactly what input feature led to a branch in the tree extremely complex. Within *Pyramids* and *Dungeon Escape*, a single float value in their 172 and 410 dimensional inputs respectively could lead to a whole different decision, and for the scope of a decision tree aimed to be human-readable, a suitable solution could not be found without severely limiting tree depth. For *Food Collector* the following tree was created:

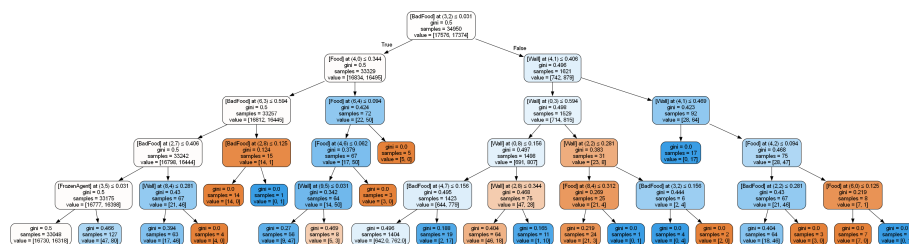


Figure 4.5 Food Collector Decision Tree

However, to convert this into a set of human-readable IF–THEN rules, a text based approach was used, a small example of this is visible below:

Although we effectively translated agent policy logic into human-readable rules, the policy’s high-dimensional nature still proves difficult to understand, with actions nested under multiple rules and no overall clear decision-making present in the rules.

While surrogate models allow a small look into the agent's decision-making processes, they do not provide much past that in the way of meaningful explanations, regardless of how the data is displayed or processed. Hence why SHAP was chosen to be displayed in real time as opposed to decision tree rules.

```

IF [Wall] at (0,9) <= 0.34:
  IF [Wall] at (0,6) <= 0.28:
    IF [Wall] at (0,5) <= 0.41:
      IF [Wall] at (1,4) <= 0.09:
        IF [Wall] at (0,2) <= 0.34:
          IF [Wall] at (1,0) <= 0.28:
            IF [Wall] at (1,8) <= 0.34:
              THEN Move Backward + Strafe Right + Turn Left
            ELSE:
              THEN Move Backward + Strafe No Action + Turn Left
          ELSE:
            IF [Wall] at (0,1) <= 0.09:
              THEN Move Backward + Strafe No Action + Turn Left
            ELSE:
              THEN Move Backward + Strafe Right + Turn Left

```

Figure 4.6 Example of IF–THEN Rules

4.3.2 Overall and Step-Wise SHAP Explanations

The following section will go through the results and findings from the calculated overall and step-wise SHAP values for each of the agents.

To understand which semantic features most influenced decision-making in the Pyramids environment, the overall SHAP values computed in Section 3.4.2 were plotted and analysed. Actions 1 (Move Forward) and 3 (Turn Right) were selected for detailed analysis, as they were the most frequently chosen actions by the model.

As shown in Figure 4.7, the agent’s movement decisions were predominantly influenced by its Velocity and spatial awareness cues such as Wall Detected, Block Detected, and Proximity. These features consistently ranked highest in SHAP importance for both actions.

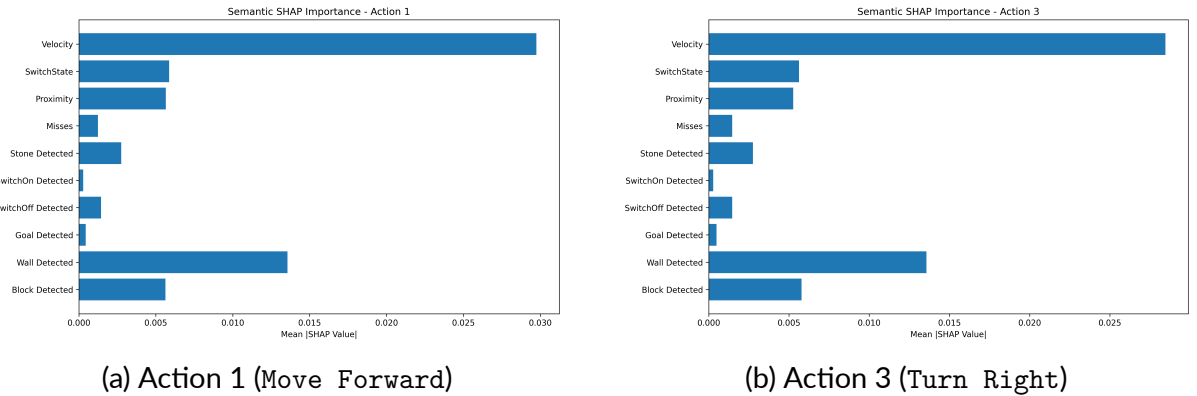


Figure 4.7 Semantic SHAP importance for the two most frequently chosen actions in the Pyramids environment.

SwitchState and Stone Detected also held some influence, likely reflecting the agent’s need to monitor environment state changes mid-task. Conversely, features like Goal Detected, Misses, and SwitchOff Detected exhibited negligible importance, indicating they were either too rare or not relevant for the dominant behaviours.

Figure 4.8 compares SHAP importance values for Actions 1, 3, and 5 across the three agents in Dungeon Escape. HasKey consistently emerged as the most influential

input, particularly for Agent 1, suggesting its primary role in key retrieval and door unlocking. DragonDead also showed strong influence, especially in post-combat behaviours like Action 5. Spatial features such as WallDetected and Proximity contributed moderately, reflecting their role in navigation and collision avoidance. Although all agents were trained on the same shared policy, these plots highlight behavioural divergence between them based on their roles and experiences during training.

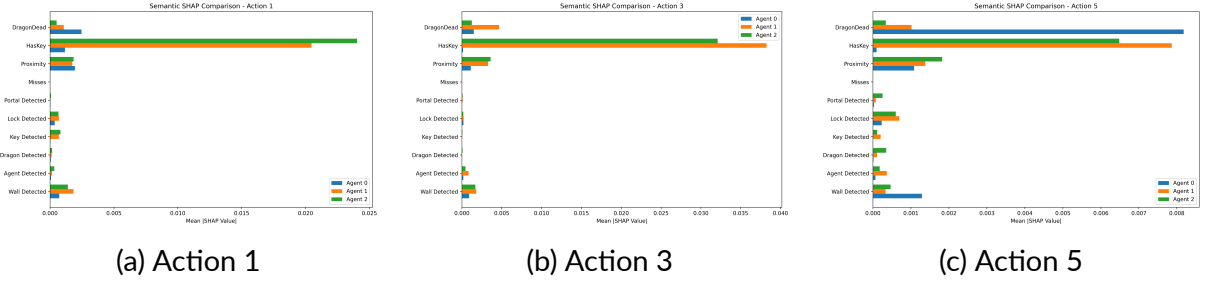


Figure 4.8 Semantic SHAP comparisons across all three agents in the Dungeon Escape environment.

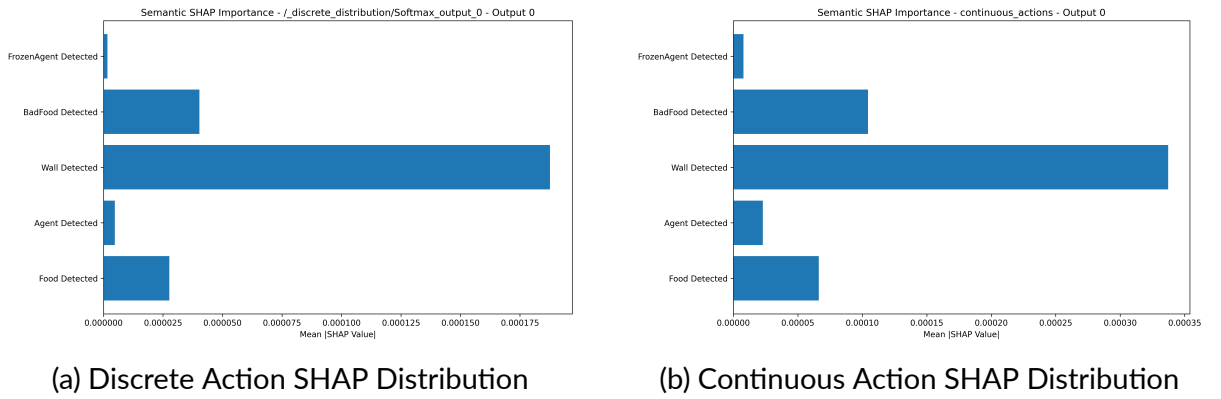


Figure 4.9 Semantic SHAP importance for one continuous action and one discrete action for the Food Collector environment

In the Food Collector environment, semantic SHAP analysis was used to determine the most influential perceptual features driving agent behaviour across both continuous and discrete action branches. Despite the presence of multiple categories in the agent's $40 \times 40 \times 5$ observation tensor, results consistently highlighted one dominant influence: the presence of nearby walls.

Across all actions, regardless of whether they were discrete or continuous, wall proximity emerged as the most significant factor affecting decision-making. This consistent weighting toward wall detection suggests the agent's behaviour was heavily shaped by spatial navigation constraints, prioritising collision avoidance or pathing efficiency over object interaction or competitive dynamics.

Step-Wise SHAP calculations provided meaningful insights into what were the most important features to each agent in real-time as they interacted with their environments. While the overall SHAP calculations above show similar findings, the details these step-wise calculations provide allow for a deeper understanding of the step-by-step decision making of the agents, rather than the feature which was valued the majority of the time.

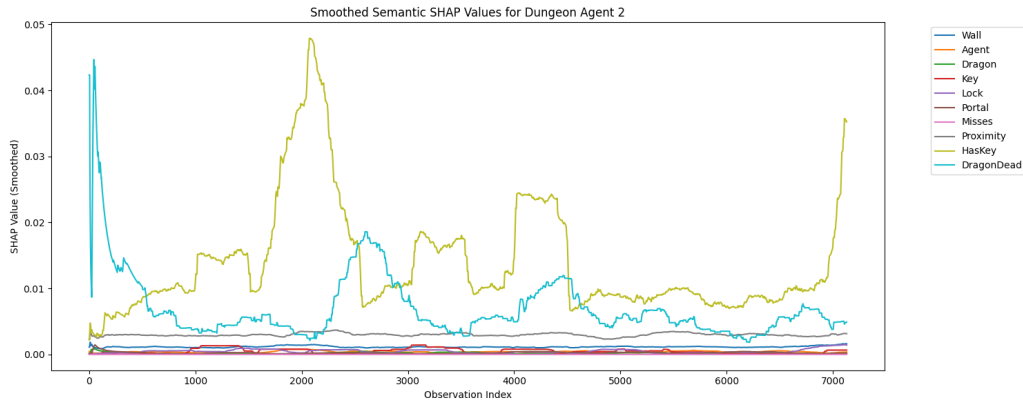


Figure 4.10 Step-Wise SHAP for Agent 2 Within Dungeon Escape

Whereas the overall SHAP calculation for Dungeon Escape primarily showed Dragon Dead and HasKey to be the most important features, Figure 4.10 shows how these features increase and decrease in importance as the agents progress through the episodes.

4.3.3 Grid-Based Saliency Mapping

To complement the semantic explanations provided by SHAP, saliency maps were generated for the Food Collector agent’s $40 \times 40 \times 5$ observation tensor. These maps highlight spatial regions within each channel that most influenced the agent’s discrete and continuous action predictions.

Figure 4.12 displays the aggregated saliency responses for a single semantic input type across all discrete and continuous actions respectively. Across both action branches, certain trends were apparent. Most notably, strong activations occurred near the agent’s immediate vicinity—especially in the centre of the grid—reflecting a focus on nearby obstacles and objects. Walls, food, and other agents consistently produced concentrated regions of saliency, aligning with findings from SHAP analyses that these features played a central role in decision-making.

However, while saliency maps provided useful spatial cues, their interpretation remained ambiguous without supporting context. Unlike SHAP values, which assign specific importance weights to inputs, saliency maps require visual inspection and do not offer direct semantic attribution. Despite this limitation, they were useful in

confirming that agent attention was spatially consistent with expected patterns, such as avoiding walls or targeting food.



Figure 4.12 Semantic saliency comparisons for a single input feature across all actions in the Food Collector environment.

4.4 Real-Time Unity Visualisation

The step-wise SHAP values discussed in Section 3.5 enabled real-time visualisation of agent decision-making within seeded Unity environments. Ideally, these explanations would have been computed live, without requiring pre-seeding or offline analysis. However, due to hardware limitations, generating SHAP values at every time-step during inference was computationally infeasible. Even connecting Unity to a Python backend via WebSocket introduced substantial latency, rendering real-time explanations impractical.

To address this, a seeded post-hoc approach was adopted, wherein SHAP values were computed externally and replayed in synchrony with the agent's recorded

decisions. This technique offers a scalable conceptual framework for visualising explanations in other RL environments, provided that agent behaviours can be reliably reproduced.

The visualisations were implemented as dynamic bar charts rendered via Unity's UI canvas. This was achieved for the Pyramids and Dungeon Escape environments, while the Food Collector environment's design complexity precluded such integration without extensive modification.

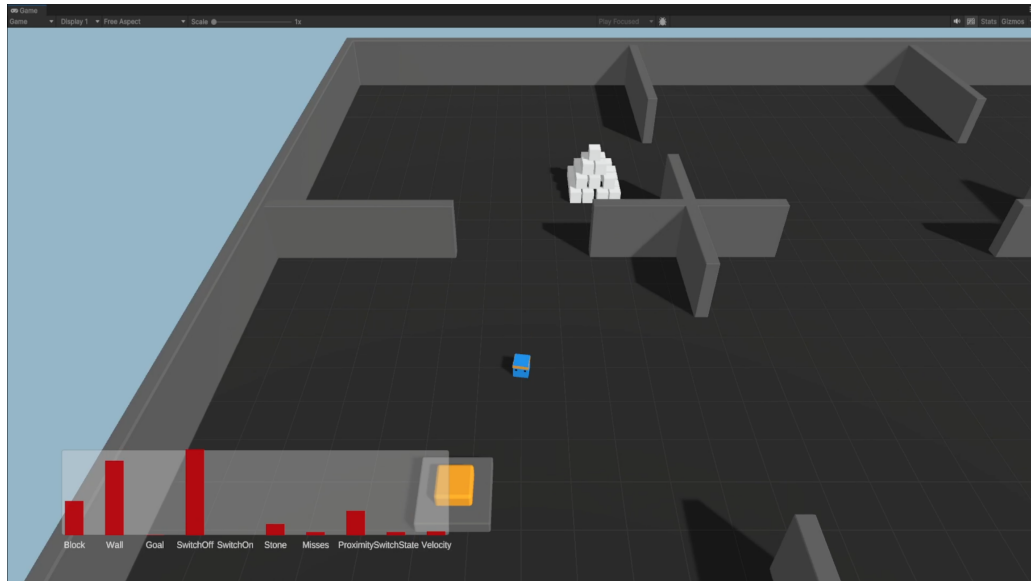


Figure 4.13 Real-Time Pyramids SHAP visualisation. The agent is seen prioritising the switch once it is in view.

4.5 Chapter Summary

This chapter evaluated the project across environment design, agent performance, and the practical application of explainability techniques. The Pyramids environment proved most stable and interpretable, while Dungeon Escape introduced training variability due to stochastic elements, and Food Collector—despite training effectively—posed significant challenges for explanation and visualisation. Surrogate models successfully approximated discrete agent behaviour in most cases, though their interpretability was limited in high-dimensional input spaces. Overall and step-wise SHAP analyses revealed meaningful semantic insights into agent decision-making, highlighting key features such as velocity, spatial awareness, and task-specific cues. Finally, a novel seeded visualisation strategy was developed to display SHAP scores within Unity environments in real time, offering a viable framework for post-hoc interpretability in interactive simulation settings.

5 Conclusion

5.1 Revisiting the Aims and Objectives

This project set out to evaluate the effectiveness of model-agnostic explainability techniques when applied to RL agents and to assess the Unity engine’s viability as a platform for in-simulation explanation delivery. Four objectives were defined:

O1: Environment Implementation. Three environments—Pyramids, Dungeon Escape, and Food Collector—were implemented or adapted for this project. These environments offered diverse challenges, including multi-agent dynamics, continuous actions, and variable observation structures. Pyramids was the most straightforward, while Food Collector proved most complex in terms of explainability.

O2: Agent Training and Inference Logging. RL agents were trained using the ML-Agents toolkit with PPO and MA-POCA algorithms. Post-training, inference runs were used to log observation–action pairs for offline analysis, supporting both classification and regression-based explainability methods.

O3: XAI Technique Integration. Model-agnostic techniques—including Random Forest and Decision Tree surrogates, SHAP (both overall and step-wise), and saliency mapping—were implemented in Python. These methods were tailored to each environment’s observation structure and agent design, revealing key semantic features that influenced decisions.

O4: In-Engine Visualisation. A seeded, post-hoc pipeline was developed to integrate precomputed SHAP values into Unity environments. Real-time bar charts were displayed in sync with agent behaviour in Pyramids and Dungeon Escape, showcasing Unity’s potential for explainable RL visualisation despite the absence of real-time computation.

Overall, the project successfully demonstrated that semantic feature attribution techniques like SHAP can yield meaningful insights into policy behaviour and can be embedded within high-fidelity simulation environments like Unity.

5.2 Project Criticisms

The project faced several limitations. Most notably, real-time SHAP generation within Unity proved computationally infeasible, necessitating a seeded post-hoc approach that restricted flexibility and scalability. In terms of model fidelity, surrogate models struggled in high-dimensional or continuous domains, particularly in Food Collector, where the surrogate model performed poorly when approximating the agent’s continuous movement vector, as shown by its low R^2 score.

Decision tree models, while interpretable in principle, were hard to distil into meaningful IF-THEN rules due to input dimensionality. For environments like *Dungeon Escape*, this made symbolic logic impractical without severe reductions in tree complexity.

From a research design perspective, a key shortcoming was the absence of human-centred evaluation. The visual explanations produced were not tested with users for clarity, usability, or trust enhancement—factors critical for XAI systems. Finally, hyperparameter tuning for training was carried out through manual iteration rather than systematic optimisation, possibly impacting both agent performance and downstream explainability.

5.3 Future Work

Several directions could improve and extend this work. First, optimisation of explanation generation—via more efficient algorithms or GPU-accelerated SHAP variants—could help move toward true real-time in-environment visualisation. Second, explanation techniques better suited to continuous control (e.g., attention-based saliency, trajectory summarisation, or reward decomposition) may offer clearer insights for complex policies.

The challenges of multi-agent systems also warrant dedicated focus. Future research could explore how to disambiguate policy intent across agents sharing models and actions, particularly in partially observable environments.

Importantly, incorporating formal user studies would allow for a human-centred validation of the explanations, ensuring that they are not only technically accurate but also practically useful. This includes testing visual overlays, symbolic rules, and saliency maps across different user types.

Finally, the development of hybrid frameworks—combining interpretable architectures, saliency methods, and causal models—within Unity may provide a more complete picture of agent behaviour, closing the gap between technical transparency and user-aligned understanding.

References

- [1] S. Ali *et al.*, “Explainable artificial intelligence (xai): What we know and what is left to attain trustworthy artificial intelligence,” *Information Fusion*, vol. 99, p. 101805, 2023, ISSN: 1566-2535. DOI: <https://doi.org/10.1016/j.inffus.2023.101805>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1566253523001148>.
- [2] L. Wells and T. Bednarz, “Explainable ai and reinforcement learning—a systematic review of current approaches and trends,” *Frontiers in Artificial Intelligence*, vol. 4, 2021, ISSN: 2624-8212. DOI: 10.3389/frai.2021.550030. [Online]. Available: <https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2021.550030>.
- [3] A. Juliani *et al.*, “Unity: A general platform for intelligent agents,” *arXiv preprint arXiv:1809.02627*, 2020. [Online]. Available: <https://arxiv.org/pdf/1809.02627.pdf>.
- [4] A. Heuillet, F. Couthouis, and N. Díaz-Rodríguez, *Explainability in deep reinforcement learning*, 2020. arXiv: 2008.06693 [cs.AI]. [Online]. Available: <https://arxiv.org/abs/2008.06693>.
- [5] S. Milani, N. Topin, M. Veloso, and F. Fang, *A survey of explainable reinforcement learning*, 2022. arXiv: 2202.08434 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2202.08434>.
- [6] R. S. Shefin, M. A. Rahman, T. Le, and S. Alqahtani, *Xsrl: Safety-aware explainable reinforcement learning – safety as a product of explainability*, 2024. arXiv: 2412.19311 [cs.AI]. [Online]. Available: <https://arxiv.org/abs/2412.19311>.
- [7] T. Hickling, A. Zenati, N. Aouf, and P. Spencer, *Explainability in deep reinforcement learning, a review into current methods and applications*, 2023. arXiv: 2207.01911 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2207.01911>.
- [8] N. Kibe, A. Okunola, and N. Beeb, “Evaluating the explainability of reinforcement learning decisions in real-time data stream analytics,” Apr. 2025.
- [9] I. Buyuksalih, S. Bayburt, G. Buyuksalih, A. Baskaraca, H. Karim, and A. Rahman, “3d modelling and visualization based on the unity game engine – advantages and challenges,” vol. IV-4/W4, Nov. 2017, pp. 161–166. DOI: 10.5194/isprs-annals-IV-4-W4-161-2017.
- [10] A. Hussain, H. Shakeel, F. Hussain, N. Uddin, and T. Ghouri, “Unity game development engine: A technical survey,” *University of Sindh Journal of Information and Communication Technology*, vol. 4, Oct. 2020.

- [11] T. K. Mohd, F. Bravo-Garcia, L. Love, M. Gujadhur, and J. Nyadu, "Analyzing strengths and weaknesses of modern game engines," *International Journal of Computer Theory and Engineering*, vol. 15, no. 1, pp. 54–60, 2023, ISSN: 1793-8201. DOI: 10.7763/ijcte.2023.v15.1330. [Online]. Available: <http://dx.doi.org/10.7763/IJCTE.2023.V15.1330>.
- [12] U. Technologies, *ML-Agents Overview - Unity ML-Agents Toolkit* – unity-technologies.github.io, [Accessed 10-05-2025], 2025. [Online]. Available: <https://unity-technologies.github.io/ml-agents/ML-Agents-Overview/#key-components>.
- [13] L. Almón-Manzano, R. Pastor-Vargas, and J. M. C. Troncoso, "Deep reinforcement learning in agents' training: Unity ml-agents," in *Bio-inspired Systems and Applications: from Robotics to Ambient Intelligence*, J. M. Ferrández Vicente, J. R. Álvarez-Sánchez, F. de la Paz López, and H. Adeli, Eds., Cham: Springer International Publishing, 2022, pp. 391–400, ISBN: 978-3-031-06527-9.
- [14] L. P. Kaelbling, M. L. Littman, and A. W. Moore, "Reinforcement learning: A survey," *J. Artif. Intell. Res.*, vol. 4, pp. 237–285, May 1996.
- [15] Z. Ding *et al.*, "Introduction to reinforcement learning," eng, in *Deep Reinforcement Learning*, Singapore: Springer Singapore, 2020, pp. 47–123, ISBN: 9811540942.
- [16] A. Plaat, *Deep Reinforcement Learning*. Springer Nature Singapore, 2022, ISBN: 9789811906381. DOI: 10.1007/978-981-19-0638-1. [Online]. Available: <http://dx.doi.org/10.1007/978-981-19-0638-1>.
- [17] A. G. Barto, "Reinforcement learning: An introduction. by richard's sutton," *SIAM Rev*, vol. 6, no. 2, p. 423, 2021.
- [18] W. C. Cheung, "Exploration-exploitation trade-off in reinforcement learning on online markov decision processes with global concave rewards," eng, *arXiv.org*, 2019, ISSN: 2331-8422.
- [19] R. Sutton, D. Mcallester, S. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," *Adv. Neural Inf. Process. Syst*, vol. 12, Feb. 2000.
- [20] I. Grondman, L. Busoniu, G. A. D. Lopes, and R. Babuska, "A survey of actor-critic reinforcement learning: Standard and natural policy gradients," *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 42, no. 6, pp. 1291–1307, 2012. DOI: 10.1109/TSMCC.2012.2218595.
- [21] A. Plaat, *Deep reinforcement learning*, eng. Singapore: Springer Nature Singapore Pte Ltd., 2022, ISBN: 9789811906381.

- [22] R. Kozlica, S. Wegenkittl, and S. Hirländer, *Deep q-learning versus proximal policy optimization: Performance comparison in a material sorting task*, 2023. arXiv: 2306.01451 [cs.AI]. [Online]. Available: <https://arxiv.org/abs/2306.01451>.
- [23] S. Munikoti, D. Agarwal, L. Das, M. Halappanavar, and B. Natarajan, *Challenges and opportunities in deep reinforcement learning with graph neural networks: A comprehensive review of algorithms and applications*, 2022. arXiv: 2206.07922 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2206.07922>.
- [24] A. Somani, A. Horsch, and D. K. Prasad, *Interpretability in Deep Learning*, eng, First edition. Cham, Switzerland: Springer, 2023, ISBN: 9783031206399.
- [25] P. Linardatos, V. Papastefanopoulos, and S. Kotsiantis, "Explainable ai: A review of machine learning interpretability methods," *Entropy*, vol. 23, no. 1, 2021, ISSN: 1099-4300. DOI: 10.3390/e23010018. [Online]. Available: <https://www.mdpi.com/1099-4300/23/1/18>.
- [26] A. Puthanveetil Madathil et al., "Intrinsic and post-hoc xai approaches for fingerprint identification and response prediction in smart manufacturing processes," *Journal of Intelligent Manufacturing*, vol. 35, no. 8, pp. 4159–4180, Feb. 2024, ISSN: 1572-8145. DOI: 10.1007/s10845-023-02266-2. [Online]. Available: <http://dx.doi.org/10.1007/s10845-023-02266-2>.
- [27] C. Glanois et al., *A survey on interpretable reinforcement learning*, 2022. arXiv: 2112.13112 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2112.13112>.
- [28] S. Lundberg and S.-I. Lee, *A unified approach to interpreting model predictions*, 2017. arXiv: 1705.07874 [cs.AI]. [Online]. Available: <https://arxiv.org/abs/1705.07874>.
- [29] J. Narkhede, "Comparative evaluation of post-hoc explainability methods in ai: Lime, shap, and grad-cam," in *2024 4th International Conference on Sustainable Expert Systems (ICSES)*, 2024, pp. 826–830. DOI: 10.1109/ICSES63445.2024.10762963.
- [30] D. Vale and A. El-Sharif, "Explainable artificial intelligence (xai) post-hoc explainability methods: Risks and limitations in non-discrimination law," *AI and Ethics*, vol. 2, pp. 1–12, Mar. 2022. DOI: 10.1007/s43681-022-00142-y.
- [31] Y. Bekkemoen, "Explainable reinforcement learning (xrl): A systematic literature review and taxonomy," *Machine Learning*, vol. 113, no. 1, pp. 355–441, Nov. 2023, ISSN: 1573-0565. DOI: 10.1007/s10994-023-06479-7. [Online]. Available: <http://dx.doi.org/10.1007/s10994-023-06479-7>.

- [32] P. Sequeira and M. Gervasio, "Interestingness elements for explainable reinforcement learning: Understanding agents' capabilities and limitations," en, *Artificial Intelligence*, vol. 288, p. 103 367, Nov. 2020, ISSN: 00043702. DOI: 10.1016/j.artint.2020.103367. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S000437022030117X> (visited on 05/12/2025).
- [33] G. Vilone and L. Longo, "Notions of explainability and evaluation approaches for explainable artificial intelligence," *Information Fusion*, vol. 76, pp. 89–106, 2021, ISSN: 1566-2535. DOI: <https://doi.org/10.1016/j.inffus.2021.05.009>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1566253521001093>.
- [34] B. Hayes and J. A. Shah, "Improving robot controller transparency through autonomous policy explanation," in *Proceedings of the 2017 ACM/IEEE International Conference on Human-Robot Interaction*, ser. HRI '17, Vienna, Austria: Association for Computing Machinery, 2017, pp. 303–312, ISBN: 9781450343367. DOI: 10.1145/2909824.3020233. [Online]. Available: <https://doi.org/10.1145/2909824.3020233>.
- [35] T. Hachaj and M. Piekarczyk, "On explainability of reinforcement learning-based machine learning agents trained with proximal policy optimization that utilizes visual sensor data," *Applied Sciences*, vol. 15, no. 2, 2025, ISSN: 2076-3417. DOI: 10.3390/app15020538. [Online]. Available: <https://www.mdpi.com/2076-3417/15/2/538>.
- [36] P. Madumal, T. Miller, L. Sonenberg, and F. Vetere, *Explainable Reinforcement Learning Through a Causal Lens*, arXiv:1905.10958 [cs], Nov. 2019. DOI: 10.48550/arXiv.1905.10958. [Online]. Available: <http://arxiv.org/abs/1905.10958> (visited on 05/12/2025).
- [37] A. Verma, V. Murali, R. Singh, P. Kohli, and S. Chaudhuri, "Programmatically interpretable reinforcement learning," in *Proceedings of the 35th International Conference on Machine Learning*, 2018, pp. 5052–5061.
- [38] Z. Juozapaitis, A. Koul, A. Fern, M. Erwig, and F. Doshi-Velez, "Explainable reinforcement learning via reward decomposition," eng, 2019.
- [39] H. Itaya, W. Yin, T. Hirakawa, T. Yamashita, H. Fujiyoshi, and K. Sugiura, "Visual Explanation With Action Query Transformer in Deep Reinforcement Learning and Visual Feedback via Augmented Reality," *IEEE Access*, vol. 13, pp. 56 338–56 354, 2025, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2025.3554493. [Online]. Available:

- <https://ieeexplore.ieee.org/abstract/document/10938565> (visited on 05/12/2025).
- [40] L. He, N. Aouf, and B. Song, "Explainable Deep Reinforcement Learning for UAV autonomous path planning," en, *Aerospace Science and Technology*, vol. 118, p. 107 052, Nov. 2021, ISSN: 12709638. DOI: 10.1016/j.ast.2021.107052. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S1270963821005629> (visited on 05/12/2025).
 - [41] F. Cruz, R. Dazeley, P. Vamplew, and I. Moreira, "Explainable robotic systems: Understanding goal-driven actions in a reinforcement learning scenario," *Neural Computing and Applications*, vol. 35, no. 25, pp. 18 113–18 130, Sep. 2023, arXiv:2006.13615 [cs], ISSN: 0941-0643, 1433-3058. DOI: 10.1007/s00521-021-06425-5. [Online]. Available: <http://arxiv.org/abs/2006.13615> (visited on 05/12/2025).
 - [42] X. Wang, Y. Chen, J. Yang, L. Wu, Z. Wu, and X. Xie, "A Reinforcement Learning Framework for Explainable Recommendation," in *2018 IEEE International Conference on Data Mining (ICDM)*, Singapore: IEEE, Nov. 2018, pp. 587–596, ISBN: 978-1-5386-9159-5. DOI: 10.1109/ICDM.2018.00074. [Online]. Available: <https://ieeexplore.ieee.org/document/8594883/> (visited on 05/12/2025).
 - [43] X. Peng, M. O. Riedl, and P. Ammanabrolu, *Inherently explainable reinforcement learning in natural language*, 2022. arXiv: 2112.08907 [cs.HC]. [Online]. Available: <https://arxiv.org/abs/2112.08907>.
 - [44] M. L. Olson, R. Khanna, L. Neal, F. Li, and W.-K. Wong, "Counterfactual state explanations for reinforcement learning agents via generative deep learning," *Artificial Intelligence*, vol. 295, p. 103 455, Jun. 2021, ISSN: 0004-3702. DOI: 10.1016/j.artint.2021.103455. [Online]. Available: <http://dx.doi.org/10.1016/j.artint.2021.103455>.
 - [45] C. A. Cruz and T. Igarashi, *Interactive explanations: Diagnosis and repair of reinforcement learning based agent behaviors*, 2021. arXiv: 2105.12938 [cs.HC]. [Online]. Available: <https://arxiv.org/abs/2105.12938>.
 - [46] U. Ehsan, B. Harrison, L. Chan, and M. O. Riedl, *Rationalization: A neural machine translation approach to generating natural language explanations*, 2017. arXiv: 1702.07826 [cs.AI]. [Online]. Available: <https://arxiv.org/abs/1702.07826>.
 - [47] *ML-agents/docs/Learning-Environment-Examples.md at develop · Unity-Technologies/ml-agents · GitHub*. [Online]. Available: <https://github.com/Unity-Technologies/ml->

`agents/blob/develop/docs/Learning-Environment-Examples.md` (visited on 05/13/2025).

Appendix A Agent Training

This section contains the hyperparameters used within each training environment used for this dissertation.

A.1 Pyramids Hyperparameters

```
1 default_settings: null
2 behaviors:
3   Pyramids:
4     trainer_type: ppo
5     hyperparameters:
6       batch_size: 1024
7       buffer_size: 10240
8       learning_rate: 0.0003
9       beta: 0.005
10      epsilon: 0.2
11      lambda: 0.95
12      num_epoch: 3
13      shared_critic: false
14      learning_rate_schedule: linear
15      beta_schedule: linear
16      epsilon_schedule: linear
17      checkpoint_interval: 500000
18      network_settings:
19        normalize: false
20        hidden_units: 128
21        num_layers: 2
22        vis_encode_type: simple
23        memory: null
24        goal_conditioning_type: hyper
25        deterministic: false
26      reward_signals:
27        extrinsic:
28          gamma: 0.99
29          strength: 1.0
30          network_settings:
31            normalize: false
32            hidden_units: 128
33            num_layers: 2
34            vis_encode_type: simple
35            memory: null
36            goal_conditioning_type: hyper
37            deterministic: false
```

```

38     init_path: null
39     keep_checkpoints: 5
40     even_checkpoints: false
41     max_steps: 5000000
42     time_horizon: 64
43     summary_freq: 50000
44     threaded: false
45     self_play: null
46     behavioral_cloning: null
47 env_settings:
48     env_path: null
49     env_args: null
50     base_port: 5005
51     num_envs: 1
52     num_areas: 1
53     timeout_wait: 60
54     seed: -1
55     max_lifetime_restarts: 10
56     restarts_rate_limit_n: 1
57     restarts_rate_limit_period_s: 60
58 engine_settings:
59     width: 84
60     height: 84
61     quality_level: 5
62     time_scale: 20.0
63     target_frame_rate: -1
64     capture_frame_rate: 60
65     no_graphics: false
66     no_graphics_monitor: false
67 environment_parameters: null
68 checkpoint_settings:
69     run_id: buttonReward
70     initialize_from: null
71     load_model: false
72     resume: false
73     force: true
74     train_model: false
75     inference: false
76     results_dir: results
77 torch_settings:
78     device: null
79 debug: false

```

A.2 Dungeon Escape Hyperparameters

```

1 default_settings: null
2 behaviors:
3   DungeonEscape:
4     trainer_type: poca
5     hyperparameters:
6       batch_size: 2048
7       buffer_size: 81920
8       learning_rate: 0.0001
9       beta: 0.001
10      epsilon: 0.2
11      lambda: 0.95
12      num_epoch: 4
13      learning_rate_schedule: linear
14      beta_schedule: linear
15      epsilon_schedule: linear
16      checkpoint_interval: 500000
17      network_settings:
18        normalize: true
19        hidden_units: 512
20        num_layers: 4
21        vis_encode_type: simple
22        memory: null
23        goal_conditioning_type: hyper
24        deterministic: false
25      reward_signals:
26        extrinsic:
27          gamma: 0.98
28          strength: 2.0
29          network_settings:
30            normalize: true
31            hidden_units: 512
32            num_layers: 4
33            vis_encode_type: simple
34            memory: null
35            goal_conditioning_type: hyper
36            deterministic: false
37      init_path: null
38      keep_checkpoints: 5
39      even_checkpoints: false
40      max_steps: 5000000
41      time_horizon: 1024
42      summary_freq: 50000
43      threaded: false
44      self_play: null
45      behavioral_cloning: null
46 env_settings:
47   env_path: dev_build/UnityEnvironment

```

```

48  env_args: null
49  base_port: 5005
50  num_envs: 1
51  num_areas: 1
52  timeout_wait: 60
53  seed: -1
54  max_lifetime_restarts: 10
55  restarts_rate_limit_n: 1
56  restarts_rate_limit_period_s: 60
57  engine_settings:
58    width: 84
59    height: 84
60    quality_level: 5
61    time_scale: 20
62    target_frame_rate: -1
63    capture_frame_rate: 60
64    no_graphics: false
65    no_graphics_monitor: false
66  environment_parameters: null
67  checkpoint_settings:
68    run_id: dunTestNoRB
69    initialize_from: null
70    load_model: false
71    resume: false
72    force: true
73    train_model: false
74    inference: false
75    results_dir: results
76  torch_settings:
77    device: null
78  debug: true

```

A.3 Food Collector Hyperparameters

```

1  default_settings: null
2  behaviors:
3    GridFoodCollector:
4      trainer_type: ppo
5      hyperparameters:
6        batch_size: 1024
7        buffer_size: 40960
8        learning_rate: 0.0003
9        beta: 0.005
10       epsilon: 0.2
11       lambda: 0.95

```

```

12     num_epoch: 3
13     shared_critic: false
14     learning_rate_schedule: linear
15     beta_schedule: linear
16     epsilon_schedule: linear
17     checkpoint_interval: 500000
18     network_settings:
19         normalize: false
20         hidden_units: 256
21         num_layers: 3
22         vis_encode_type: simple
23         memory: null
24         goal_conditioning_type: hyper
25         deterministic: false
26     reward_signals:
27         extrinsic:
28             gamma: 0.99
29             strength: 1.0
30             network_settings:
31                 normalize: false
32                 hidden_units: 256
33                 num_layers: 3
34                 vis_encode_type: simple
35                 memory: null
36                 goal_conditioning_type: hyper
37                 deterministic: false
38     init_path: null
39     keep_checkpoints: 5
40     even_checkpoints: false
41     max_steps: 2000000
42     time_horizon: 128
43     summary_freq: 50000
44     checkpoint_interval: 500000
45     keep_checkpoints: 5
46     threaded: false
47     self_play: null
48     behavioral_cloning: null
49 env_settings:
50     env_path: null
51     env_args: null
52     base_port: 5005
53     num_envs: 1
54     num_areas: 1
55     timeout_wait: 60
56     seed: -1
57     max_lifetime_restarts: 10
58     restarts_rate_limit_n: 1

```

```
59   restarts_rate_limit_period_s: 60
60 engine_settings:
61   width: 84
62   height: 84
63   quality_level: 5
64   time_scale: 20
65   target_frame_rate: -1
66   capture_frame_rate: 60
67   no_graphics: false
68   no_graphics_monitor: false
69 environment_parameters: null
70 checkpoint_settings:
71   run_id: foodTest
72   initialize_from: null
73   load_model: false
74   resume: false
75   force: true
76   train_model: false
77   inference: false
78   results_dir: results
79 torch_settings:
80   device: null
81 debug: true
```

A.4 Environment Descriptions from Unity Documentation

The following figures outline the original environment specifications from the ML-Agents Documentation [3] for the Pyramids, Dungeon Escape and Food Collector environments:

A.4.1 Pyramids

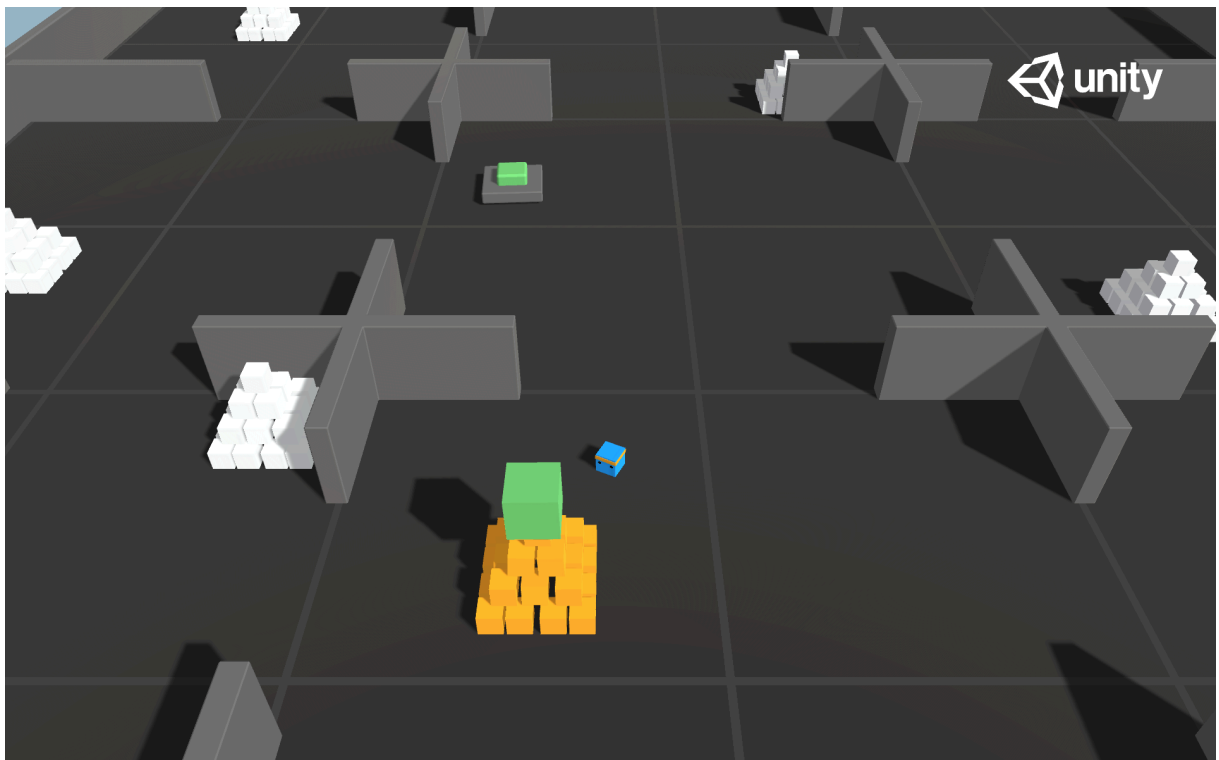


Figure A.1 The Pyramids environment.

- **Set-up:** The agent must press a button to spawn a pyramid, navigate to it, knock it over, and reach the golden brick previously placed at the top.
- **Goal:** Reach the golden brick on top of the spawned pyramid.
- **Agents:** Single-agent environment.
- **Agent Reward Function (independent):**
 - +2 for reaching the golden brick.
 - -0.001 per step to penalise delay.
- **Behaviour Parameters:**

- **Observation Space:** 148-dimensional vector observation composed of raycasts detecting Switch, Bricks, Golden Brick, and Walls, along with a scalar value indicating switch state.
- **Actions:** One discrete action branch with four actions: rotate left, rotate right, move forward, and move backward.
- **Float Properties:** None
- **Benchmark Mean Reward:** 1.75

A.4.2 Dungeon Escape

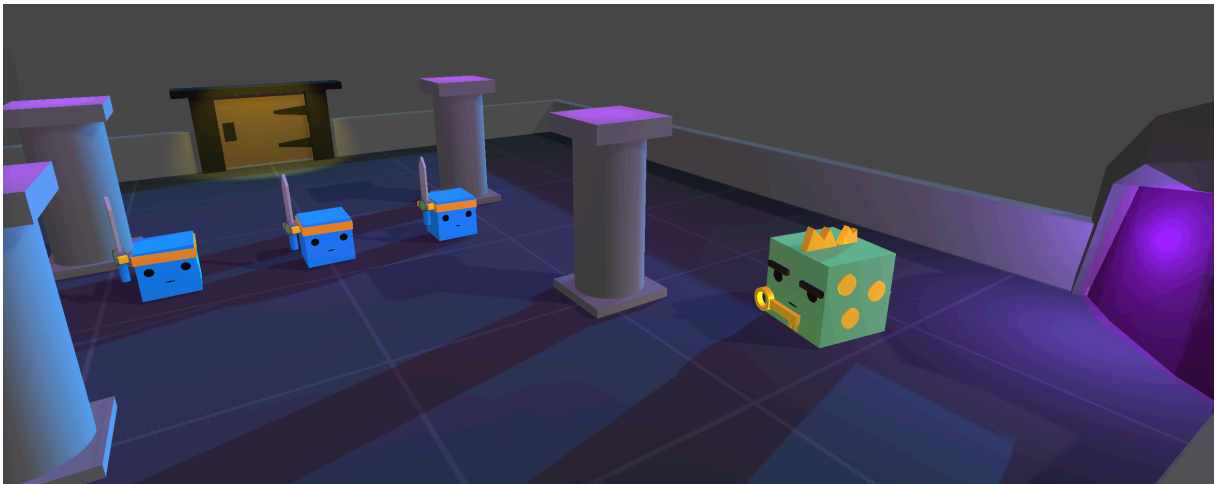


Figure A.2 The Dungeon Escape environment.

- **Set-up:** Agents are trapped in a dungeon with a dragon and must work together to escape. One agent must locate and slay the dragon, sacrificing itself to drop a key. The remaining agents can then retrieve this key and unlock the dungeon door. If the agents take too long, the dragon escapes through a portal and the environment resets.
- **Goal:** Unlock the dungeon door and escape.
- **Agents:** The environment includes three agents grouped in a multi-agent group and one dragon with fixed movement logic.
- **Agent Reward Function:**
 - +1 group reward if any agent successfully unlocks the door and escapes.
- **Behaviour Parameters:**

- **Observation Space:** Ray Perception Sensor detecting tagged objects including: Wall, Agent, Dragon, Key, Lock, and Portal. Also includes a vector observation indicating key possession.
- **Actions:** One discrete action branch with 7 options: turn left, turn right, move forward, move backward, strafe left, strafe right, or no-op.
- **Float Properties:** None
- **Benchmark Mean Reward:** 1.0 (Group reward)

A.4.3 Food Collector

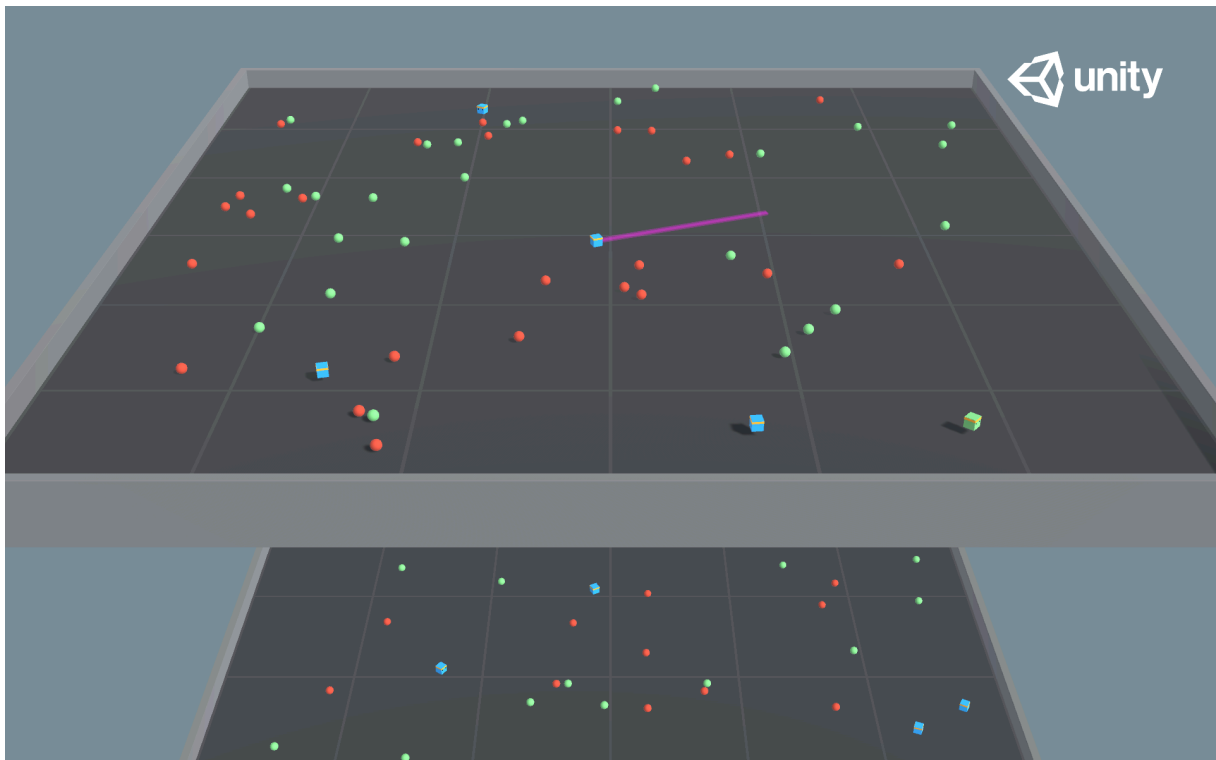


Figure A.3 The Food Collector environment.

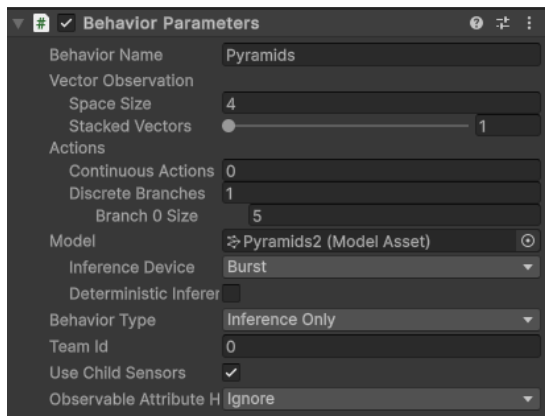
- **Set-up:** A multi-agent environment where agents compete to collect food.
- **Goal:** Agents must learn to collect as many green food spheres as possible while avoiding red spheres.
- **Agents:** The environment contains 5 agents with identical Behaviour Parameters.
- **Agent Reward Function (independent):**
 - +1 for interaction with green spheres

- -1 for interaction with red spheres
- **Behaviour Parameters:**
 - **Vector Observation space:** 53-dimensional, consisting of:
 - * Agent velocity (2 values)
 - * Frozen and laser-shot flags (2 values)
 - * Grid-based perception: 40×40 grid with 6 object categories
 - **Actions:**
 - * 3 continuous actions: forward motion, side motion, and rotation
 - * 1 discrete action branch for laser firing (2 possible actions: shoot or no action)
 - **Visual Observations (optional):**
 - * First-person camera per agent
 - * One vector flag representing the frozen state
 - * Training requires both visual input and frozen flag vector; use the `VisualFoodCollector` scene
- **Float Properties:**
 - `laser_length`: Length of the laser used by the agent
 - * Default: 1
 - * Recommended range: 0.2 – 7
 - `agent_scale`: Scale of the agent (equal across all dimensions)
 - * Default: 1
 - * Recommended range: 0.5 – 5
- **Benchmark Mean Reward: 10**

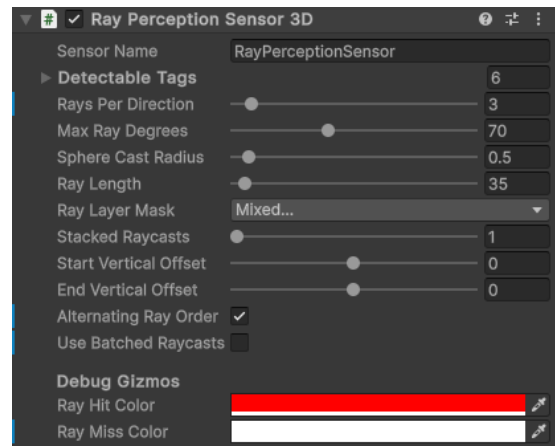
A.5 In-Editor Agent Parameters

Below are the in-editor parameters assigned to the components for the RL agents used within this dissertation.

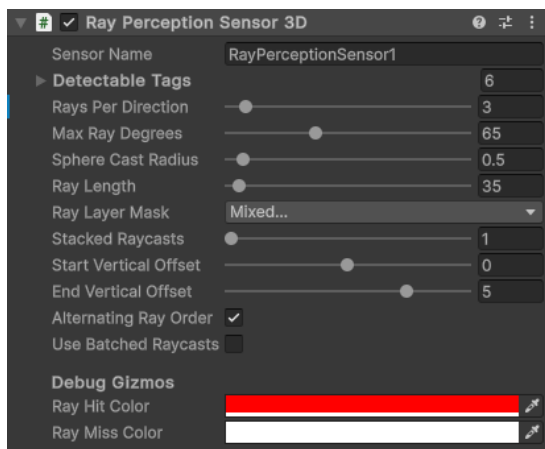
A.5.1 Pyramids



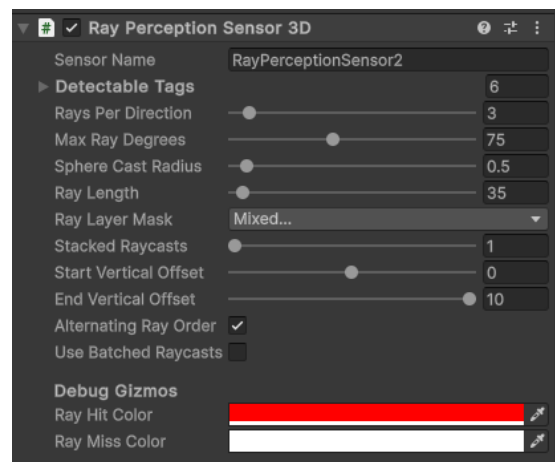
(a) Behaviour Parameters



(b) Ray Sensor 0



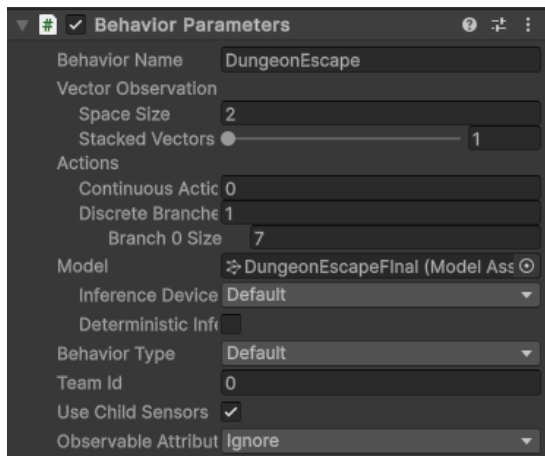
(c) Ray Sensor 1



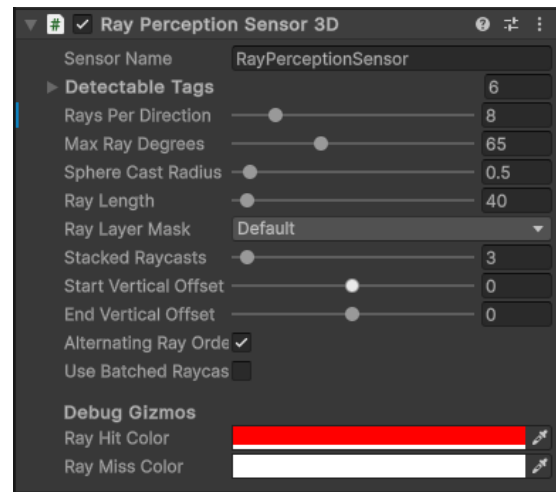
(d) Ray Sensor 2

Figure A.4 Configuration screenshots for the Pyramids agent and its three stacked ray perception sensors.

A.5.2 Dungeon Escape



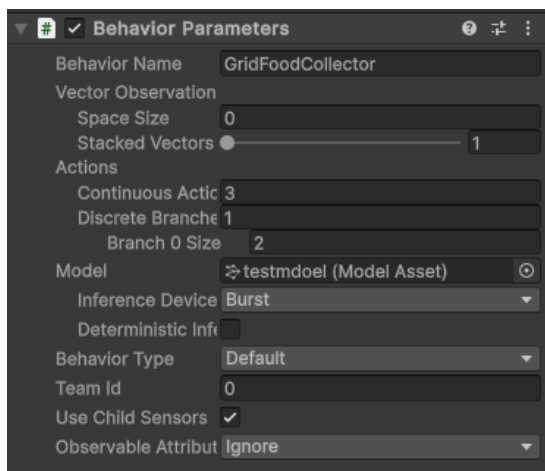
(a) Behaviour Parameters



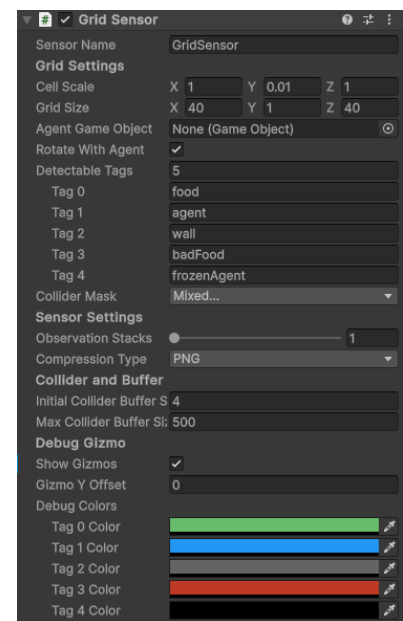
(b) Ray Sensor

Figure A.5 Configuration screenshots for the Dungeon Escape agent and its ray perception sensor.

A.5.3 Food Collector



(a) Behaviour Parameters



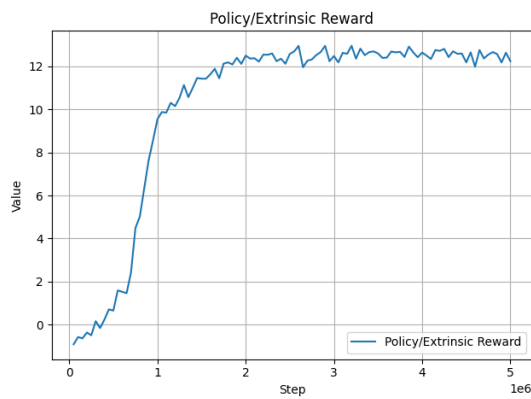
(b) Grid Sensor

Figure A.6 Configuration screenshots for the Food Collector agent and its Grid sensor.

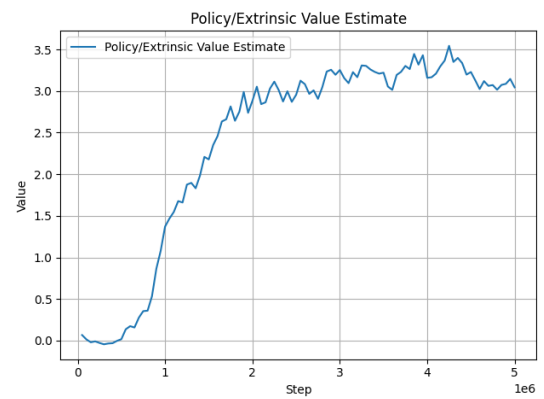
A.6 Training Results

When training with UnityML Agents, a `TensorBoard.tfevents` file is generated, containing all the data logged during training by the toolkit. This section will display all the graphs able to be plotted using the data from these files.

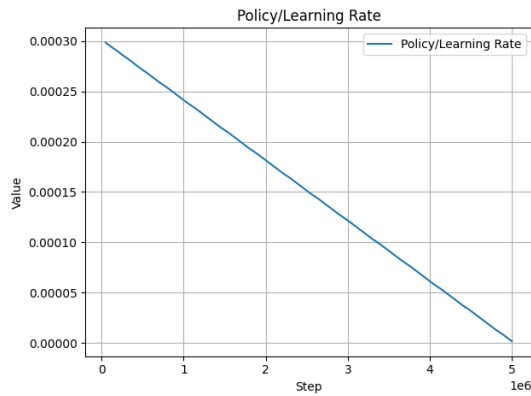
A.6.1 Pyramids



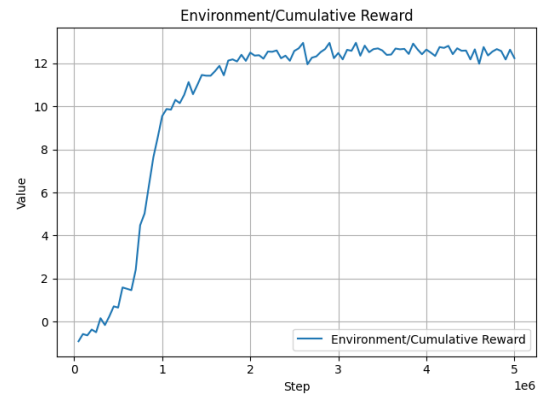
(a) Policy/Extrinsic Reward



(b) Policy/Extrinsic Value Estimate

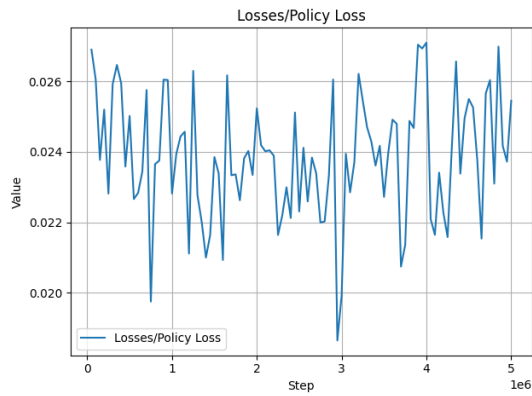


(c) Policy/Learning Rate

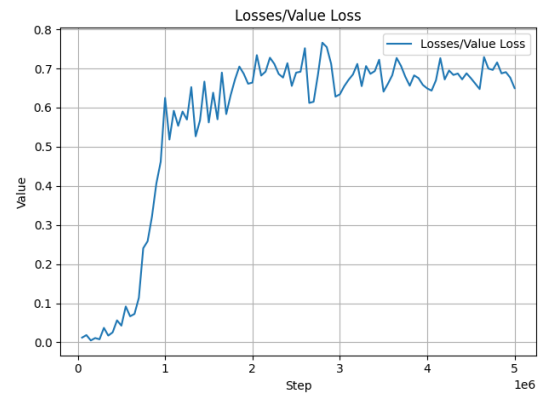


(d) Environment/Cumulative Reward

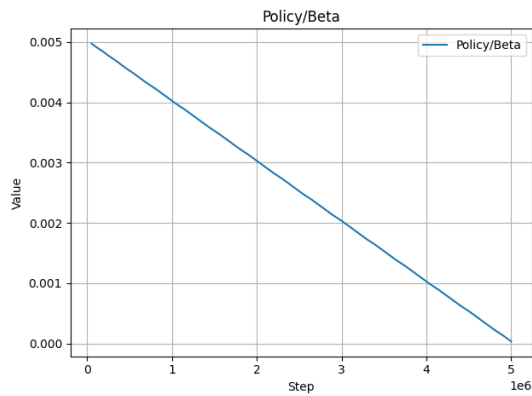
Figure A.7 Training metrics for the `Pyramids` environment (1/2).



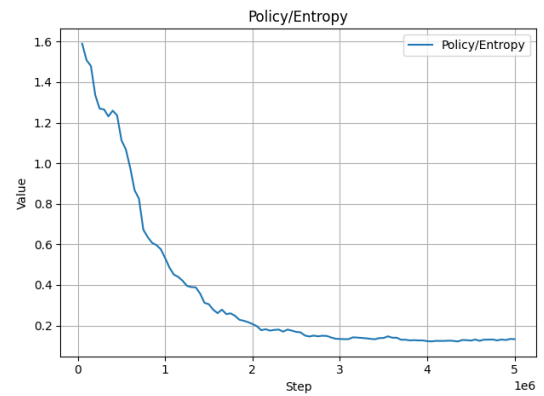
(e) Losses/Policy Loss



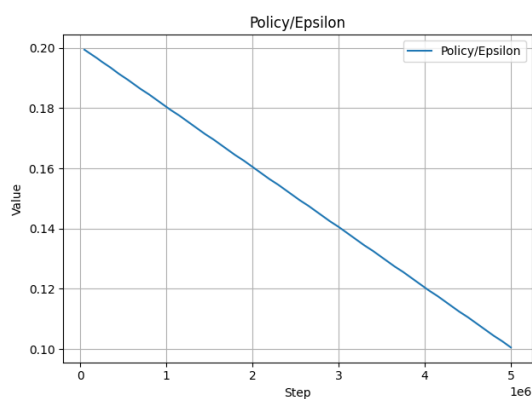
(f) Losses/Value Loss



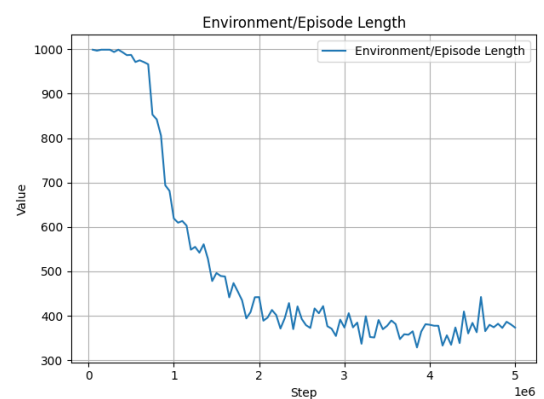
(g) Policy/Beta



(h) Policy/Entropy



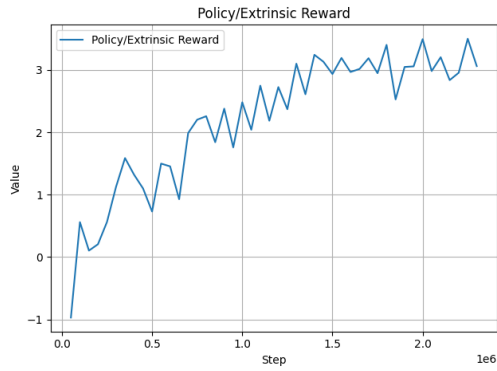
(i) Policy/Epsilon



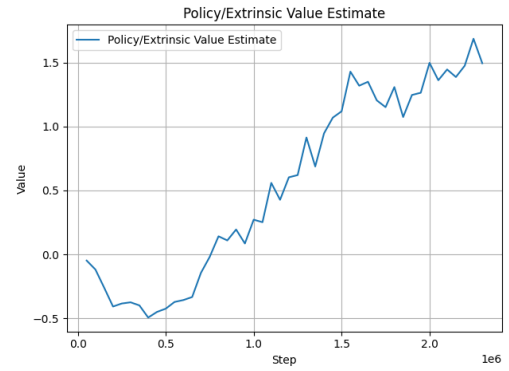
(j) Environment/Episode Length

Figure A.7 Training metrics for the Pyramids environment (2/2).

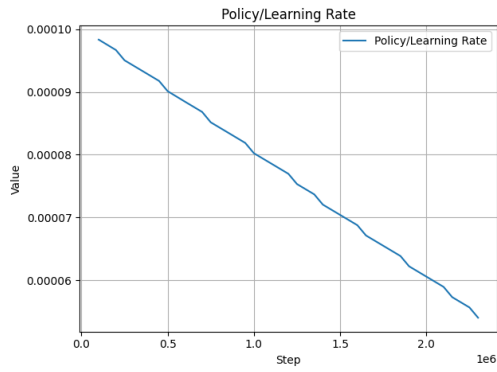
A.6.2 Dungeon Escape



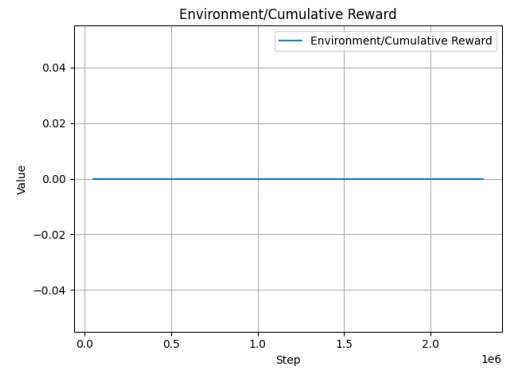
(a) Policy/Extrinsic Reward



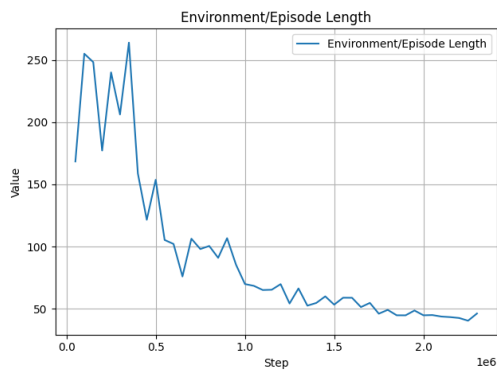
(b) Policy/Extrinsic Value Estimate



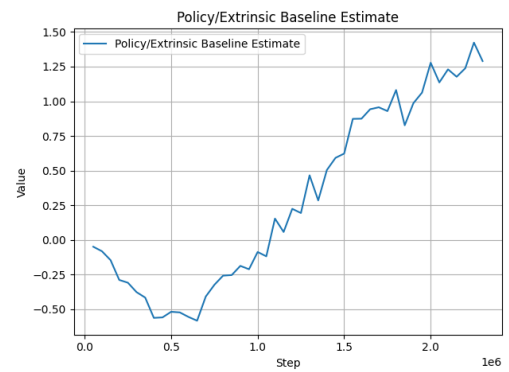
(c) Policy/Learning Rate



(d) Environment/Cumulative Reward

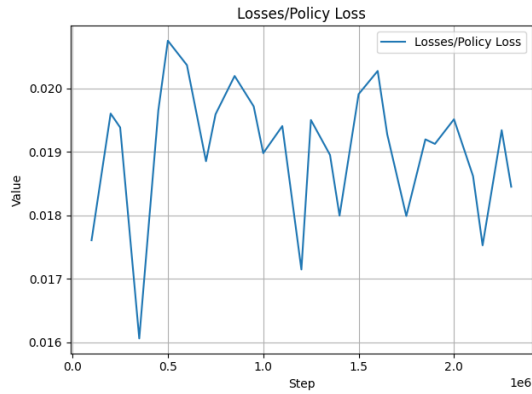


(e) Environment/Episode Length

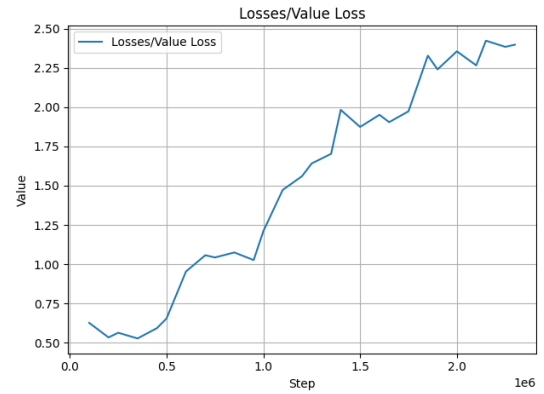


(f) Policy/Extrinsic Baseline Estimate

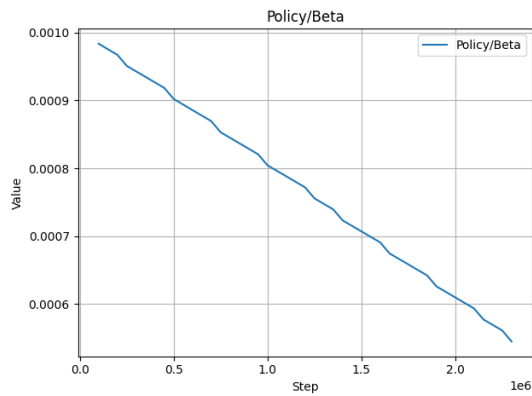
Figure A.8 Training metrics for the Dungeon Escape environment (1/2).



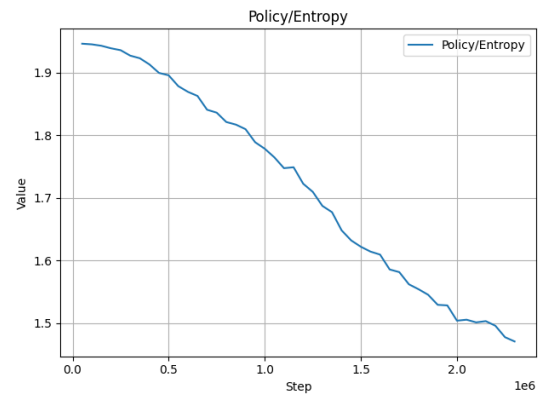
(g) Losses/Policy Loss



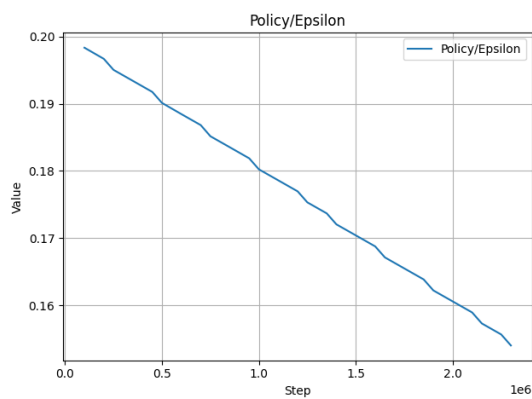
(h) Losses/Value Loss



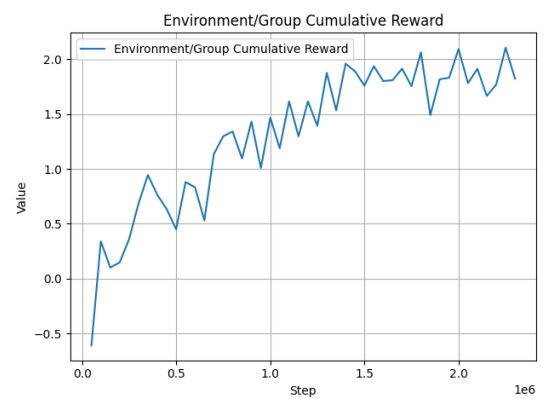
(i) Policy/Beta



(j) Policy/Entropy



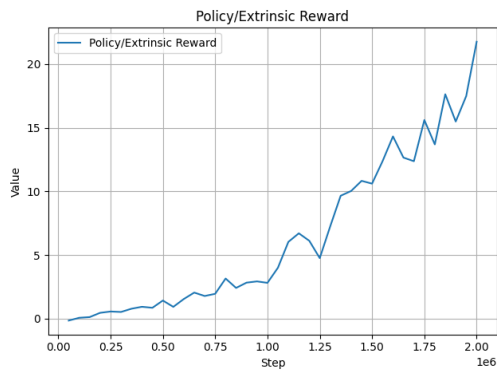
(k) Policy/Epsilon



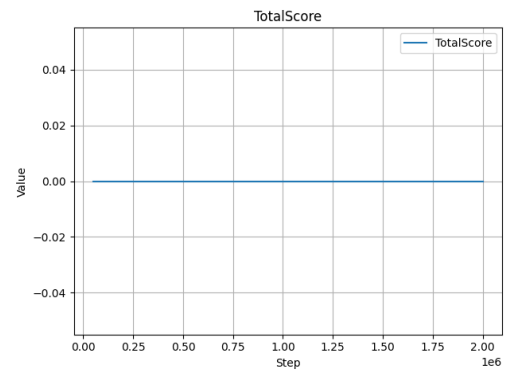
(l) Environment/Group Cumulative Reward

Figure A.8 Training metrics for the Dungeon Escape environment (2/2).

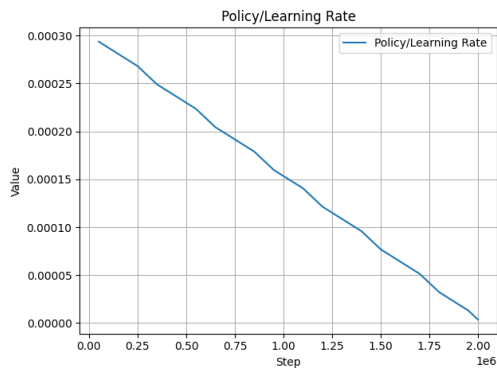
A.6.3 Food Collector



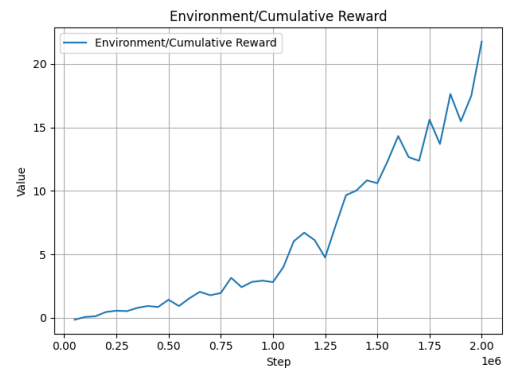
(a) Policy/Extrinsic Reward



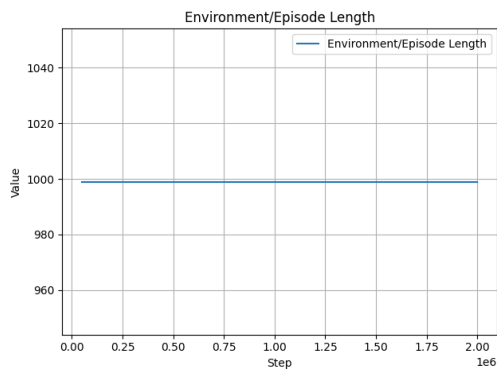
(b) Total Score



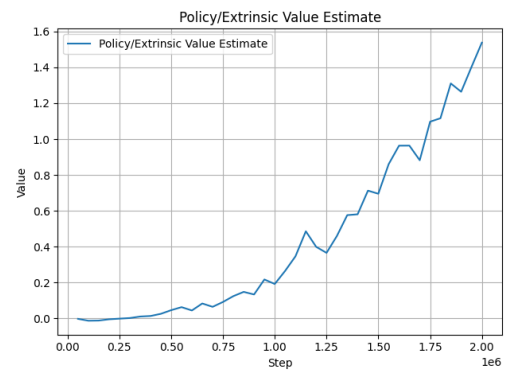
(c) Policy/Learning Rate



(d) Environment/Cumulative Reward

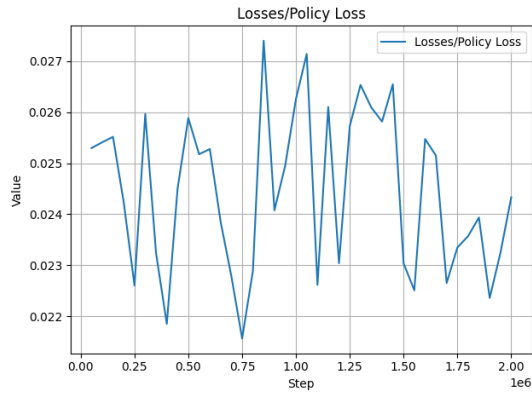


(e) Environment/Episode Length

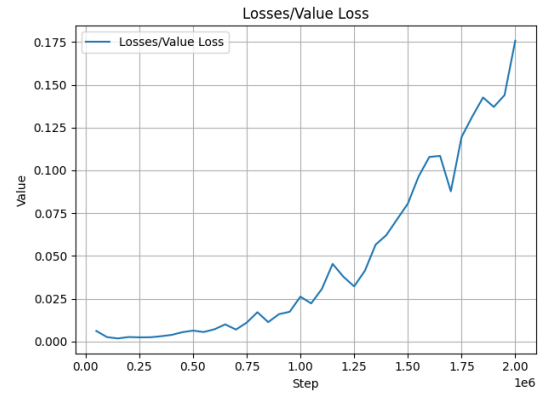


(f) Policy/Extrinsic Value Estimate

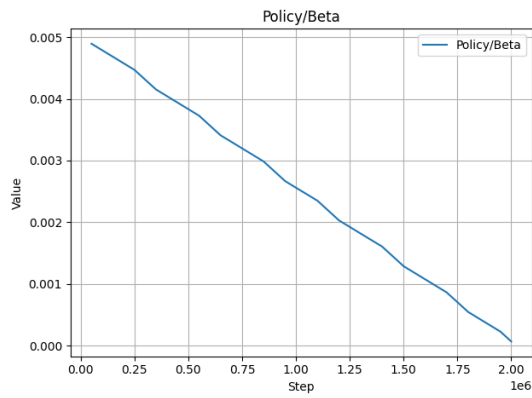
Figure A.9 Training metrics for the Food Collector environment (1/2).



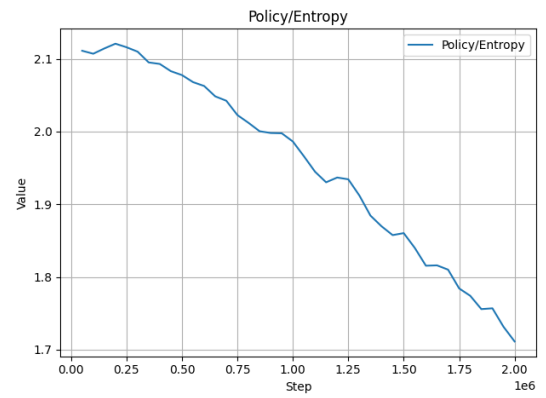
(g) Losses/Policy Loss



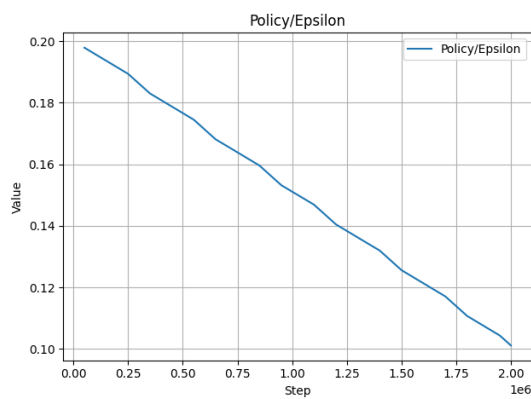
(h) Losses/Value Loss



(i) Policy/Beta



(j) Policy/Entropy



(k) Policy/Epsilon

Figure A.9 Training metrics for the Food Collector environment (2/2).

Appendix B Text-Based Decision Tree Explanation

This chapter will contain the full text-based explanations generated from the Food Collector decision tree surrogates. There will be an explanation for discrete actions and an explanation for continuous actions.

B.1 Discrete Actions Explanation

```
1 IF BadFood occupies 3% of patch (3,2):
2   IF Food occupies 34% of patch (4,0):
3     IF BadFood occupies 59% of patch (6,3):
4       IF BadFood occupies 41% of patch (2,7):
5         IF FrozenAgent occupies 3% of patch (3,5):
6           THEN class 0
7         ELSE:
8           THEN class 1
9       ELSE:
10        IF Wall occupies 28% of patch (8,4):
11          THEN class 1
12        ELSE:
13          THEN class 0
14      ELSE:
15        IF BadFood occupies 12% of patch (2,8):
16          THEN class 0
17        ELSE:
18          THEN class 1
19      ELSE:
20        IF Food occupies 9% of patch (6,4):
21          IF Food occupies 6% of patch (4,6):
22            IF Wall occupies 3% of patch (9,5):
23              THEN class 1
24            ELSE:
25              THEN class 0
26          ELSE:
27            THEN class 0
28        ELSE:
29          THEN class 0
30      ELSE:
31        IF Wall occupies 41% of patch (4,1):
32          IF Wall occupies 59% of patch (0,3):
33            IF Wall occupies 16% of patch (0,8):
34              IF BadFood occupies 16% of patch (4,7):
```

```

35         THEN class 1
36     ELSE:
37         THEN class 1
38     ELSE:
39         IF Wall occupies 34% of patch (2,8):
40             THEN class 0
41         ELSE:
42             THEN class 1
43     ELSE:
44         IF Wall occupies 28% of patch (2,2):
45             IF Food occupies 31% of patch (8,4):
46                 THEN class 0
47             ELSE:
48                 THEN class 1
49         ELSE:
50             IF BadFood occupies 16% of patch (3,2):
51                 THEN class 1
52             ELSE:
53                 THEN class 0
54     ELSE:
55         IF Wall occupies 47% of patch (4,1):
56             THEN class 1
57         ELSE:
58             IF Food occupies 9% of patch (4,2):
59                 IF BadFood occupies 28% of patch (2,2):
60                     THEN class 1
61                 ELSE:
62                     THEN class 0
63         ELSE:
64             IF Food occupies 12% of patch (6,0):
65                 THEN class 0
66             ELSE:
67                 THEN class 1

```

B.2 Continuous Actions Explanation

```

1 IF [Wall] at (0,9) <= 0.34:
2   IF [Wall] at (0,6) <= 0.28:
3     IF [Wall] at (0,5) <= 0.41:
4       IF [Wall] at (1,4) <= 0.09:
5         IF [Wall] at (0,2) <= 0.34:
6           IF [Wall] at (1,0) <= 0.28:
7             IF [Wall] at (1,8) <= 0.34:
8               THEN Move Backward + Strafe Right + Turn Left
9             ELSE:
10              THEN Move Backward + Strafe No Action + Turn Left
11            ELSE:
12              IF [Wall] at (0,1) <= 0.09:
13                THEN Move Backward + Strafe No Action + Turn Left
14              ELSE:
15                THEN Move Backward + Strafe Right + Turn Left
16            ELSE:
17              IF [Wall] at (1,2) <= 0.22:
18                IF [Wall] at (0,0) <= 0.09:
19                  THEN Move Backward + Strafe Right + Turn Left
20                ELSE:
21                  THEN Move Backward + Strafe Right + Turn No Action
22              ELSE:
23                IF [Wall] at (3,3) <= 0.22:
24                  THEN Move Backward + Strafe Right + Turn Left
25                ELSE:
26                  THEN Move Backward + Strafe No Action + Turn Left
27            ELSE:
28              IF [Wall] at (0,4) <= 0.34:
29                IF [Wall] at (1,1) <= 0.22:
30                  IF [Wall] at (0,2) <= 0.03:
31                    THEN Move Backward + Strafe Left + Turn Left
32                  ELSE:
33                    THEN Move Backward + Strafe Right + Turn No Action
34                ELSE:
35                  IF [Wall] at (1,7) <= 0.41:
36                    THEN Move Backward + Strafe Left + Turn Left
37                  ELSE:
38                    THEN Move Backward + Strafe Left + Turn Left
39              ELSE:
40                IF [Wall] at (0,5) <= 0.03:
41                  IF [Wall] at (4,5) <= 0.03:
42                    THEN Move Backward + Strafe Right + Turn Left
43                  ELSE:
44                    THEN Move Backward + Strafe Left + Turn Left

```

```

45     ELSE:
46         IF [Wall] at (1,4) <= 0.47:
47             THEN Move Backward + Strafe Right + Turn Left
48         ELSE:
49             THEN Move Backward + Strafe Right + Turn Left
50     ELSE:
51         IF [Wall] at (1,5) <= 0.16:
52             IF [Wall] at (1,7) <= 0.03:
53                 IF [Wall] at (1,1) <= 0.03:
54                     IF [Wall] at (0,4) <= 0.22:
55                         THEN Move No Action + Strafe Right + Turn No Action
56                     ELSE:
57                         THEN Move Backward + Strafe Right + Turn No Action
58                 ELSE:
59                     IF [Wall] at (0,3) <= 0.28:
60                         THEN Move No Action + Strafe Left + Turn Left
61                     ELSE:
62                         THEN Move Backward + Strafe Right + Turn Left
63             ELSE:
64                 IF [BadFood] at (2,5) <= 0.28:
65                     IF [Wall] at (3,9) <= 0.34:
66                         THEN Move Backward + Strafe Right + Turn Left
67                     ELSE:
68                         THEN Move No Action + Strafe Right + Turn No Action
69                 ELSE:
70                     IF [Wall] at (1,5) <= 0.09:
71                         THEN Move Forward + Strafe Right + Turn Right
72                     ELSE:
73                         THEN Move Forward + Strafe Right + Turn Right
74             ELSE:
75                 IF [Wall] at (2,4) <= 0.03:
76                     IF [Wall] at (6,5) <= 0.34:
77                         IF [Wall] at (0,3) <= 0.47:
78                             THEN Move Forward + Strafe Right + Turn Right
79                         ELSE:
80                             THEN Move Backward + Strafe Right + Turn Right
81                     ELSE:
82                         IF [Wall] at (9,6) <= 0.16:
83                             THEN Move No Action + Strafe Right + Turn No Action
84                         ELSE:
85                             THEN Move Forward + Strafe Right + Turn Right
86                 ELSE:
87                     IF [Wall] at (5,2) <= 0.09:
88                         IF [Wall] at (9,2) <= 0.47:
89                             THEN Move No Action + Strafe Right + Turn No Action
90                         ELSE:
91                             THEN Move Forward + Strafe Right + Turn Right

```

```

92     ELSE:
93         IF [Wall] at (1,4) <= 0.22:
94             THEN Move Backward + Strafe Left + Turn Left
95         ELSE:
96             THEN Move No Action + Strafe Left + Turn Left
97 ELSE:
98     IF [Wall] at (1,6) <= 0.03:
99     IF [Wall] at (1,5) <= 0.03:
100    IF [Wall] at (0,5) <= 0.09:
101    IF [Wall] at (2,9) <= 0.34:
102        IF [Food] at (4,8) <= 0.03:
103            THEN Move Backward + Strafe Right + Turn No Action
104        ELSE:
105            THEN Move Backward + Strafe Right + Turn Right
106    ELSE:
107        IF [Wall] at (0,7) <= 0.53:
108            THEN Move Forward + Strafe Right + Turn Right
109        ELSE:
110            THEN Move Forward + Strafe Right + Turn Right
111 ELSE:
112    IF [Wall] at (0,1) <= 0.16:
113    IF [Wall] at (2,4) <= 0.06:
114        THEN Move Backward + Strafe Right + Turn No Action
115    ELSE:
116        THEN Move Backward + Strafe Right + Turn Left
117 ELSE:
118    IF [Wall] at (0,4) <= 0.34:
119        THEN Move Forward + Strafe Right + Turn Right
120    ELSE:
121        THEN Move Backward + Strafe Right + Turn Right
122 ELSE:
123    IF [Wall] at (0,5) <= 0.41:
124    IF [Wall] at (9,5) <= 0.03:
125    IF [Agent] at (9,7) <= 0.03:
126        THEN Move Backward + Strafe No Action + Turn Left
127    ELSE:
128        THEN Move No Action + Strafe Right + Turn Right
129 ELSE:
130    IF [Wall] at (0,6) <= 0.59:
131        THEN Move Backward + Strafe Right + Turn Left
132    ELSE:
133        THEN Move Backward + Strafe Right + Turn Right
134 ELSE:
135    IF [Wall] at (1,2) <= 0.34:
136    IF [Wall] at (9,5) <= 0.47:
137        THEN Move Backward + Strafe Right + Turn Right
138    ELSE:

```

```

139         THEN Move Backward + Strafe No Action + Turn Left
140     ELSE:
141         IF [Food] at (3,7) <= 0.12:
142             THEN Move Backward + Strafe Left + Turn Left
143         ELSE:
144             THEN Move Backward + Strafe Right + Turn Right
145     ELSE:
146         IF [Wall] at (0,4) <= 0.09:
147             IF [Wall] at (3,8) <= 0.03:
148                 IF [Wall] at (3,5) <= 0.03:
149                     IF [Wall] at (2,7) <= 0.03:
150                         THEN Move Backward + Strafe Right + Turn Left
151                     ELSE:
152                         THEN Move Backward + Strafe Right + Turn Left
153                 ELSE:
154                     IF [Wall] at (0,5) <= 0.03:
155                         THEN Move Backward + Strafe No Action + Turn Left
156                     ELSE:
157                         THEN Move Backward + Strafe Right + Turn Left
158             ELSE:
159                 IF [Wall] at (0,7) <= 0.03:
160                     IF [Wall] at (0,6) <= 0.59:
161                         THEN Move Backward + Strafe Left + Turn Left
162                     ELSE:
163                         THEN Move Backward + Strafe Left + Turn Left
164                 ELSE:
165                     IF [Wall] at (5,9) <= 0.44:
166                         THEN Move Backward + Strafe Right + Turn Left
167                     ELSE:
168                         THEN Move Backward + Strafe Right + Turn No Action
169             ELSE:
170                 IF [Wall] at (0,2) <= 0.41:
171                     IF [Wall] at (2,7) <= 0.03:
172                         IF [Wall] at (2,3) <= 0.28:
173                             THEN Move Backward + Strafe Right + Turn No Action
174                         ELSE:
175                             THEN Move Backward + Strafe Right + Turn Left
176                     ELSE:
177                         IF [Wall] at (1,6) <= 0.22:
178                             THEN Move Backward + Strafe Right + Turn Left
179                         ELSE:
180                             THEN Move Backward + Strafe Left + Turn Left
181                 ELSE:
182                     IF [Wall] at (6,7) <= 0.03:
183                         IF [Wall] at (0,3) <= 0.47:
184                             THEN Move Backward + Strafe Right + Turn Left
185                     ELSE:

```

```

186         THEN Move Backward + Strafe Right + Turn Right
187     ELSE:
188         IF [Wall] at (6,8) <= 0.22:
189             THEN Move Backward + Strafe Right + Turn Left
190         ELSE:
191             THEN Move Backward + Strafe Right + Turn Left
192 ELSE:
193     IF [Wall] at (1,6) <= 0.09:
194         IF [Wall] at (0,8) <= 0.34:
195             IF [Wall] at (1,9) <= 0.03:
196                 IF [Wall] at (5,5) <= 0.16:
197                     IF [Food] at (1,8) <= 0.03:
198                         IF [Wall] at (0,9) <= 0.41:
199                             THEN Move No Action + Strafe Right + Turn No Action
200                         ELSE:
201                             THEN Move No Action + Strafe Right + Turn Right
202                     ELSE:
203                         IF [Wall] at (1,7) <= 0.19:
204                             THEN Move Forward + Strafe Right + Turn Right
205                         ELSE:
206                             THEN Move Forward + Strafe Right + Turn Right
207                     ELSE:
208                         IF [BadFood] at (7,3) <= 0.09:
209                             IF [Wall] at (4,2) <= 0.50:
210                                 THEN Move Backward + Strafe Right + Turn Left
211                             ELSE:
212                                 THEN Move Backward + Strafe No Action + Turn No Action
213                         ELSE:
214                             IF [BadFood] at (1,8) <= 0.06:
215                                 THEN Move Forward + Strafe Right + Turn Right
216                             ELSE:
217                                 THEN Move Forward + Strafe Right + Turn Right
218                     ELSE:
219                         IF [Wall] at (9,5) <= 0.22:
220                             IF [Wall] at (4,5) <= 0.22:
221                                 IF [Wall] at (7,4) <= 0.53:
222                                     THEN Move Forward + Strafe Right + Turn Right
223                                 ELSE:
224                                     THEN Move No Action + Strafe Right + Turn Left
225                             ELSE:
226                                 IF [Wall] at (7,2) <= 0.09:
227                                     THEN Move Forward + Strafe Right + Turn No Action
228                                 ELSE:
229                                     THEN Move No Action + Strafe Left + Turn Right
230                         ELSE:
231                             IF [Wall] at (0,8) <= 0.03:
232                                 IF [Wall] at (8,1) <= 0.12:

```

```

233         THEN Move Forward + Strafe Left + Turn Right
234     ELSE:
235         THEN Move Backward + Strafe Left + Turn Left
236     ELSE:
237         IF [Wall] at (9,4) <= 0.09:
238             THEN Move Forward + Strafe Right + Turn Right
239         ELSE:
240             THEN Move No Action + Strafe Left + Turn No Action
241     ELSE:
242         IF [Wall] at (0,4) <= 0.34:
243             IF [Wall] at (1,9) <= 0.03:
244                 IF [Wall] at (0,9) <= 0.47:
245                     IF [Wall] at (6,1) <= 0.53:
246                         THEN Move Backward + Strafe Right + Turn Left
247                     ELSE:
248                         THEN Move No Action + Strafe Right + Turn Right
249                 ELSE:
250                     IF [Wall] at (1,5) <= 0.09:
251                         THEN Move Backward + Strafe Right + Turn Right
252                     ELSE:
253                         THEN Move Backward + Strafe Right + Turn No Action
254                 ELSE:
255                     IF [Wall] at (1,3) <= 0.16:
256                         IF [Wall] at (1,9) <= 0.22:
257                             THEN Move Forward + Strafe Right + Turn Right
258                         ELSE:
259                             THEN Move Forward + Strafe Right + Turn Right
260                     ELSE:
261                         IF [BadFood] at (4,7) <= 0.06:
262                             THEN Move Forward + Strafe Left + Turn Left
263                         ELSE:
264                             THEN Move Forward + Strafe Right + Turn No Action
265                 ELSE:
266                     IF [Wall] at (0,1) <= 0.47:
267                         IF [Wall] at (6,2) <= 0.47:
268                             IF [BadFood] at (3,5) <= 0.03:
269                                 THEN Move Backward + Strafe Right + Turn No Action
270                             ELSE:
271                                 THEN Move Backward + Strafe Right + Turn Left
272                         ELSE:
273                             IF [BadFood] at (9,3) <= 0.16:
274                                 THEN Move Forward + Strafe Right + Turn Left
275                             ELSE:
276                                 THEN Move No Action + Strafe Right + Turn Right
277                     ELSE:
278                         IF [BadFood] at (0,4) <= 0.16:
279                             IF [BadFood] at (0,6) <= 0.34:

```

```

280         THEN Move Forward + Strafe Right + Turn Right
281     ELSE:
282         THEN Move Backward + Strafe Right + Turn Left
283     ELSE:
284         IF [BadFood] at (1,1) <= 0.06:
285             THEN Move Backward + Strafe Right + Turn Left
286         ELSE:
287             THEN Move Forward + Strafe Right + Turn Left
288     ELSE:
289         IF [Wall] at (1,4) <= 0.34:
290             IF [Wall] at (1,7) <= 0.16:
291                 IF [Wall] at (1,9) <= 0.03:
292                     IF [Wall] at (5,2) <= 0.03:
293                         IF [BadFood] at (1,9) <= 0.09:
294                             THEN Move Backward + Strafe Right + Turn Left
295                         ELSE:
296                             THEN Move Forward + Strafe Right + Turn Right
297                     ELSE:
298                         IF [Wall] at (1,5) <= 0.25:
299                             THEN Move Backward + Strafe Right + Turn Left
300                         ELSE:
301                             THEN Move Backward + Strafe Right + Turn No Action
302                 ELSE:
303                     IF [Wall] at (1,6) <= 0.53:
304                         IF [Wall] at (4,1) <= 0.28:
305                             THEN Move Backward + Strafe Right + Turn Left
306                         ELSE:
307                             THEN Move Forward + Strafe Right + Turn No Action
308                     ELSE:
309                         IF [Wall] at (8,0) <= 0.62:
310                             THEN Move Backward + Strafe Right + Turn Right
311                         ELSE:
312                             THEN Move Backward + Strafe Right + Turn No Action
313                 ELSE:
314                     IF [Wall] at (2,4) <= 0.53:
315                         IF [Food] at (7,6) <= 0.09:
316                             IF [Wall] at (5,0) <= 0.03:
317                                 THEN Move Backward + Strafe No Action + Turn Left
318                             ELSE:
319                                 THEN Move Backward + Strafe Right + Turn Left
320                         ELSE:
321                             IF [Wall] at (2,5) <= 0.44:
322                                 THEN Move Forward + Strafe Right + Turn Right
323                             ELSE:
324                                 THEN Move No Action + Strafe Right + Turn Right
325                     ELSE:
326                         IF [BadFood] at (6,7) <= 0.12:

```

```

327         IF [Wall] at (8,5) <= 0.25:
328             THEN Move Backward + Strafe Left + Turn Left
329         ELSE:
330             THEN Move Backward + Strafe Left + Turn Left
331         ELSE:
332             THEN Move Backward + Strafe Right + Turn Right
333     ELSE:
334         IF [Wall] at (0,7) <= 0.34:
335             IF [Wall] at (0,8) <= 0.34:
336                 IF [FrozenAgent] at (6,4) <= 0.09:
337                     IF [BadFood] at (9,0) <= 0.06:
338                         THEN Move Backward + Strafe Left + Turn Left
339                     ELSE:
340                         THEN Move Backward + Strafe Left + Turn Right
341                 ELSE:
342                     THEN Move Backward + Strafe Left + Turn Right
343             ELSE:
344                 IF [Wall] at (2,3) <= 0.50:
345                     IF [BadFood] at (4,6) <= 0.22:
346                         THEN Move Backward + Strafe Left + Turn Left
347                     ELSE:
348                         THEN Move Backward + Strafe Left + Turn Right
349                 ELSE:
350                     THEN Move No Action + Strafe Left + Turn Right
351         ELSE:
352             IF [Wall] at (2,2) <= 0.22:
353                 IF [Wall] at (1,7) <= 0.09:
354                     IF [FrozenAgent] at (5,5) <= 0.06:
355                         THEN Move Backward + Strafe No Action + Turn Left
356                     ELSE:
357                         THEN Move Backward + Strafe Right + Turn Left
358                 ELSE:
359                     IF [BadFood] at (2,1) <= 0.09:
360                         THEN Move Backward + Strafe Left + Turn Left
361                     ELSE:
362                         THEN Move No Action + Strafe Right + Turn Left
363             ELSE:
364                 IF [Food] at (5,9) <= 0.09:
365                     IF [BadFood] at (2,0) <= 0.06:
366                         THEN Move Backward + Strafe Right + Turn Left
367                     ELSE:
368                         THEN Move Backward + Strafe Right + Turn Right
369                 ELSE:
370                     THEN Move Forward + Strafe Left + Turn Right

```

Appendix C SHAP Calculation Figures

Below is a display of all the figures produced as a result of the SHAP calculations carried out in this dissertation.

C.1 Pyramids

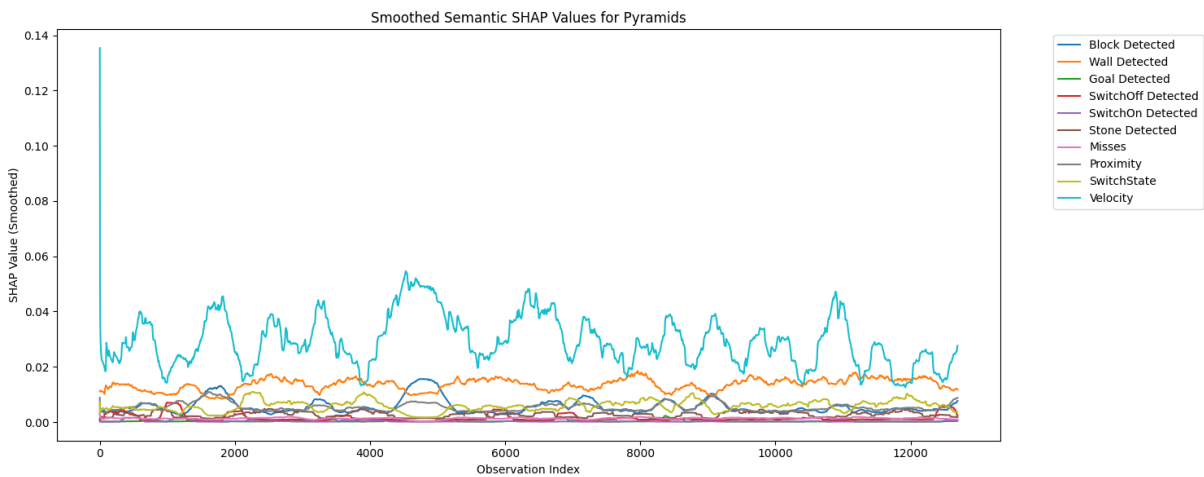


Figure C.1 Smoothed Semantic SHAP Values over Time for Pyramids Environment

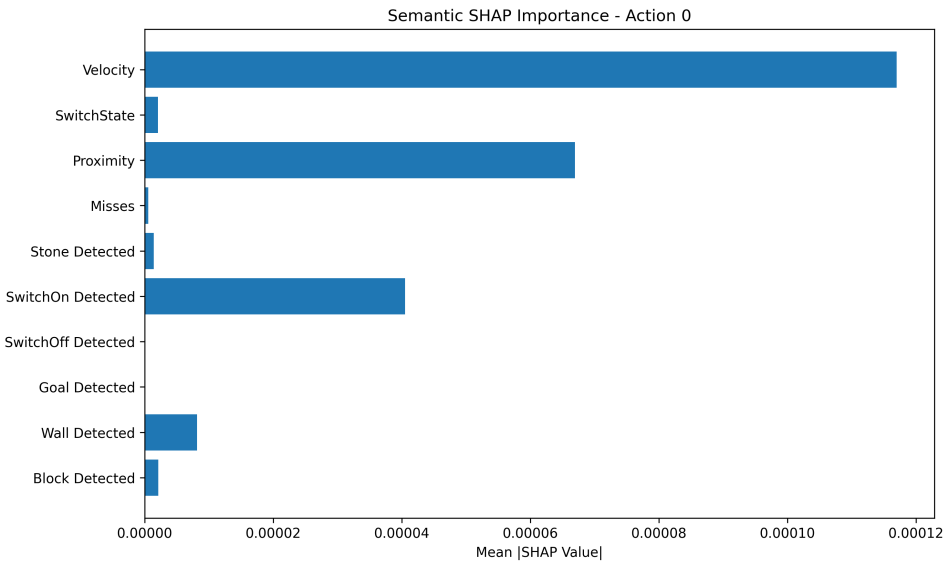


Figure C.2 Semantic SHAP Importance for Action 0 (Pyramids)

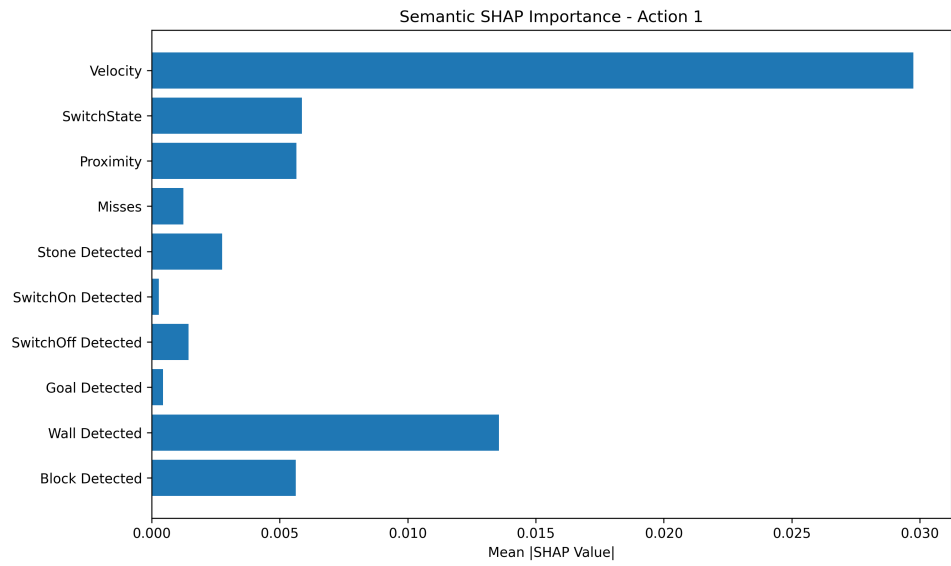


Figure C.3 Semantic SHAP Importance for Action 1 (Pyramids)

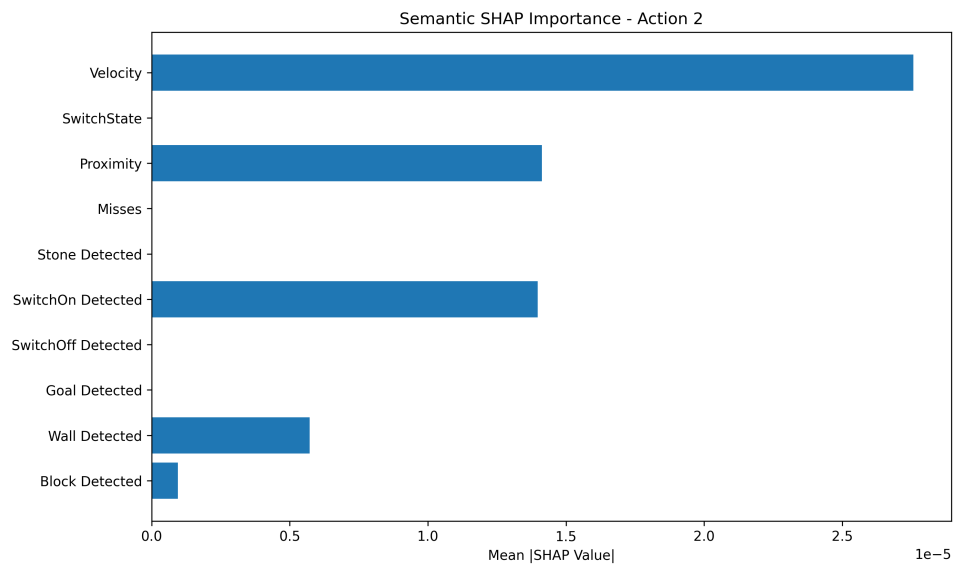


Figure C.4 Semantic SHAP Importance for Action 2 (Pyramids)

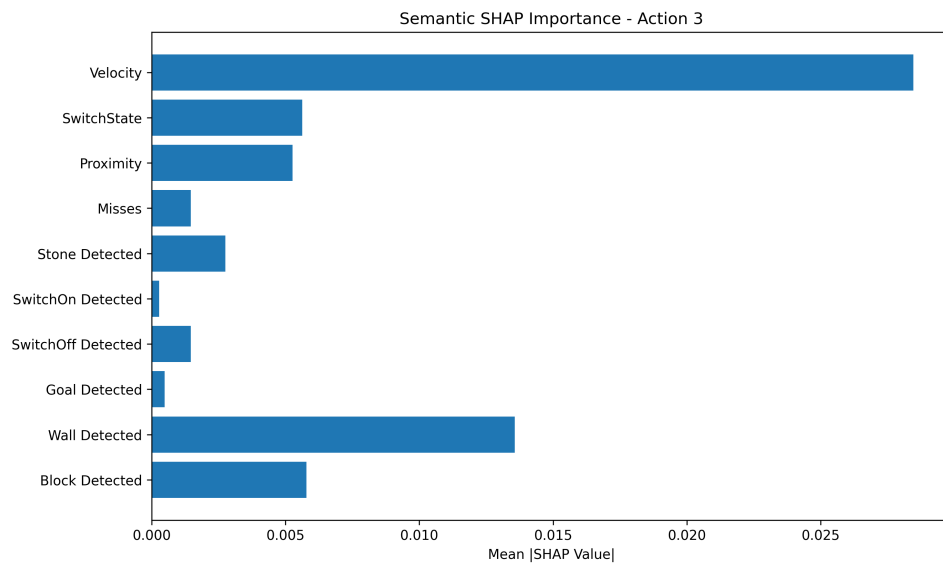


Figure C.5 Semantic SHAP Importance for Action 3 (Pyramids)

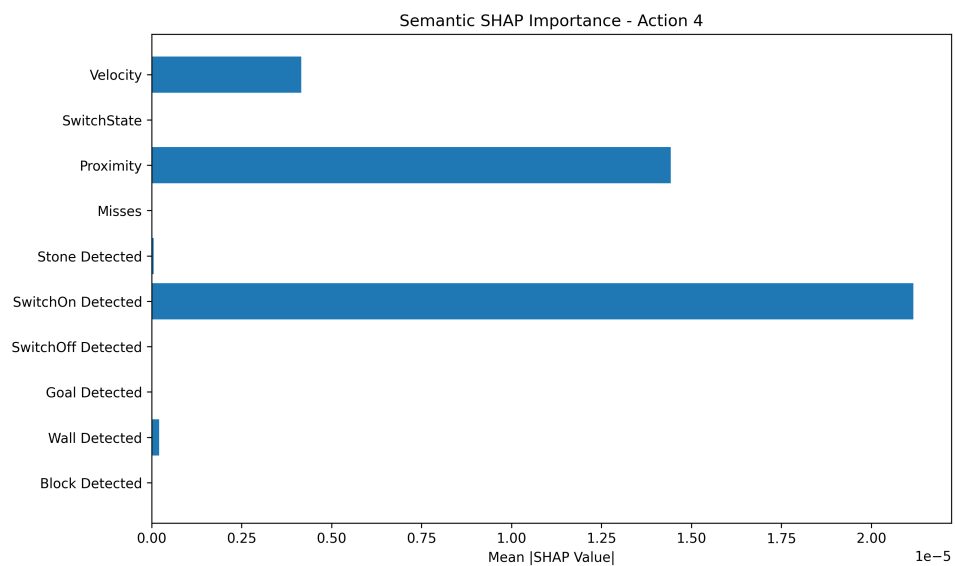


Figure C.6 Semantic SHAP Importance for Action 4 (Pyramids)

C.2 Dungeon Escape

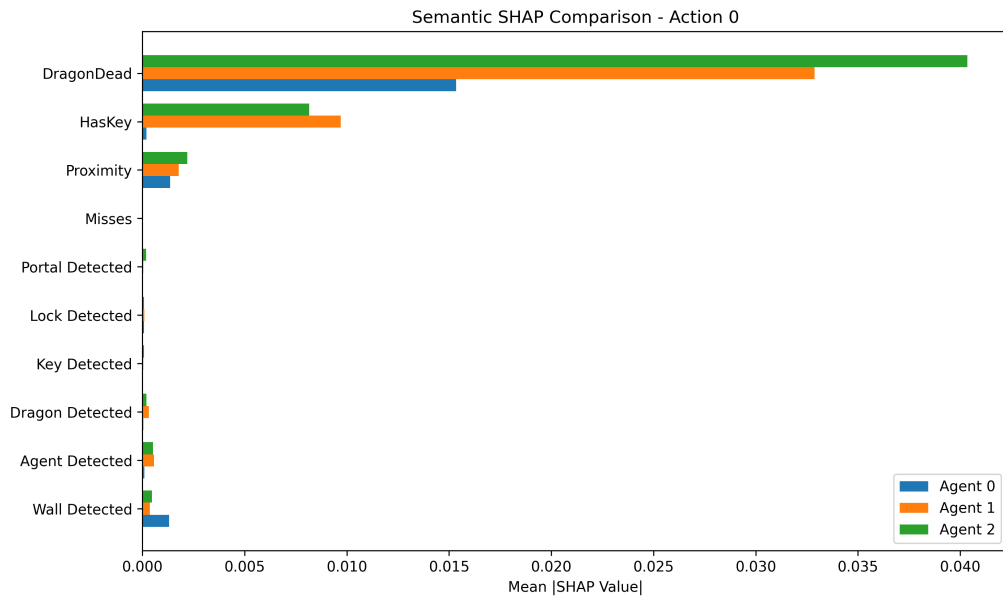


Figure C.7 SHAP comparison between agents - Action 0 (Dungeon)

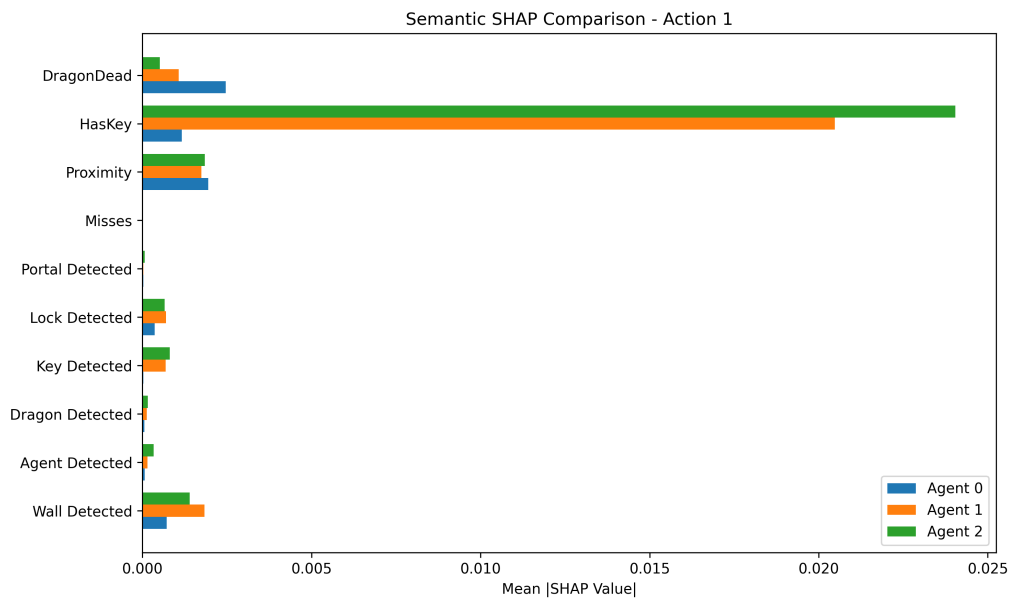


Figure C.8 SHAP comparison between agents - Action 1 (Dungeon)

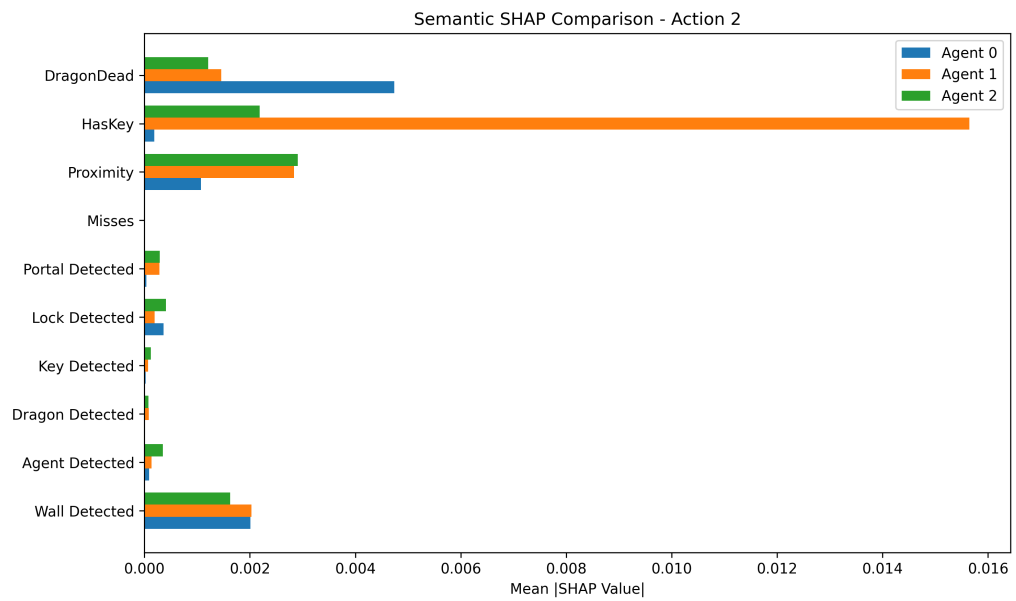


Figure C.9 SHAP comparison between agents - Action 2 (Dungeon)

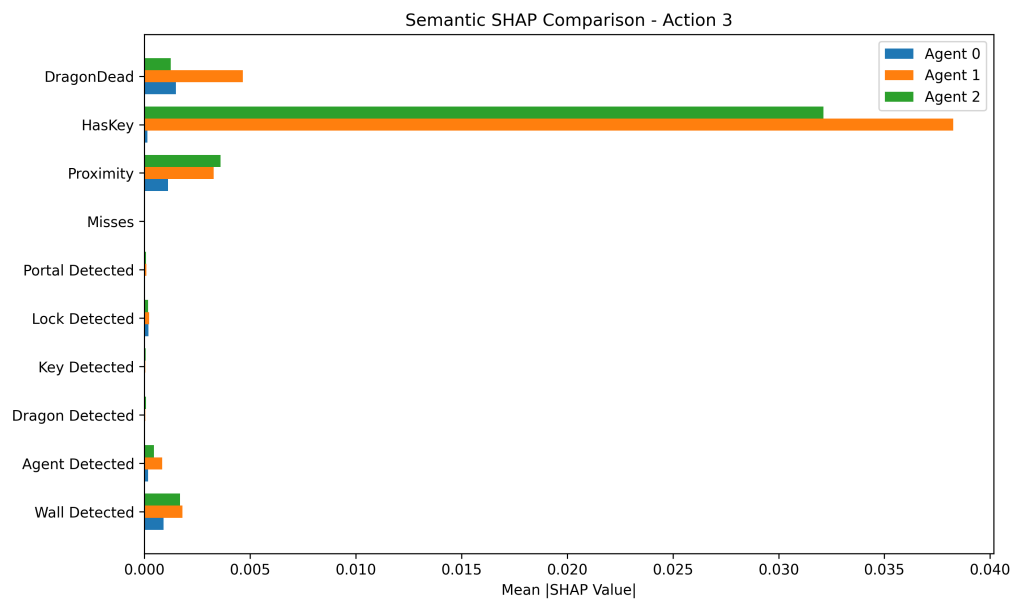


Figure C.10 SHAP comparison between agents - Action 3 (Dungeon)

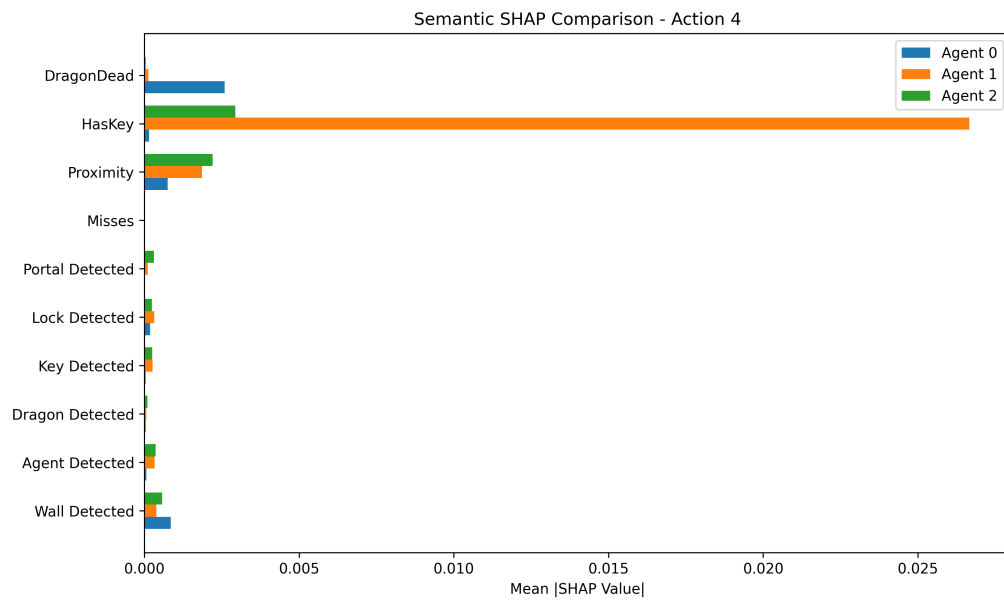


Figure C.11 SHAP comparison between agents - Action 4 (Dungeon)

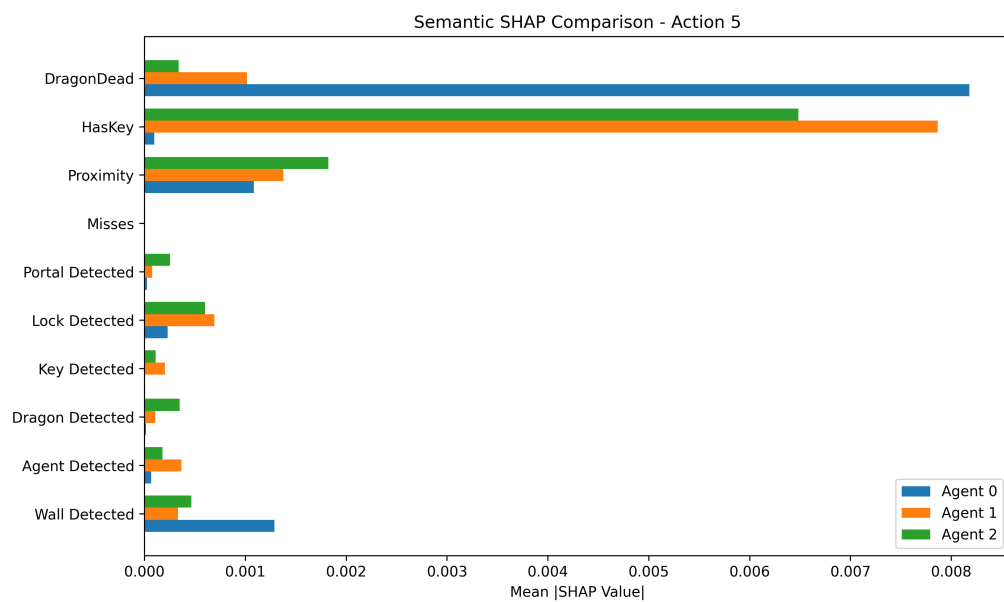


Figure C.12 SHAP comparison between agents - Action 5 (Dungeon)

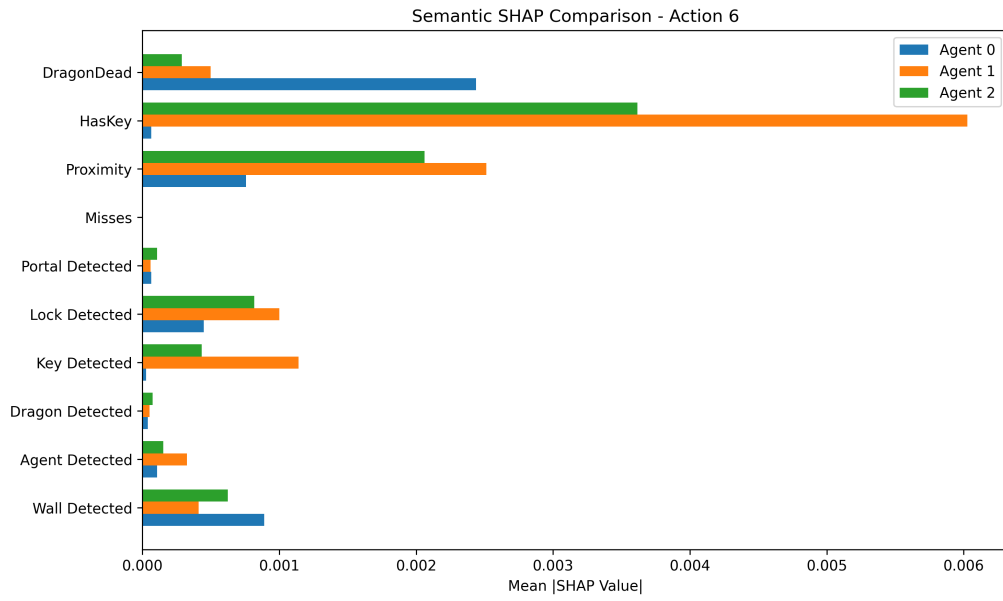


Figure C.13 SHAP comparison between agents - Action 6 (Dungeon)

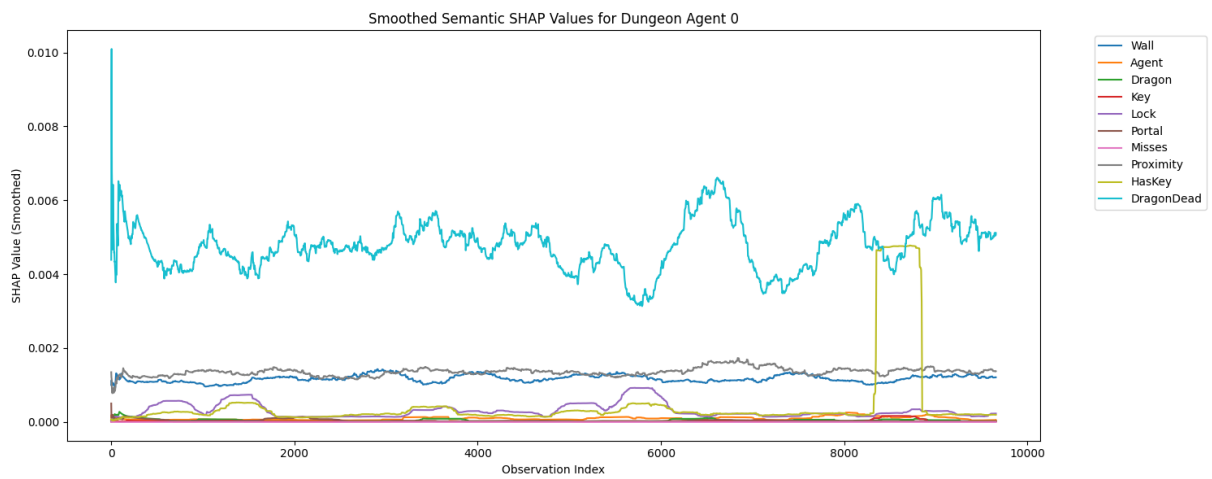


Figure C.14 Smoothed SHAP values for Dungeon Agent 0

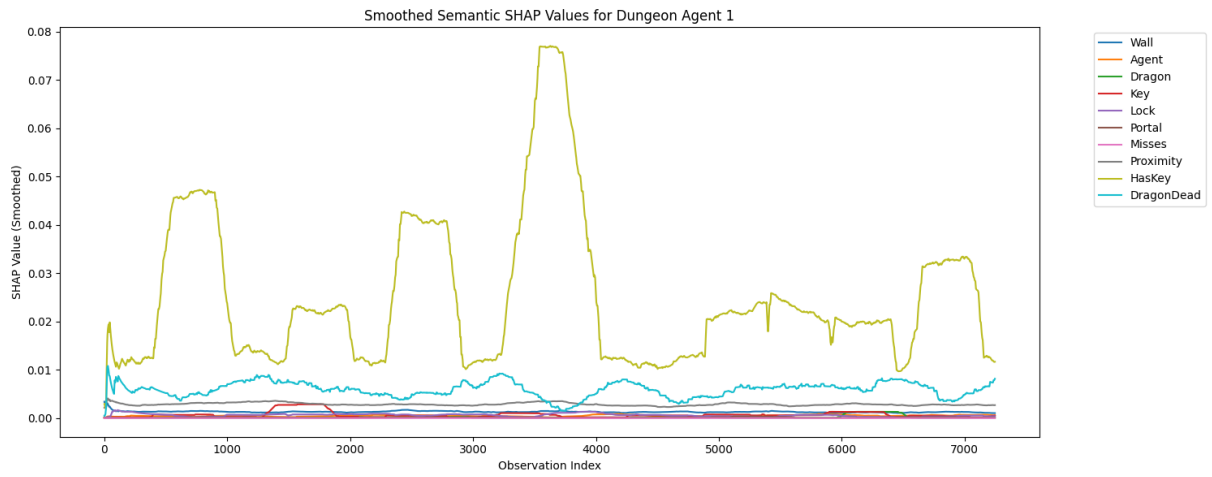


Figure C.15 Smoothed SHAP values for Dungeon Agent 1

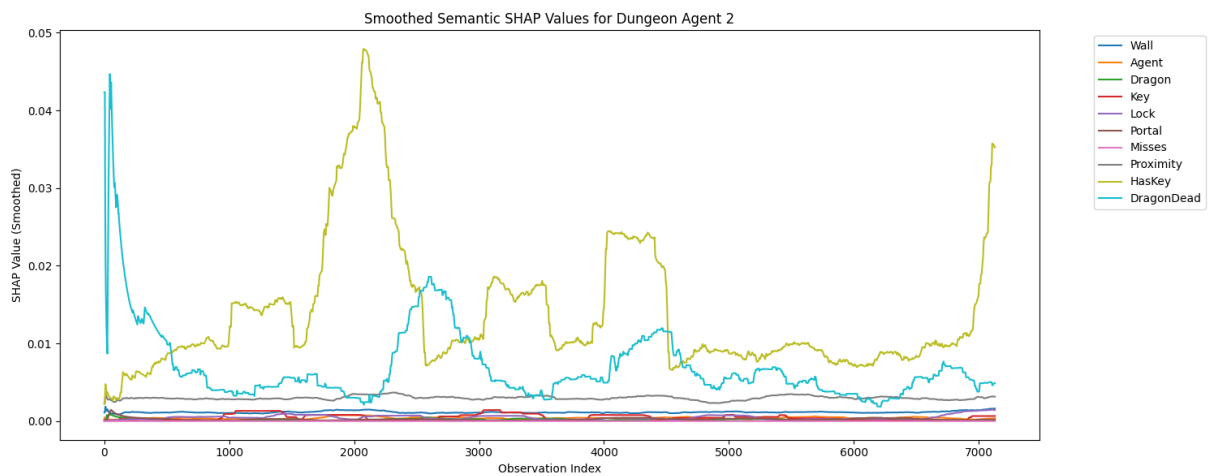
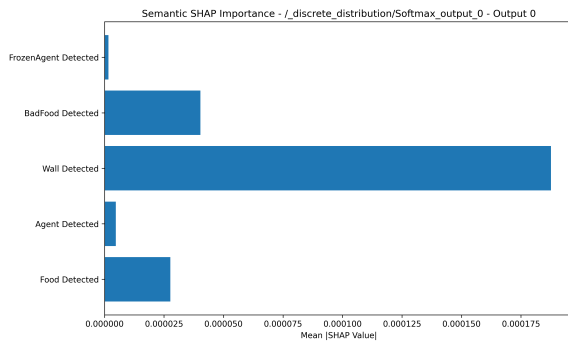
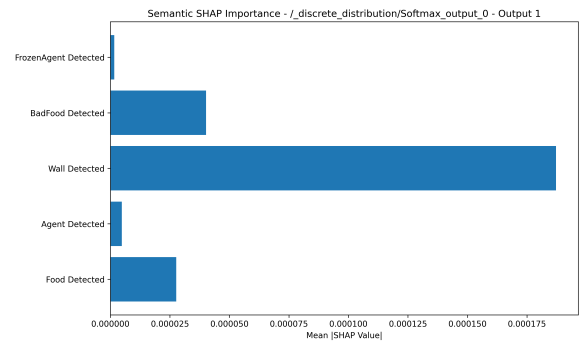


Figure C.16 Smoothed SHAP values for Dungeon Agent 2

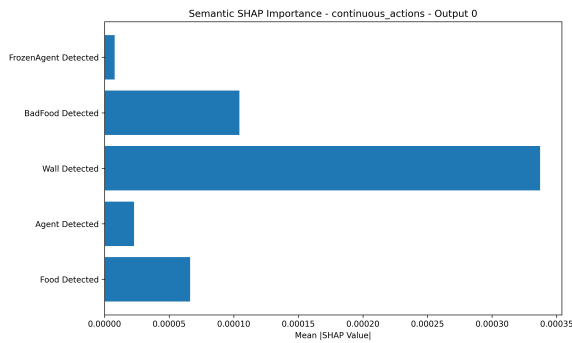
C.3 Food Collector



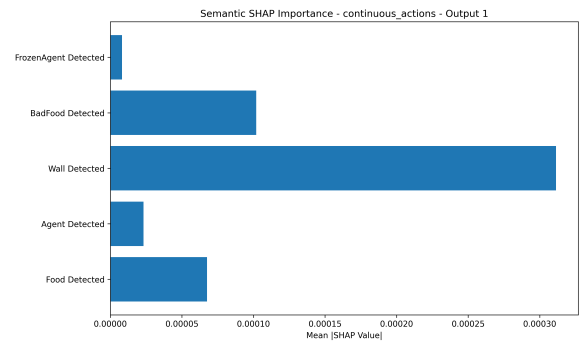
(a) Discrete Action 0



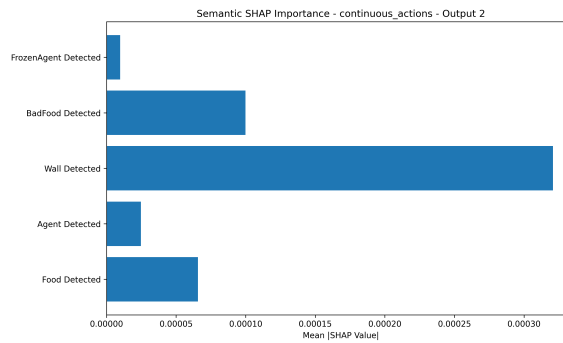
(b) Discrete Action 1



(c) Continuous Action 0



(d) Continuous Action 1



(e) Continuous Action 2

Figure C.17 Semantic SHAP Importance Across Actions in the Food Environment

Appendix D Food Collector Saliency Grid Results

Below are the full saliency map calculations from the Food Collector environment:



Figure D.1 Continuous Action Saliency Maps



Figure D.2 Discrete Action Saliency Maps

Appendix E Dungeon Environment Real-Time SHAP Visualisation

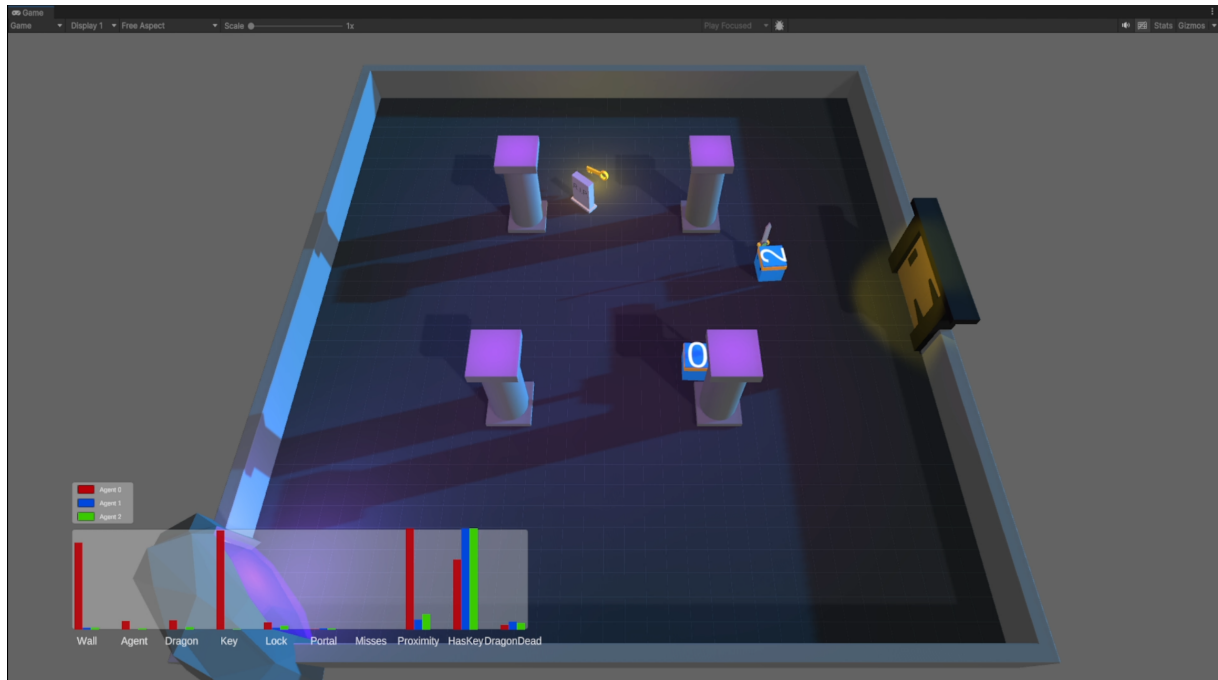


Figure E.1 Dungeon Escape SHAP Visualisation

Appendix F Code Availability

The code and data required to reproduce this project can be found at the following GitHub Repository: <https://github.com/luke-camillleri-22/FYP>

Please take note of the setup instructions inside of the repository's README.

Unity may be a very sensitive tool, therefore keep in mind that your success may vary with in-engine results.